EF CORE PLUGIN DEVELOPMENT · REFERENCE AND PRACTICE

# The EF Core Plugin Handbook

From the options extension to the NuGet package: everything you need for professional and even very complex extensions of Entity Framework Core. Explained throughout with a complete multi-tenancy plugin for PostgreSQL and SQL Server.


| | |
|---|---|
| **Edition** | 1.2 |
| **Versions** | EF Core 10, Npgsql.EntityFrameworkCore.PostgreSQL 10, Microsoft.EntityFrameworkCore.SqlServer 10, .NET 10 |
| **Sample** | Complete solution at **github.com/guenni81/EF-Core-Plugin-Handbook**: core package, relational package, provider packages for PostgreSQL, SQL Server and Cosmos DB, a sample package for custom SQL functions, two sample apps, unit and integration tests |
| **Verified** | The sample code is built against EF Core 10 and Npgsql 10; unit and integration tests run against real PostgreSQL and SQL Server. |
| **Structure** | 11 parts, 48 chapters, 6 appendices (A–F) |
| **License** | MIT License, © 2026 guenni81. Free to use, copy, modify and share, including commercially; see the last page. |


## About this book

Many EF Core extensions start out as a handful of extension methods and then grow without a plan. If you understand how EF Core is built internally, however, you can write extensions that feel like part of EF Core itself: they are configured through the options, run inside the internal service provider, translate custom functions to SQL, generate migrations, write readable snapshots, and combine cleanly with other plugins.

This book explains each of these building blocks in two layers. First comes the **principle**: how does the mechanism work in EF Core, which variants exist, and what do you need to watch out for? Then comes the **implementation** in the sample plugin, with the code that is also in the repository. The chapters build on each other, but can also be read individually as a reference. If you come with a concrete problem and are looking for the right mechanism, start with the decision guide in chapter 4. The code in the repository and the snippets in this edition use English comments and messages. Each snippet's label names the file.

| Box | Meaning |
|---|---|
| **Important** | Pitfalls that lead to incorrect behavior or security holes. |
| **Tip** | Proven practice that saves work or improves quality. |
| **Note** | Background, context and references to other chapters. |
| **What happens if …?** | A plausible mistake: what you observe, why it happens, how to do it right, and which test guards against it. The wrong code stands right before it. |

> **Public and internal APIs**
>
> Some APIs that plugins need live in namespaces ending in `.Internal`. EF Core calls them “pubternal”: publicly visible, but without compatibility guarantees. The book marks these places, and chapter 27 explains how to deal with them.

# Contents


## I Foundations 5

1 What is an EF Core plugin? 6

2 EF Core’s architecture from a plugin’s point of view 8

3 The sample scenario 12

4 Choosing the right extension point 15

## II Infrastructure 24

5 The options extension 25

6 Registering services 29

7 Public API and integration into the application 36

8 State, lifetimes and thread safety 39

## III The model 44

9 Annotations and Fluent API 45

10 Conventions 49

11 Model validation 57

12 Type mappings and value converters 58

## IV Queries 61

13 The query pipeline 62

14 Translating methods and members 67

15 Preventing client-side evaluation 71

16 Further query extension points 73

17 Materialization: from rows to objects 74

## V Run time 81

18 Interceptors 82

19 Diagnostics and logging 90

**VI Migrations and design time 94**

20 How migrations work 95

21 Declarative SQL from annotations 96

22 Custom migration operations 99

23 Design-time services 101

24 Compiled models, trimming and NativeAOT 104

25 Scaffolding (reverse engineering) 107

**VII Quality and delivery 108**

26 Test strategy 109

27 Versioning and compatibility 114

28 Packaging and documentation 122

29 The checklist from A to Z 123

**VIII Advanced extension points 128**

30 Bulk operations and the limits of interceptors 129

31 Value generation 132

32 The update pipeline 134

33 Complex types and JSON 137

34 Models per runtime context 139

35 Advanced migrations 140

**IX Operation and integration 142**

36 Row Level Security in production 143

37 Tenant-safe relationships 149

38 Retrofitting and operating multi-tenancy 152

39 Plugin performance 157

40 Using the plugin in ASP.NET Core 162

41 CI and API stability 166

**X Deep extensions 168**

42 Custom SQL expression nodes 169

43 Aggregate translators 176

44 Property-level Fluent API and runtime annotations 178

45 Scaffolding in practice 181

**XI Multiple providers 183**

46 What belongs in the core 184

47 A second provider: SQL Server 186

48 Without SQL: what carries over to Cosmos DB 192

**XII Appendices 196**

A Extension points and lifetimes 197

B Files of the sample plugin 202

C Glossary 205

D Debugging toolbox 206

E Exercises with solutions 208

F Sources 210

License 213

Part I

# Foundations

What a plugin is, how EF Core is built internally, which scenario accompanies us through the book, and which extension point fits which problem.


› Chapter 1: What is an EF Core plugin?

› Chapter 2: EF Core’s architecture from a plugin’s point of view

› Chapter 3: The sample scenario

› Chapter 4: Choosing the right extension point

# 1 What is an EF Core plugin?

Version dependency: **stable**

The term “plugin” is used for very different things. For this book, it helps to distinguish four levels that differ in how deeply they reach into EF Core:

| Level | Description | Examples |
|---|---|---|
| 1 · Helpers | Extension methods on `DbContext` or `IQueryable` that only call public API. | Paging helpers, `WhereIf`, seeding methods |
| 2 · Model helpers | Fluent API extensions and annotations, without services of their own inside EF Core. | Custom `HasXyz()` methods, ModelBuilder helpers |
| 3 · Real plugins | Own services in the internal service provider: conventions, query translation, type mappings, migrations, design time. | NetTopologySuite plugin, NodaTime plugin, EFCore.NamingConventions |
| 4 · Providers | A complete database integration including connection, update pipeline and SQL generation. | Npgsql, Pomelo (MySQL), SQLite, Cosmos (chapter 48 builds a level 3 plugin on it) |

This book covers **level 3** completely. Levels 1 and 2 need no book of their own: they only use EF’s public API, and chapter 9 covers what is specific to Fluent API extensions. The tools of level 4 (such as replacing the SQL generator) are explained where plugins need them.

## Who this book is for

For developers who already use EF Core confidently: `DbContext`, migrations, LINQ queries and dependency injection are assumed. No knowledge of EF’s internals is required; the book introduces them where they are needed. Experience with PostgreSQL helps with the sample, but every SQL construct is explained when it first appears. Chapter 2 ends with a complete mini plugin that shows all building blocks on a single page before the large sample begins.

## Two reading paths

The book is a textbook and a reference at the same time. Two paths lead through it on purpose; both end with a result that can be checked:

| | My first plugin | Reference for advanced readers |
|---|---|---|
| Prerequisites | EF Core as a user: `DbContext`, migrations, LINQ, dependency injection | The first path or an own plugin in production |
| Chapters | 1, 2 (with the mini plugin), 3, 5, 6, 7, 10, 14, 18, 26, 28 | 4 and appendix A as the entry, then as needed 8, 13, 17, 20, 27, 36–38 and parts X and XI |
| Result | An own plugin with an options extension, a convention, a translator and an interceptor that builds as a package | A reasoned choice of an extension point: lifetime, composability, API status and upgrade risk |

| | My first plugin | Reference for advanced readers |
|---|---|---|
| Checkable by | `MinimalPluginTests` as a template for the own tests, plus the consumer test (page 105) applied to the own package | The compatibility matrix in chapter 6, the check of the lifetimes against EF's service tables and the upgrade checklist |

The exercises in appendix E belong to both paths: each one ties in with a test of the sample that checks the solution.

## What a level 3 plugin can do

A complete plugin can extend EF Core at practically every point: it can make new CLR types available as column types, translate custom functions in LINQ to SQL, adapt the model automatically through conventions, accept metadata through the Fluent API and attributes, validate the model, intervene when saving and when opening connections, log its own events, add custom SQL to migrations and extend the `dotnet ef` tooling. The sample in this book uses every one of these possibilities.

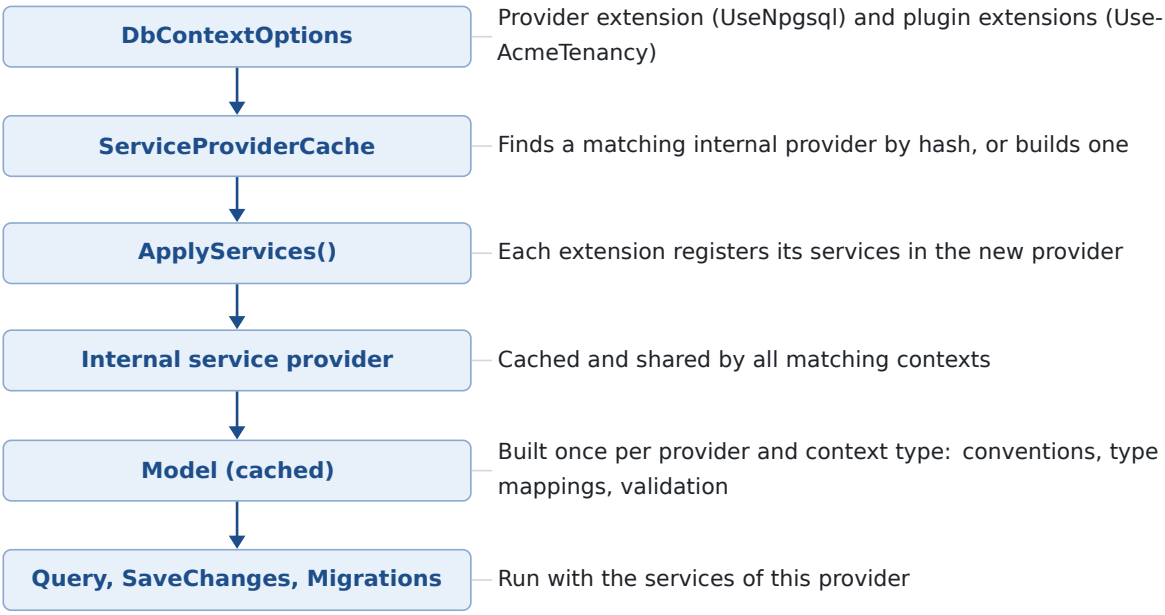
> **Role models in the ecosystem**
>
> Before writing your own, look at existing plugins. **EFCore.NamingConventions** is a small, clean convention plugin. The **NetTopologySuite** plugin in the EF Core repository shows type mappings and translations. Npgsql's **NodaTime** plugin shows a provider-specific plugin activated through the provider's options builder.

# 2 EF Core’s architecture from a plugin’s point of view

Version dependency: **medium**

## Two containers: the internal service provider

Every `DbContext` resolves its infrastructure from an **internal** `IServiceProvider`. It is separate from the application’s DI container and contains several hundred services: type mapping, model building, the query pipeline, change tracking, the update pipeline, migrations. A plugin hooks in by registering additional services in this container or replacing existing ones. The book also calls it the *internal provider* for short; that never means the database provider such as Npgsql.




Two caches are decisive here. The **service provider** is cached based on the options: contexts with the same configuration share one provider. The **model** is cached per provider (and context type). Both make EF Core fast, but they have direct consequences for plugins, which chapter 5 covers in detail.

The model exists in two forms. The **design-time model** contains all annotations and is the basis for migrations and snapshots. At run time, `context.Model` returns a lean, read-only **runtime model** that keeps only the annotations needed at run time. Which ones these are and how a plugin controls them is shown in chapters 21 and 44; but already in part III, plugin code that reads at run time sees only the runtime model.

## The path through the book

The parts of the book follow the path of a request through this architecture: options and internal service provider (part II), model (part III), query and materialization (part IV), interceptors and diagnostics (part V), migrations and design time (part VI). In this order, the update pipeline of `Save`

`Changes` comes after the interceptors; because plugins rarely extend it directly, it sits with the advanced extension points (chapter 32). Operating against the database follows in part IX.

## Lifetimes

Every EF service has a fixed lifetime. **Singleton** here means: once per internal service provider, not per application. **Scoped** means: once per context instance. A singleton must not hold a scoped service, and a service that needs different data per context must not be a singleton. Appendix A lists the lifetimes of all extension points; chapter 8 summarizes which state belongs where.

## The dependencies pattern

EF services do not receive their dependencies individually, but as a parameter object, such as `MigrationsSqlGeneratorDependencies` or `ProviderConventionSetBuilderDependencies`. When you derive from an EF service, you accept this object in the constructor and pass it on. EF Core adds new dependencies to the object without breaking constructors. For plugins this is a blessing: derived services stay compatible across minor versions.

`Migrations/Internal/TenancyMigrationsSqlGenerator.cs (header)`

```
public class TenancyMigrationsSqlGenerator : NpgsqlMigrationsSqlGenerator
{
    public TenancyMigrationsSqlGenerator(
        MigrationsSqlGeneratorDependencies dependencies,     // parameter object from EF
        INpgsqlSingletonOptions npgsqlSingletonOptions)      // additionally from the provider
        : base(dependencies, npgsqlSingletonOptions)
    {
    }
```

> **Code style in the sample**
>
> All classes in the sample use classic constructors with `private readonly` fields and `ArgumentNullException.ThrowIfNull`, no primary constructors. Their parameters are mutable throughout the class body, offer no natural place for validation and hide which state is stored. Classic constructors also match the style of EF Core and Npgsql, whose classes a plugin derives from. The only exception is positional records such as `RlsPolicy`, where the syntax generates real `init` properties.

## Extend or replace?

There are two fundamentally different ways to extend EF Core. **Plugin interfaces** such as `IMethodCallTranslatorPlugin` or `IConventionSetPlugin` are designed for exactly this. They can be registered multiple times, EF queries all registrations, and several plugins coexist. **Replacing** means swapping a single service such as `IMigrationsSqlGenerator` for a derived class. That is sometimes unavoidable, but only one replacement can win per context.

> **Plugin interfaces before replacement**
>
> Always check first whether a plugin interface exists. Replacing a single service collides with every other plugin that replaces the same service and forces you to derive from the provider's implementation. That ties the plugin to one provider and its internal API. Chapter 6 shows how to stay composable anyway.

Chapter 4 classifies every individual extension point: whether it is a plugin interface or a single service, and which simpler alternative to consider first.

## Warm-up: a complete plugin in five pieces

Before the large sample, here is the smallest plugin that is still complete. It makes PostgreSQL's `reverse()` available in LINQ as `EF.Functions.Reverse(...)`. Every later chapter elaborates one of its five pieces:

tests/Acme.EntityFrameworkCore.Tenancy.Tests/MinimalPluginTests.cs (1/2, shortened)

```
// 1. The function users call in LINQ; it is never executed
public static class ReverseDbFunctionsExtensions
{
    public static string Reverse(this DbFunctions _, string value)
        => throw new InvalidOperationException("Reverse can only be used in LINQ queries.");
}

// 2. The translator turns the method call into a SQL function
//    (chapter "Translating methods and members")
public class ReverseTranslator : IMethodCallTranslator
{
    // Constructor stores the ISqlExpressionFactory in _sql

    public SqlExpression? Translate(
        SqlExpression? instance, MethodInfo method, IReadOnlyList<SqlExpression> arguments,
        IDiagnosticsLogger<DbLoggerCategory.Query> logger)
        => method == ReverseMethod
            ? _sql.Function("reverse", [arguments[1]], nullable: true,
                argumentsPropagateNullability: [true], typeof(string),
                arguments[1].TypeMapping)
            : null;
}

// 3. The plugin hands the translator to EF (chapter "Translating methods and members")
public class ReverseTranslatorPlugin : IMethodCallTranslatorPlugin
{
    public ReverseTranslatorPlugin(ISqlExpressionFactory sql)
    {
        Translators = [new ReverseTranslator(sql)];
    }

    public IEnumerable<IMethodCallTranslator> Translators { get; }
}
```

tests/Acme.EntityFrameworkCore.Tenancy.Tests/MinimalPluginTests.cs (2/2, shortened)

```
// 4. The options extension registers the plugin in EF's internal provider
//    (chapters "The options extension" and "Registering services")
public class ReverseOptionsExtension : IDbContextOptionsExtension
{
    private DbContextOptionsExtensionInfo? _info;

    public DbContextOptionsExtensionInfo Info => _info ??= new ExtensionInfo(this);

    public void ApplyServices(IServiceCollection services)
        => new EntityFrameworkRelationalServicesBuilder(services)
```

```
            .TryAdd<IMethodCallTranslatorPlugin, ReverseTranslatorPlugin>();

    public void Validate(IDbContextOptions options)
    {
    }

    // ExtensionInfo: not a provider, no settings -> hash 0, always the same provider
}

// 5. The Use method activates the plugin
//    (chapter "Public API and integration into the application")
public static DbContextOptionsBuilder UseReverse(this DbContextOptionsBuilder builder)
{
    ((IDbContextOptionsBuilderInfrastructure)builder).AddOrUpdateExtension(
        builder.Options.FindExtension<ReverseOptionsExtension>()
        ?? new ReverseOptionsExtension());
    return builder;
}
```

Usage

```
options.UseNpgsql(connectionString).UseReverse();

db.Customers.Where(c => EF.Functions.Reverse(c.Name) == "abc");
// SQL: WHERE reverse(c."Name") = 'abc'
```

That is all a working plugin needs. EF finds the translator because the extension registers it in the internal service provider, and every context configured with `UseReverse()` shares that provider. Because the argument references a column, the function needs no evaluatable expression filter (chapter 15 explains when it does). The test `MinimalPluginTests` checks the translation and that the same query fails without the plugin. The tenancy plugin consists of exactly these pieces, only many more of them.

# 3 The sample scenario

Version dependency: **stable**

The sample plugin **Acme.EntityFrameworkCore.Tenancy** makes EF Core multi-tenant with PostgreSQL. The database itself enforces tenant separation through **Row Level Security** (RLS): every table with tenant data gets a policy that only shows rows of the tenant whose ID is set in the current database session. Mappings in which such a table would lack the tenant column are rejected by the plugin while the model is built (page 52). Even a faulty LINQ query or raw SQL therefore only sees the rows of the tenant that is set. Whoever can run arbitrary SQL under the application role, however, can also change the session setting; chapter 36 deals with this limit.

From the user's point of view it looks like this:

User's view

```
public class Order : ITenantOwned               // marker from the core package
{
    public int Id { get; set; }
    public TenantId TenantId { get; set; }      // strongly typed, automatically uuid
    public decimal Total { get; set; }
}

services.AddAcmeTenancy<HttpTenantProvider>();
services.AddDbContext<ShopContext>((sp, o) => o
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp, tenancy => tenancy.ApplyQueryFilter()));

// Querying and saving without any tenant logic in application code
db.Orders.Add(new Order { Total = 42 });         // the plugin sets TenantId
var mine = await db.Orders.ToListAsync();        // RLS filters in the database
```

To make this work, the plugin uses almost every extension point covered in this book; the others have examples of their own, marked as sample package or teaching example in the table:

| Requirement | Implementation | Ch. |
|---|---|---|
| Configuration via `UseAcmeTenancy()` | Options extension, settings record, options builder | 5–7 |
| `TenantId` as a column type without configuration | Type mapping plugin, value converter, JSON reader/writer | 12 |
| `EF.Functions.CurrentTenant()` in LINQ | Method translator, evaluatable expression filter | 14–15 |
| `tenantId.IsEmpty` in LINQ | Member translator | 14 |
| RLS for all tables of `ITenantOwned` entities | Convention with index and optional query filter | 10 |
| Fluent API, attribute `[NoTenantIsolation]` | Annotations, attribute convention, ConfigurationSource | 9–10 |
| Detect misconfiguration early | Validation convention, custom warnings | 11, 19 |
| Set TenantId automatically | SaveChanges interceptor | 18 |
| Tenant in the DB session | Connection interceptor | 18 |
| `ENABLE ROW LEVEL SECURITY`, `CREATE POLICY` | Annotation provider, migrations SQL generator | 20–21 |

| Requirement | Implementation | Ch. |
|---|---|---|
| Roles and grants in migrations | Custom migration operations | 22 |
| Readable snapshots | Annotation code generator, design-time services | 23 |
| Compiled models and AOT | `Default` instances, source generators | 24 |
| `EF.Functions.IsNotDistinctFrom`, `EF.Functions.Every` | Custom SQL node, aggregate translator; a separate sample package without tenancy | 42–43 |
| `now()` in `UPDATE` | Update SQL generator; teaching example in the test project | 32 |
| Tenant property with any name | `IsTenantProperty()`, runtime annotation | 44 |
| Scaffold RLS from an existing database | Custom database model factory | 45 |
| SQL Server | Second provider package on the same core | 46–47 |

## Package architecture

At its heart the plugin consists of four NuGet packages. The **core package** contains everything that is independent of the database and that assemblies with entity classes may reference without pulling in a provider; it works with any EF Core provider. The **relational package** adds what every SQL database needs: the session interceptor on the connection and the check of the table mappings. The **PostgreSQL package** is the complete implementation this book uses to explain everything. The **SQL Server package** uses the same core and relational layer and shows in chapter 47 what a second provider has to implement itself. A fifth package, `Acme.EntityFrameworkCore.Tenancy.Cosmos`, builds on the core alone and shows what carries over to a database without SQL (chapter 48). Besides these, the small package `Acme.EntityFrameworkCore.Functions.PostgreSQL` holds the examples for custom SQL nodes and aggregates; they have nothing to do with multi-tenancy (chapter 42).

Structure of the four packages

```
Acme.EntityFrameworkCore.Tenancy              Core (EF Core only, any provider)
├─ TenantId, TenantIdConverter, TenantIdJsonReaderWriter, TenancyException
├─ ITenantOwned, ITenantProvider, NoTenantIsolationAttribute
├─ EF.Functions.CurrentTenant(), TenancyDbFunctionsAttribute
├─ Metadata/         runtime annotation, IsTenantProperty(), candidate lookup
├─ Conventions/      value generation for TenantId keys
├─ ValueGeneration/  TenantIdValueGenerator and factory
├─ Query/            evaluatable expression filter (for all packages)
├─ Interceptors/     SaveChanges, materialization and bulk update interceptors
├─ Diagnostics/      event IDs, logging definitions, logger extensions
├─ Testing/          test mode for SQLite and InMemory
└─ DependencyInjection/  AddAcmeTenancy<T>()

Acme.EntityFrameworkCore.Tenancy.Relational    relational layer (EF Core relational)
├─ Interceptors/     abstract session interceptor (DbConnection), tracing
├─ Metadata/         TenantTableMappingValidator (TPT, TPC, entity splitting, ...)
└─ Database.ApplyTenantSession()

Acme.EntityFrameworkCore.Tenancy.PostgreSQL    complete implementation
├─ Infrastructure/   options extension, builder, singleton options
├─ Storage/          TenantIdTypeMapping, type mapping plugin
├─ Query/            translators (CurrentTenant, TenantId members)
├─ Metadata/         RLS annotations, RlsPolicy, annotation provider
├─ Builders/         Fluent API (HasRowLevelSecurity, HasPolicy, ...)
```

```
├─ Conventions/      isolation, attribute, validation, convention set plugin
├─ Migrations/       SQL generator, custom operations
├─ Design/           code generators, design-time services
├─ Scaffolding/      read RLS and policies from the database
└─ Runtime/          session interceptors (connection, transaction pooling)

Acme.EntityFrameworkCore.Tenancy.SqlServer     second provider
                                               (chapter "A second provider: SQL Server")
└─ Infrastructure/, Storage/, Query/, Metadata/, Conventions/, Migrations/, Runtime/
```

**Tip**

The folder structure follows EF Core's own (Infrastructure, Storage, Query, Metadata, Migrations, Design). Anyone who knows EF Core finds their way immediately, and when looking things up in the EF source you know where the counterpart lives.

# 4 Choosing the right extension point

Version dependency: **stable**

The parts of this book follow EF Core's architecture: infrastructure, model, queries, run time, migrations. A concrete problem rarely does. "Every row belongs to a tenant" can be solved with a query filter, a SaveChanges interceptor, a convention or with policies from migrations, and each of these solutions has a different price and different gaps. The sample plugin from chapter 3 therefore combines several of them.

This chapter starts from the problem and leads to the right mechanism. It is a map, not a reference: how a mechanism works is explained in the chapter it names. Appendix A goes the other way: from the mechanism to lifetime, cardinality, API status, registration and composability. On a first reading, skim the chapter and come back with a concrete problem.

## The lowest depth of intervention

The most important rule is the same as in "Extend or replace?" (chapter 2), only broader: **choose the lowest depth of intervention that solves the problem completely.** Each step in the following table costs more compatibility than the one before:

| Depth | Means | Price |
|---|---|---|
| Application | `OnModelCreating`, `ConfigureConventions`, `HasDbFunction`, `migrationBuilder.Sql()`, overriding `SaveChanges` | No plugin; every application repeats the code |
| Metadata | Annotations and Fluent API extensions | Only take effect once a service reads them |
| Plugin interface | Conventions, translator, type mapping and filter plugins, interceptors | Can be registered multiple times, coexists with other plugins |
| Decorating | Wrapping a single service and delegating to it | Only if the public interface suffices; the activation order matters |
| Replacing | Replacing a single service with a class derived from the provider's | One winner per context; ties the plugin to the provider and often to internal API |
| Deep intervention | Custom SQL nodes, custom differ, postprocessor | Several single services at once; hardly composable with similar plugins |

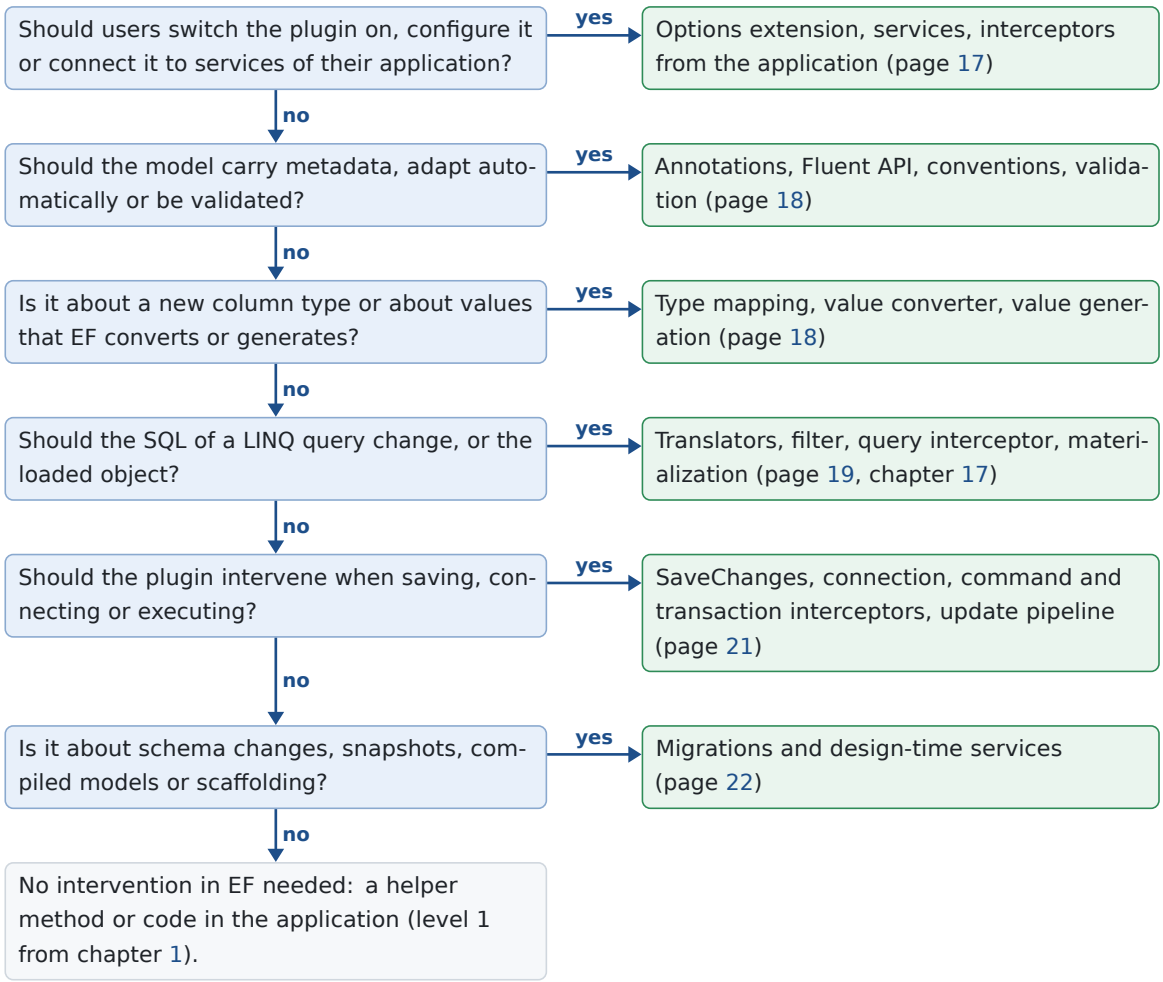
"Completely" is meant literally. A query filter is simpler than row level security, but it does not protect against raw SQL (chapter 30). The simpler solution only wins if it gives the same guarantee. Five questions settle whether it does:

| Question | Why it matters |
|---|---|
| When does the mechanism run? | During model building once per internal provider, during translation once per query shape, during execution every time, at design time only in `dotnet ef`. This determines which state it may use (chapter 39). |

| Question | Why it matters |
|---|---|
| Is there a simpler alternative? | The lower depth wins if it gives the same guarantee. |
| Public or internal? | Internal API (EF1001) can change in any minor version (chapter 27). |
| Multiple or single? | Services that can be registered multiple times coexist with other plugins; for a single service exactly one wins. |
| Which lifetime? | A singleton may neither hold scoped services nor per-context data; a scoped service may do both (chapter 8). |

## The responsible area

The first step is almost always the question *where* the problem lies. The following decision tree leads to the responsible area; each area then has its own section with a table.



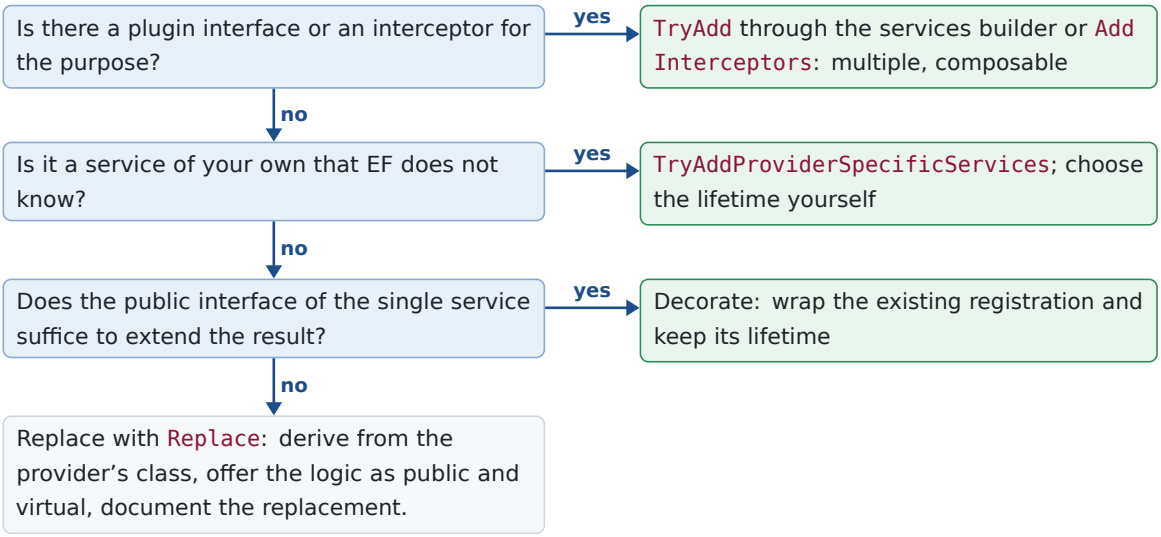

Each row of the area tables describes a problem, the mechanism for it, the simpler alternative to check first, and the typical wrong choice. Below the mechanism, its properties are shown in grey:

| Label | Meaning |
|---|---|
| public | Gets by with EF Core's public API. |
| provider base | Derives from the provider's implementation. Ties the plugin to that provider; with Npgsql, the base classes or their dependencies usually live in `.Internal` namespaces (EF1001). |

| Label | Meaning |
|---|---|
| pubternal | Even EF's own type lives in an `.Internal` namespace. |
| multiple, single | Cardinality according to EF Core 10's service tables (appendix A). |
| no service | Works through the model or the options, not through the internal service provider. |
| Singleton, Scoped | Lifetime in the internal provider; with "design time" in the service provider of `dotnet ef` (chapter 23). |

## Configuration and services

Every level 3 plugin starts with an options extension. The actual decision is made for each individual service: how does it get into the internal provider? The tree follows the order from "Extend or replace?" (chapter 2):




| Problem and mechanism | Check first | Typical wrong choice | Ch. |
|---|---|---|---|
| **The plugin should be switched on and configured.** `IDbContextOptionsExtension`, `Use` method, options builder<br>public · no service, part of the options | Does the plugin need services of its own? Pure model configuration gets by with a Fluent API extension (level 2). | Putting values that change per context (tenant, user) into the options or the hash; changing the extension after it has been added. | 5, 7 |
| **A singleton service needs the settings.** `ISingletonOptions`<br>public · multiple · Singleton | Scoped services can inject `IDbContextOptions` directly. | Registering it only under its own interface: then EF never initializes the instance. | 6 |
| **The plugin brings services of its own.** `ApplyServices` with the services builder<br>public · lifetime from EF's table or chosen yourself | The decision tree above: plugin interface before replacement. | Registering EF services with `AddSingleton` and the wrong lifetime; putting plugin services into the application container instead of the internal provider. | 6 |
| **A single service should do something extra.** Decorator around the registered implementation<br>public · single · lifetime of the original | Is there a plugin interface for the same purpose? | `Replace` although the public interface suffices; activating the decorator before the provider. | 6 |

| Problem and mechanism | Check first | Typical wrong choice | Ch. |
|---|---|---|---|
| **The plugin needs services of the application (tenant, user, clock).** Interceptor instances from the application container via `AddInterceptors`<br>public · multiple · Scoped or Singleton | Is the value fixed at configuration time? Then an option suffices. | Trying to resolve application services in the internal provider; scoped interceptors with DbContext pooling, which keep the tenant of the first request. | 7, 40 |

## Model, types and values

Everything in this area is decided during model building, that is once per internal provider and context type, and ends up in the cached model. Value converters and value generators do run on every read, write or add, but as the model has configured them. Run-time values therefore have no place in this configuration.

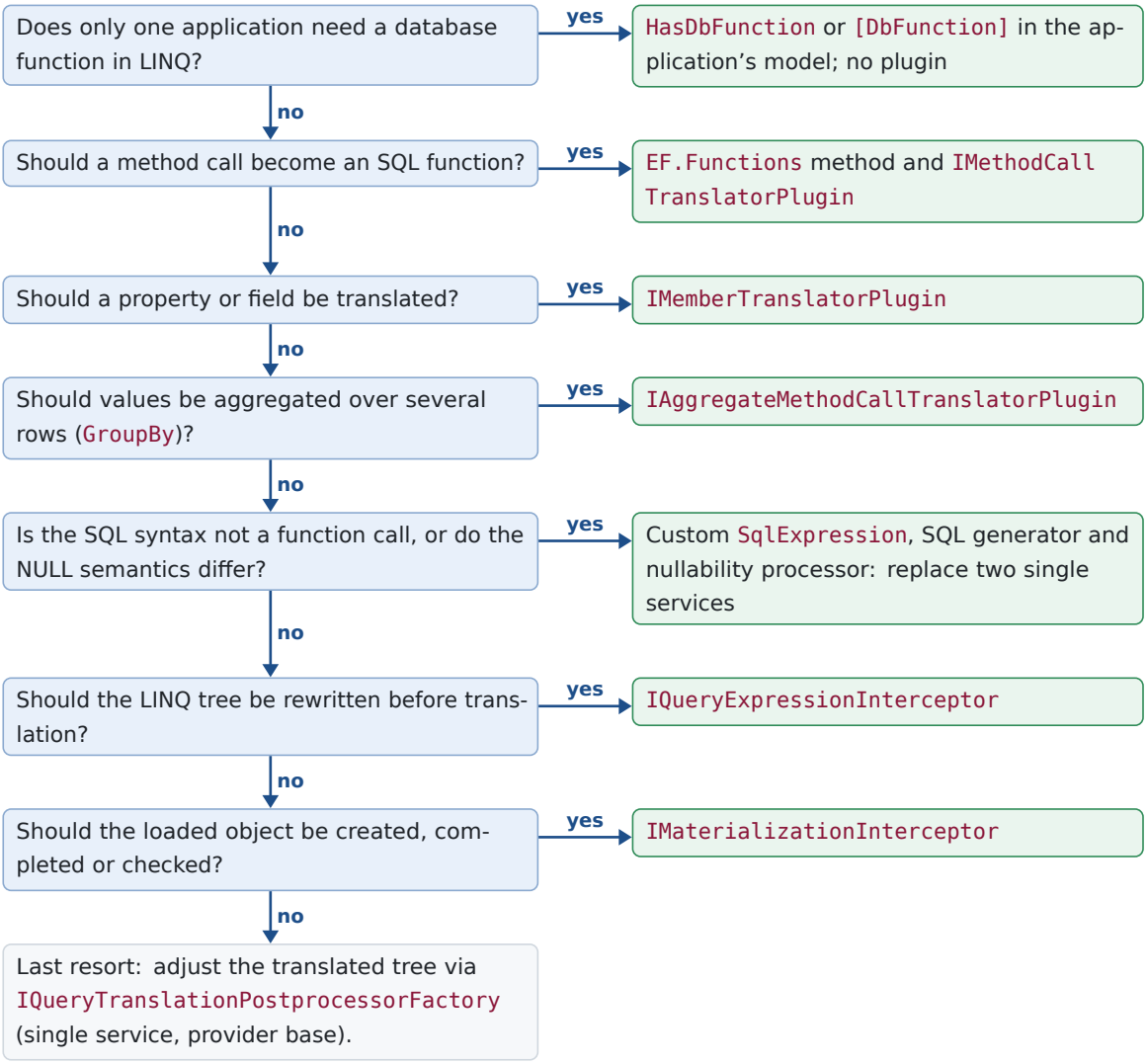
| Problem and mechanism | Check first | Typical wrong choice | Ch. |
|---|---|---|---|
| **The plugin needs to store configuration in the model.** Annotations with their own prefix<br>public · no service | Does existing configuration of EF or the provider cover the need (comment, check constraint, index method)? | Storing objects instead of simple values; passing default values on to the relational model, which creates empty migrations; dropping an explicit default, which a convention then overrides. | 9 |
| **Users should configure the plugin per entity or property.** Fluent API extension, convention builder overload, attribute with attribute convention<br>public · no service | Does a global setting in the options builder suffice? | Putting the logic into the generic instead of the non-generic variant (the snapshot uses the latter); writing to the metadata in conventions instead of through the convention builder, which alone respects stronger configuration sources. | 9, 10, 44 |
| **The model should adapt automatically.** Convention via `IConventionSetPlugin`<br>public · multiple · Scoped | Does it affect only one application? Then `OnModelCreating` or `ConfigureConventions` suffice. | A reactive convention that depends on names or base types instead of `ModelFinalizing`; replacing `IModelCustomizer` (single, singleton). | 10 |
| **Misconfiguration should be detected early.** `IModelFinalizedConvention`, warnings with custom event IDs<br>public · via the convention set plugin | Can the Fluent API already check the input (like the policy builder)? | Replacing `IModelValidator` (single, singleton, provider base); reporting questionable configuration as an exception instead of a warning. | 11, 19 |
| **The model depends on run-time data (schema per tenant).** `IModelCacheKeyFactory`<br>public · single · Singleton | Can the value be evaluated at run time in SQL (parameter, session setting)? Then a single model remains. | Putting run-time data into the options hash (one internal provider per value); unchecked input as the key. | 34 |

| Problem and mechanism | Check first | Typical wrong choice | Ch. |
|---|---|---|---|
| **Run-time code needs information from the model.** Annotation with a simple value that stays in the runtime model (runtime annotation in this book's sense); for derived, cached values EF's runtime annotation API (`GetOrAdd RuntimeAnnotationValue`)<br>public · no service | Nothing else: EF copies simple values into the runtime and the compiled model by itself. | Writing a custom code generator "so that the annotation survives"; recomputing derived values on every access. | 44 |
| **A custom CLR type should work as a column without configuration.** `IRelational TypeMappingSourcePlugin` with a custom `RelationalType Mapping`<br>public · multiple · Singleton | Does a value converter suffice that users set with `HasConversion` or `ConfigureConventions`? | Replacing `IRelational TypeMappingSource` (single, provider base); forgetting `Clone` or the static `Default` property; forgetting the `Json ValueReaderWriter`, without which JSON columns fail during model building. | 12, 33 |
| **Values should be converted when reading and writing.** `ValueConverter`<br>public · no service | Should the type work everywhere, or have its own literals or column type? Then the converter belongs in a type mapping (row above). | Expecting members of the model type to be translated in queries although the database only knows the provider value; a new converter instance per mapping instead of a shared one. | 12, 24 |
| **EF should generate values for new entities.** `Value Generator` with a factory, set by a convention via `HasValue GeneratorFactory(Type)`<br>public · no service | Can the database generate the value (identity, sequence, `Has DefaultValueSql`)? | `HasValueGenerator` instead of the factory variant (compiled models fail); replacing `IValue GeneratorSelector` (single, scoped, provider base). | 31 |

## Queries

At which stage of the pipeline a desired change sits is shown by the map in chapter 13; this section then chooses the mechanism. Most query extensions run once per query shape, and the result is cached. Values that change per execution therefore belong in the SQL as a parameter or database function, never as a constant. The following tree is ordered by increasing depth of intervention; one question concerns every custom function and therefore comes first:

> **Methods without data reference**
>
> If a custom method has no argument that refers to a column (`CurrentTenant()`, session or time functions), it needs an `IEvaluatableExpressionFilterPlugin` in addition to the translator. Otherwise EF evaluates it in .NET beforehand.




| Problem and mechanism | Check first | Typical wrong choice | Ch. |
|---|---|---|---|
| **A database function should be callable in LINQ.** `EF.Functions` method and `IMethodCallTranslatorPlugin`<br>public · multiple · Scoped | `HasDbFunction` in the application, if needed with `HasTranslation`. | Throwing an exception for foreign methods instead of returning `null`; inserting run-time values as constants; declaring the wrong nullability. | 14 |
| **A property or field of a custom type should be translated.** `IMemberTranslatorPlugin`<br>public · multiple · Scoped | Can the value be mapped as a regular or computed column? | Creating constants without the column's type mapping, which produces a wrong literal. | 14 |
| **The function has no argument with a data reference.** `IEvaluatableExpressionFilterPlugin` in addition to the translator<br>public · multiple · Singleton | Does the method have a column argument? Then the plugin is not needed. | Forgetting the plugin: EF executes the method in .NET, and the exception comes from its body. | 15 |

| Problem and mechanism | Check first | Typical wrong choice | Ch. |
|---|---|---|---|
| **An aggregate function should be available after `GroupBy`.** `IAggregateMethodCallTranslatorPlugin`<br>public · multiple · Scoped | Can the result be built from existing aggregates (`Count` with a condition, `Min`, `Max`)? | Treating empty sets and NULL values differently from the .NET side. | 43 |
| **The SQL syntax is not a function call, or the NULL semantics differ.** Custom `SqlExpression` with `IQuerySqlGeneratorFactory` and `IRelationalParameterBasedSqlProcessorFactory`<br>provider base · single · Singleton or Scoped; `Quote` experimental (EF9100) | Can the expression be composed from functions, operators and `CASE` with the `ISqlExpressionFactory`? | Handling the node only in the SQL generator: without the nullability processor EF throws, without the generator the operator silently disappears. | 42 |
| **The LINQ tree should be rewritten before translation (query rewriting).** `IQueryExpressionInterceptor`<br>public · multiple · Singleton | Does a named query filter or a translator suffice? | Using per-request state although the result is cached per query shape; passing a new instance per context, which creates a new internal provider every time; registering it as `ISingletonInterceptor`, then EF never calls it. | 16, 30 |
| **The translated expression tree should be adjusted.** `IQueryTranslationPostprocessorFactory`<br>provider base · single · Scoped | Can it be done earlier, with a query interceptor or a translator? | Replacing the service without deriving from the provider's implementation; then its post-processing steps are missing. | 13, 16 |
| **Loaded objects should be created, initialized or checked (materialization).** `IMaterializationInterceptor`<br>public · multiple · Singleton | Constructor binding (EF passes `DbContext`, `ILazyLoader`, `IEntityType`) or the `ChangeTracker.Tracked` event in the application. | Keeping per-context state in the interceptor; expensive work per row; passing a new instance per context. | 17 |

## Execution and saving

These mechanisms run on every execution and are therefore on the hot path. They also only see what goes through EF: a guarantee that should also hold for raw SQL or other tools belongs in the database, not in an interceptor.

| Problem and mechanism | Check first | Typical wrong choice | Ch. |
|---|---|---|---|
| **Values should be set or changes checked when saving.** `ISaveChangesInterceptor`<br>public · multiple · Scoped | Does it affect only one application? Then overriding `SaveChanges` or subscribing to the `DbContext.SavingChanges` event suffices. | Implementing only the asynchronous variant; relying on `ExecuteUpdate` and raw SQL passing through it. | 18, 30 |

| Problem and mechanism | Check first | Typical wrong choice | Ch. |
|---|---|---|---|
| **Session state should be set when a connection is opened.** `IDbConnectionInterceptor` public · multiple · Scoped | Is the value the same for all connections? Then it belongs in the connection string. | Relying on the interceptor when the application passes connections that are already open; implementing only the synchronous or only the asynchronous variant. | 18 |
| **Commands should be observed or extended, or their errors translated.** `IDbCommandInterceptor` public · multiple · Scoped | `TagWith` for comments in the SQL; a translator if the meaning of the query should change. | Rewriting `CommandText` by text replacement; that breaks with every change in the shape of the SQL. | 18, 19, 32 |
| **State should be bound to a transaction.** `IDbTransactionInterceptor` public · multiple · Scoped | Does the connection interceptor suffice? It does as long as no transaction pooling is involved. | Using transaction-local settings but expecting queries outside of transactions or transactions that EF does not begin itself (`UseTransaction`, `TransactionScope`) to work: they run without the state. The sample then fails closed (page 144). | 36 |
| **INSERT, UPDATE or DELETE should take a different shape (upsert, `now()` in UPDATE).** Update pipeline, for example `IUpdateSqlGenerator` provider base · single · Singleton or Scoped | SaveChanges interceptor (rewriting states), command interceptor or `ExecuteUpdate`. | Replacing the pipeline for soft delete; a SaveChanges interceptor does that. | 32 |

## Schema and tooling

Migrations work against the design-time model, the other `dotnet ef` tools in a service provider of their own. A plugin registers services for that provider through `IDesignTimeServices`, not in `ApplyServices`.

| Problem and mechanism | Check first | Typical wrong choice | Ch. |
|---|---|---|---|
| **Schema objects should be created from the model and follow its changes.** Annotations, `IRelationalAnnotationProvider` and `IMigrationsSqlGenerator` provider base · single · Singleton or Scoped | `migrationBuilder.Sql()` in the application's migrations, existing configuration such as check constraints. | Copying the annotations into the runtime model as well (ignoring `designTime`); building identifiers by string concatenation instead of through the SQL helper. | 20, 21 |

| Problem and mechanism | Check first | Typical wrong choice | Ch. |
|---|---|---|---|
| **Migrations need steps that do not belong in the model (roles, privileges).** Custom `MigrationOperation` with an extension for `MigrationBuilder`, SQL in the generator<br>public · generator: provider base · single · Scoped | `migrationBuilder.Sql()` suffices if the plugin does not have to offer the step to many applications. | Writing operations that are not idempotent; ignoring the order relative to the generated operations. | 22, 35 |
| **The differ should generate custom operations by itself.** `IMigrationsModelDiffer` and `ICSharpMigrationOperationGenerator`<br>differ pubternal, generator public · single · Scoped or Singleton (design time) | The declarative way through annotations, almost always the better choice. | Replacing the differ just because a changed annotation needs SQL; the SQL generator does that. | 22 |
| **Snapshots should show Fluent API instead of `HasAnnotation`.** `IAnnotationCodeGenerator`, loaded through `IDesignTimeServices` and `DesignTimeServicesReference`<br>provider base · single · Singleton (design time) | Do nothing: `HasAnnotation` in the snapshot works, it is just hard to read. | Registering design-time services in `ApplyServices`, that is in the internal provider; not removing translated annotations from the dictionary. | 23 |
| **The compiled model should omit design-time annotations or contain complex values.** `ICSharpRuntimeAnnotationCodeGenerator`<br>pubternal · single · Singleton (design time) | EF copies simple annotation values without a custom generator. | Also filtering runtime annotations that an interceptor needs; forgetting the relational model that EF additionally writes for NativeAOT. | 24, 44 |
| **Scaffolding should recognize the plugin's schema objects.** `IDatabaseModelFactory` and `IAnnotationCodeGenerator`<br>provider base · single · Scoped or Singleton (design time) | T4 templates (`dotnet new ef-templates`) if only the generated code should look different. | Using a data format of your own instead of the Fluent API's annotations; then snapshot and migrations do not match. | 25, 45 |

**Four wrong choices that occur in every area**

**Replacing where a plugin interface or a decorator suffices.** Every replacement excludes another plugin (chapter 6).

**Storing run-time values where EF caches.** The options hash, the model and translated queries are reused. Per-request values belong in parameters, in the database session or in interceptors from the application.

**Mistaking a run-time hook for a guarantee.** Interceptors only see what goes through EF (chapter 30).

**Confusing design time and run time.** Design-time services do not belong in the internal provider, design-time annotations not in the runtime or compiled model.

Part II

# Infrastructure

How a plugin is configured, registers its services, connects to the application’s DI container, and which state belongs where.


› Chapter 5: The options extension

› Chapter 6: Registering services

› Chapter 7: Public API and integration into the application

› Chapter 8: State, lifetimes and thread safety

# 5 The options extension

Version dependency: **medium**

The entry point of every real plugin is a class implementing `IDbContextOptionsExtension`. It has three jobs: it carries the configuration, it registers the services in the internal provider in `ApplyServices`, and it describes itself to the cache through `Info`.

## Configuration as a record

All settings of the sample live in a record. Each of them influences the model, the registered services or what services read through the singleton options. Records provide `Equals` and `GetHashCode` over all fields, which is exactly what the cache key needs. New settings are added to the record without touching hashing or comparison.

Infrastructure/TenancySettings.cs

```
public sealed record TenancySettings
{
    public static TenancySettings Default { get; } = new();

    public string SettingName { get; init; } = "app.tenant_id";
    public string PolicyName { get; init; } = "tenant_isolation";
    public bool ForceRowLevelSecurity { get; init; } = true;
    public bool ApplyQueryFilter { get; init; }
    public bool CreateTenantIndex { get; init; } = true;
    public bool VerifyMaterializedTenants { get; init; }
    public bool ProtectBulkOperations { get; init; }
    public bool UseTransactionPooling { get; init; }
    public bool EnrichOpenTelemetry { get; init; }
}
```

## Immutability

EF compares and caches options. Once added, an extension must therefore never change again; every change creates a copy. What happens otherwise is shown by failure case 4 in chapter 8: new contexts silently keep working with the old value. The pattern from EF Core itself: a protected copy constructor and a virtual `Clone` method, so derived extensions clone their own type.

Infrastructure/TenancyOptionsExtension.cs (excerpt, simplified)

```
public class TenancyOptionsExtension : IDbContextOptionsExtension
{
    private DbContextOptionsExtensionInfo? _info;

    public TenancyOptionsExtension()
    {
    }

    protected TenancyOptionsExtension(TenancyOptionsExtension copyFrom)
    {
        ArgumentNullException.ThrowIfNull(copyFrom);
```

```
        Settings = copyFrom.Settings;
    }

    public virtual TenancySettings Settings { get; private set; } = TenancySettings.Default;

    public virtual DbContextOptionsExtensionInfo Info => _info ??= new ExtensionInfo(this);

    protected virtual TenancyOptionsExtension Clone() => new(this);

    public virtual TenancyOptionsExtension WithSettings(TenancySettings settings)
    {
        var clone = Clone();
        clone.Settings = settings;
        return clone;
    }
```

## The info class and the service provider cache

The info class decides whether EF may reuse a cached internal service provider. Two methods must agree for this: `GetServiceProviderHashCode` as a fast pre-filter and `ShouldUseSameServiceProvider` as the exact comparison.

Infrastructure/TenancyOptionsExtension.cs › ExtensionInfo (simplified)

```
private sealed class ExtensionInfo : DbContextOptionsExtensionInfo
{
    public ExtensionInfo(TenancyOptionsExtension extension)
        : base(extension)
    {
    }

    private new TenancyOptionsExtension Extension => (TenancyOptionsExtension)base.Extension;

    public override bool IsDatabaseProvider => false;

    public override string LogFragment
        => $"AcmeTenancy(Setting={Extension.Settings.SettingName}) ";

    public override int GetServiceProviderHashCode() => Extension.Settings.GetHashCode();

    public override bool ShouldUseSameServiceProvider(DbContextOptionsExtensionInfo other)
        => other is ExtensionInfo o && o.Extension.Settings == Extension.Settings;

    public override void PopulateDebugInfo(IDictionary<string, string> debugInfo)
    {
        debugInfo["Acme:Tenancy:Force"] =
            Extension.Settings.ForceRowLevelSecurity.ToString();
        // ... further settings
    }
}
```

There are two classic mistakes with these methods, with opposite consequences:

| Mistake | Consequence |
|---|---|
| A model-relevant option is missing from the hash | Two contexts with different configuration share provider and model. The second gets the cached, **wrong** model. Often unnoticed in tests, because only one configuration runs there. |
| A value that varies per context is in the hash (tenant, connection string, instance) | A new internal provider including model is created for every context. Memory and startup time explode; from the 21st provider on, EF throws by default with `ManyServiceProvidersCreatedWarning` (failure case 2 in chapter 8). |

The first mistake typically arises when a plugin without settings gets one later and the info class from the first version stays as it was:

`Info class (WRONG)`

```
// From the first version without settings; SettingName was added later
public override int GetServiceProviderHashCode() => 0;

public override bool ShouldUseSameServiceProvider(DbContextOptionsExtensionInfo other)
    => other is ExtensionInfo;
```

**What happens if the hash forgets a setting?**

**Observable.** A context with `app.b` runs on the provider EF built for `app.a`: singletons and model carry the value of the first configuration. There is neither an error nor a warning, and tests with only one configuration see nothing.

**Cause.** Hash and comparison are the key of the provider cache. If both say “equal”, EF uses the cached provider; `ISingletonOptions` is not initialized again, and `Validate` does not run in the normal case (chapter 8).

**Solution.** Include all model-relevant settings in hash and comparison, most simply as a record like `TenancySettings`.

**Test:** `FailureCaseTests.Setting_missing_from_the_hash_…`; the counter-check is `OptionsTests.Different_settings_produce_different_models_…`.

**The rule of thumb for the hash**

The hash must contain exactly what **influences which services are registered, what the model looks like or what services read through `ISingletonOptions`**, and nothing else. The tenant itself changes per request and must therefore never end up in the options; it arrives at run time through an interceptor (chapter 18).

`LogFragment` appears in EF’s startup message (“initialized using provider … with options: …”) and helps when troubleshooting. `PopulateDebugInfo` supplies values for EF’s diagnostics when too many service providers are created. `IsDatabaseProvider` is `true` only for real providers.

## Validation

`Validate` is called when the service provider is created. Here the plugin checks preconditions arising from the combination of extensions and reports them with a clear message instead of a later follow-up error:

Infrastructure/TenancyOptionsExtension.cs › Validate

```
public virtual void Validate(IDbContextOptions options)
{
    if (options.FindExtension<NpgsqlOptionsExtension>() is null)
        throw new InvalidOperationException(
            "UseAcmeTenancy() requires UseNpgsql(). Other databases need their own "
            + "provider package.");
}
```

# 6 Registering services

Version dependency: **medium**

`ApplyServices` fills the internal service provider. EF provides a builder for this that knows the lifetime and cardinality of every EF service: `EntityFrameworkServicesBuilder` for core services and `EntityFrameworkRelationalServicesBuilder` additionally for relational services. It registers each service with the correct lifetime and automatically uses `TryAddEnumerable` for services that can be registered multiple times.

`Infrastructure/TenancyOptionsExtension.cs › ApplyServices`

```
public virtual void ApplyServices(IServiceCollection services)
{
    // 1) Plugin interfaces: multiple registrations, composable with other plugins
    new EntityFrameworkRelationalServicesBuilder(services)
        .TryAdd<IRelationalTypeMappingSourcePlugin, TenancyTypeMappingSourcePlugin>()
        .TryAdd<IMethodCallTranslatorPlugin, TenancyMethodCallTranslatorPlugin>()
        .TryAdd<IMemberTranslatorPlugin, TenancyMemberTranslatorPlugin>()
        .TryAdd<IEvaluatableExpressionFilterPlugin,
                TenancyEvaluatableExpressionFilterPlugin>()
        .TryAdd<IConventionSetPlugin, TenancyConventionSetPlugin>()
        .TryAdd<ISingletonOptions, ITenancySingletonOptions>(
            p => p.GetRequiredService<ITenancySingletonOptions>())
        // 2) Own services that EF does not know
        .TryAddProviderSpecificServices(m => m
            .TryAddSingleton<ITenancySingletonOptions, TenancySingletonOptions>()
            .TryAddSingleton<TenancyLoggingDefinitions, TenancyLoggingDefinitions>());

    // ... optional interceptors depending on the settings, also via the services builder
    //     (chapters "Materialization: from rows to objects", "Bulk operations and the
    //     limits of interceptors", "Diagnostics and logging")

    // 3) Replace single services of the provider
    services.Replace(ServiceDescriptor
        .Scoped<IMigrationsSqlGenerator, TenancyMigrationsSqlGenerator>());
    services.Replace(ServiceDescriptor
        .Singleton<IRelationalAnnotationProvider, TenancyAnnotationProvider>());
}
```

| Method | When | Why |
|---|---|---|
| `TryAdd<TService, TImpl>()` | EF services and plugin interfaces | Lifetime and multiple registration come from EF's table. Wrong lifetimes otherwise cause exceptions or subtly wrong behavior. |
| `TryAddProviderSpecificServices` | Own services EF does not know | The builder rejects unknown types; here you set the lifetime yourself. |
| `services.Replace(...)` | Replace a single provider service | The provider registers with `TryAdd`. `Replace` wins regardless of whether the plugin runs before or after the provider. |

**Why Replace works in both orders**

EF calls the extensions' `ApplyServices` in the order they were added. If the plugin runs first, `Replace` adds the service and the provider's later `TryAdd` does nothing. If it runs afterwards, `Replace` replaces the provider's service. Both orders lead to the same result.

Bypassing the builder means choosing the lifetime yourself, and the obvious choice is wrong:

`ApplyServices (WRONG)`

```
public void ApplyServices(IServiceCollection services)
    => services.AddSingleton<IMethodCallTranslatorPlugin, ReverseTranslatorPlugin>();
```

**What happens if a plugin interface gets the wrong lifetime?**

**Observable.** Registration and model building succeed. Only the first query that uses the function fails, with `ArgumentNullException`: "Value cannot be null. (Parameter 'provider')". The message names neither the plugin nor the lifetime.

**Cause.** EF expects translator plugins as scoped. As a singleton, the plugin is created in the root scope of the internal provider, where its scoped dependency `ISqlExpressionFactory` cannot be resolved. The internal provider does not check lifetimes (chapter 8).

**Solution.** Always register EF services and plugin interfaces through the services builder; `TryAdd` takes the lifetime from EF's table.

**Test:** `FailureCaseTests.Plugin_interface_registered_as_singleton_…`.

## Singleton options

Singleton services such as type mapping plugins cannot resolve `IDbContextOptions` per context. For them there is `ISingletonOptions`: a singleton that EF initializes once from the options when creating the provider. When EF calls `Initialize` and `Validate` is explained in chapter 8.

`Infrastructure/ITenancySingletonOptions.cs (simplified)`

```
public class TenancySingletonOptions : ITenancySingletonOptions
{
    public TenancySettings Settings { get; private set; } = TenancySettings.Default;

    // Once per internal service provider
    public virtual void Initialize(IDbContextOptions options)
        => Settings = options.FindExtension<TenancyOptionsExtension>()?.Settings
                      ?? TenancySettings.Default;

    // Options must not differ from the provider's state (shared provider)
    public virtual void Validate(IDbContextOptions options)
    {
        var settings = options.FindExtension<TenancyOptionsExtension>()?.Settings
                       ?? TenancySettings.Default;
        if (settings != Settings)
            throw new InvalidOperationException("The tenancy options differ ...");
    }
}
```

The double registration in `ApplyServices` is required: once under its own interface so services can inject it, and once as `ISingletonOptions` so EF finds and initializes it. The factory variant ensures that both registrations return *the same* instance.

## Staying composable despite replacement

When a plugin replaces a single service, a second plugin doing the same cannot be active at the same time. Professional plugins mitigate this by additionally offering the actual logic publicly and reusably. How an application then combines two plugins is shown in the section “Integration as the way out” (page 34); this option belongs in the documentation.

Better still, a plugin avoids the conflict altogether by **decorating** the service instead of replacing it: it takes whatever implementation is registered at that point (the provider’s or another plugin’s), wraps it and delegates to it. An example is a (fictional) audit plugin that attaches a trigger to every new table. In its `ApplyServices` it looks up the existing registration and replaces only the descriptor, not the logic:

tests/Acme.EntityFrameworkCore.Tenancy.Tests/CompositionTests.cs › AuditOptionsExtension

```
public void ApplyServices(IServiceCollection services)
{
    // Registered so far: the provider's generator or that of another plugin
    var inner = services.LastOrDefault(d => d.ServiceType == typeof(IMigrationsSqlGenerator))
        ?? throw new InvalidOperationException("Call UseContosoAudit() after UseNpgsql().");

    services.Replace(ServiceDescriptor.Describe(
        typeof(IMigrationsSqlGenerator),
        sp => new AuditMigrationsSqlGenerator(
            (IMigrationsSqlGenerator)CreateInstance(sp, inner),
            sp.GetRequiredService<MigrationsSqlGeneratorDependencies>()),
        inner.Lifetime));     // lifetime of the original
}

// Creates the implementation the way the original descriptor would have
private static object CreateInstance(IServiceProvider sp, ServiceDescriptor descriptor)
    => descriptor.ImplementationInstance
       ?? descriptor.ImplementationFactory?.Invoke(sp)
       ?? ActivatorUtilities.CreateInstance(sp, descriptor.ImplementationType!);
```

The trick works because EF calls `ApplyServices` in the order in which the extensions were added; when the audit plugin runs, the provider has already registered its generator. `CreateInstance` reproduces the original registration exactly: an instance, a factory, or a type whose dependencies (such as Npgsql’s singleton options) `ActivatorUtilities` resolves from the container. The decorator itself only uses the public interface:

tests/.../CompositionTests.cs › AuditMigrationsSqlGenerator (shortened)

```
public class AuditMigrationsSqlGenerator : IMigrationsSqlGenerator
{
    // Constructor: (IMigrationsSqlGenerator inner, MigrationsSqlGeneratorDependencies deps)

    public IReadOnlyList<MigrationCommand> Generate(
        IReadOnlyList<MigrationOperation> operations,
        IModel? model = null,
        MigrationsSqlGenerationOptions options = MigrationsSqlGenerationOptions.Default)
```

```
    {
        var commands = _inner.Generate(operations, model, options).ToList();   // unchanged
        var builder = new MigrationCommandListBuilder(_dependencies);
        var sql = _dependencies.SqlGenerationHelper;

        // Appended at the end: by then every table of this migration exists
        foreach (var table in operations.OfType<CreateTableOperation>())
        {
            builder.Append("CREATE TRIGGER audit_changes ")
                .Append("AFTER INSERT OR UPDATE OR DELETE ON ")
                .Append(sql.DelimitIdentifier(table.Name, table.Schema))
                .Append(" FOR EACH ROW EXECUTE FUNCTION audit.log_change()")
                .AppendLine(sql.StatementTerminator)
                .EndCommand();
        }

        commands.AddRange(builder.GetCommandList());
        return commands;
    }
}
```

In the application, the two plugins are combined without a single line of glue code:

Program.cs (simplified)

```
services.AddDbContext<ShopContext>((sp, options) => options
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp)       // replaces IMigrationsSqlGenerator
    .UseContosoAudit());      // wraps whatever is registered: here Acme's
```

**Decorate where possible, replace where necessary**

Decorators compose with each other in any order, because each one wraps whatever it finds. A plugin that uses `Replace` however discards everything registered before it, including decorators. A replacing plugin therefore has to be activated first, and a decorator must check that there is something to decorate and fail with a clear message otherwise (the provider is missing or was added later).

Decorating requires that the public interface is enough. The tenancy plugin needs protected hooks inside `CREATE TABLE` and therefore derives from Npgsql's generator. For such cases the fallback from "Integration as a way out" remains (page 34): public logic such as `CreatePolicySql`, virtual methods, and `ReplaceService` in the application.

**Tip**

The test `CompositionTests` in the sample contains the complete audit plugin. It checks that the decorator wraps Acme's generator, that the migration contains the SQL of both plugins, and that activating the decorator before the provider fails.

## Two architectures: multiple registration and single service

Whether two plugins work side by side is decided in one place: does EF ask *all* registrations of a service or exactly *one*? With a plugin interface, cooperation is part of EF's design. EF asks all translator plugins until one answers, lets every convention set plugin add its conventions and calls every interceptor. Two plugins need to know nothing about each other.

With a single service there is no such cooperation. EF asks exactly one implementation, and it has to do everything all active plugins need. If two plugins replace the same service, one wins and the other loses its function without EF warning about it. Composition is then no longer EF's job but that of the plugin authors: through decorators that delegate, or through an integration in the application.

## The rules in EF Core 10

The methods from the table at the beginning of this chapter behave like this with two plugins. The rows are reproduced in `CompositionMatrixTests` and `CompositionTests`:

| Mechanism | Behavior | Consequence |
|---|---|---|
| Options extensions | One extension per type: a second `AddOrUpdateExtension` of the same type replaces the first. `ApplyServices` runs in activation order; every extension contributes to the provider key | Different plugins coexist; the use method fetches the existing extension and changes a copy |
| `TryAdd` for a plugin interface | Adds unless the same implementation is already registered (`TryAddEnumerable`) | Idempotent, composable |
| `TryAdd` for a single service | The first registration wins. The provider registers with `TryAdd`: a plugin before the provider wins, after it it silently loses | Unsuitable for single services |
| `Replace` | The last registration wins, independent of the provider | Silently discards earlier replacements and decorators |
| Decorator | Wraps what is registered at its point in time | Must run after the provider and after replacements |
| `ReplaceService` (application) | Applied after all extensions and always wins | The application's tool for integration |
| Translator plugins | Plugins come before the provider's translators; among plugins, the one activated first answers | Same method: the order decides |
| Convention set plugins | Run in activation order; later conventions see the results of earlier ones | Same metadata: order and configuration source decide |
| Interceptors | Injected ones first, then those from `AddInterceptors` in their order | Whoever suppresses or replaces a result affects the ones that follow |

## Compatibility matrix

The following matrix rates typical combinations of two plugins. The ratings mean: **composable** (works without coordination), **order-dependent** (works if the activation order is right), **potential conflict** (works as long as the plugins do not overlap) and **integration only** (without code in the application, a function is lost).

| Combination | Rating | Reason |
|---|---|---|
| Two `IMethodCallTranslatorPlugin` | composable; potential conflict | Different methods: both active. The same method: the one activated first wins, the other is never asked |
| Two `IMemberTranslatorPlugin` | composable; potential conflict | The same mechanism as for method translators |

| Combination | Rating | Reason |
|---|---|---|
| Two `IConventionSetPlugin` | composable; order-dependent | Both run. If both set the same metadata with convention source, the later one wins; explicit configuration beats both |
| Two type mapping plugins for the same CLR type | potential conflict | EF takes the first mapping that is not `null`; the other plugin has no effect for this type |
| Two interceptors of the same kind | composable; order-dependent | Both run. Whoever suppresses a result or rewrites a command affects the ones that follow |
| Two plugins replace `IMigrationsSqlGenerator` | integration only | The one activated last wins; the first loses its SQL without a warning (in the test: no policies any more) |
| Two plugins replace `IQuerySqlGeneratorFactory` | integration only | The loser keeps its translators, but its nodes reach a generator that does not know them: the SQL is silently missing. If the winner also replaces the parameter-based processor, the query throws (chapter 42) |
| Decorator and replacement | order-dependent | Replacement first, then decorator: it wraps the replacement. The other way round, `Replace` silently discards the decorator |
| Two decorators | composable | Each wraps what it finds |
| Two options extensions of different types | composable | Both contribute to the key, both register their services; conflicts only arise in the services |
| Two plugins with the same use method name, annotation prefix or event ID range | potential conflict | Ambiguous calls, mixed metadata or messages (chapters 46, 9, 19) |
| Plugin replacement and the application's `ReplaceService` | composable if intended | The application always wins; that is the intended way to integrate |

## Integration as the way out

What works "only with integration" is solved by the application with a class of its own that combines both plugins and is registered with `ReplaceService`. That is only possible if at least one of the plugins offers its logic publicly: the application derives from its class and calls the logic of the other. That is exactly why all generator methods in the sample are virtual and `CreatePolicySql` is public:

Application (sketch)

```
// Both plugins replace IMigrationsSqlGenerator; the application combines them
public class ShopMigrationsSqlGenerator : TenancyMigrationsSqlGenerator
{
    // Constructor as in TenancyMigrationsSqlGenerator

    protected override void Generate(CreateTableOperation operation, IModel? model,
        MigrationCommandListBuilder builder, bool terminate = true)
    {
        base.Generate(operation, model, builder, terminate);   // Acme: table and RLS
        OtherPluginSql.AppendTableSql(operation, builder);     // the other plugin's logic
    }
}

options.UseNpgsql(connectionString)
```

```
        .UseAcmeTenancy(sp)
        .UseOtherPlugin()
        .ReplaceService<IMigrationsSqlGenerator, ShopMigrationsSqlGenerator>();
```

A plugin that replaces a single service and keeps its logic only in private or internal methods does not leave this way open. The application then has no choice but to decide for one of the plugins.

## Before publishing: checking coexistence

A plugin author can check with little effort how well the plugin gets along with others. `CompositionMatrixTests` shows the technique: a fictional second plugin with its own options extension that registers the same services, and assertions for both activation orders.

- ☐ List every replaced single service and name it in the documentation.
- ☐ Replace single services with `Replace`, never with `TryAdd`; decorate where the public interface suffices.
- ☐ Decorators abort with a clear message when there is nothing to wrap.
- ☐ Offer the logic of replaced services as public and virtual so that applications can integrate.
- ☐ Plugin interfaces return `null` for anything foreign and do not claim methods, types or metadata that do not belong to them.
- ☐ Own annotation prefix, own event ID range, unique use method.
- ☐ Test with a second plugin in both activation orders, and with popular plugins such as EFCore.NamingConventions (chapter 10).
- ☐ Document the required activation order, if there is one.

# 7 Public API and integration into the application

Version dependency: **stable**

## The Use method

Users activate the plugin through an extension method on `DbContextOptionsBuilder`. It fetches the existing extension or creates a new one and registers it through `IDbContextOptionsBuilder` `Infrastructure`:

`Infrastructure/TenancyDbContextOptionsBuilderExtensions.cs (excerpt)`

```
public static DbContextOptionsBuilder UseAcmeTenancy(
    this DbContextOptionsBuilder optionsBuilder,
    Action<TenancyOptionsBuilder>? configure = null)
{
    ArgumentNullException.ThrowIfNull(optionsBuilder);

    var extension = optionsBuilder.Options.FindExtension<TenancyOptionsExtension>()
                    ?? new TenancyOptionsExtension();

    ((IDbContextOptionsBuilderInfrastructure)optionsBuilder).AddOrUpdateExtension(extension);
    configure?.Invoke(new TenancyOptionsBuilder(optionsBuilder));
    return optionsBuilder;
}

// Generic overload: method chaining stays typed
public static DbContextOptionsBuilder<TContext> UseAcmeTenancy<TContext>(
    this DbContextOptionsBuilder<TContext> optionsBuilder,
    Action<TenancyOptionsBuilder>? configure = null) where TContext : DbContext
    => (DbContextOptionsBuilder<TContext>)UseAcmeTenancy(
        (DbContextOptionsBuilder)optionsBuilder, configure);
```

The file lives in the namespace `Microsoft.EntityFrameworkCore`. That way the method can be found without an additional `using`, just like `UseNpgsql`. This is the common convention in the EF ecosystem.

## The options builder

Settings are configured through a dedicated builder. It validates input immediately and writes every change back as a new extension:

`Infrastructure/TenancyOptionsBuilder.cs (excerpt, simplified)`

```
public class TenancyOptionsBuilder
{
    public TenancyOptionsBuilder(DbContextOptionsBuilder optionsBuilder)
    {
        ArgumentNullException.ThrowIfNull(optionsBuilder);

        OptionsBuilder = optionsBuilder;
    }
```

```
    protected virtual DbContextOptionsBuilder OptionsBuilder { get; }

    public virtual TenancyOptionsBuilder UseSettingName(string settingName)
        => With(s => s with
        {
            SettingName = TenancyIdentifiers.ValidateSettingName(settingName),
        });

    public virtual TenancyOptionsBuilder ApplyQueryFilter(bool apply = true)
        => With(s => s with { ApplyQueryFilter = apply });

    protected virtual TenancyOptionsBuilder With(
        Func<TenancySettings, TenancySettings> change)
    {
        var extension = OptionsBuilder.Options.FindExtension<TenancyOptionsExtension>()
                        ?? new TenancyOptionsExtension();

        ((IDbContextOptionsBuilderInfrastructure)OptionsBuilder)
            .AddOrUpdateExtension(extension.WithSettings(change(extension.Settings)));
        return this;
    }
}
```

**Alternative: activation through the provider builder**

Provider-specific plugins are sometimes activated through the provider's options builder instead, for example `UseNpgsql(cs, o => o.UseNodaTime())`. For this you write an extension method on `NpgsqlDbContextOptionsBuilder` and reach the actual builder through `IRelationalDbContextOptionsBuilderInfrastructure.OptionsBuilder`. The PostgreSQL package uses the simpler route via `DbContextOptionsBuilder`, the SQL Server package the provider builder (chapter 47). Why the provider builder is the better choice as soon as there are several provider packages is explained in chapter 46.

## Connecting two containers

Some parts of a plugin need services of the *application*: the connection interceptor needs the current tenant, which only the application knows. The internal EF provider cannot resolve app services, though. The clean solution: such components live in the app container and are handed to EF via `AddInterceptors`.

DependencyInjection/… (core), Infrastructure/… (PostgreSQL), excerpts, simplified

```
// Core package: only register the tenant provider in the app container
public static IServiceCollection AddAcmeTenancy<TTenantProvider>(
    this IServiceCollection services,
    ServiceLifetime lifetime = ServiceLifetime.Scoped)
    where TTenantProvider : class, ITenantProvider
{
    services.TryAdd(ServiceDescriptor.Describe(
        typeof(ITenantProvider), typeof(TTenantProvider), lifetime));
    return services;
}

// Provider package: Use overload creates the interceptors from this provider
public static DbContextOptionsBuilder UseAcmeTenancy(
    this DbContextOptionsBuilder optionsBuilder,
```

```
    IServiceProvider applicationServices,
    Action<TenancyOptionsBuilder>? configure = null)
{
    var tenants = applicationServices.GetRequiredService<ITenantProvider>();

    return optionsBuilder
        .UseAcmeTenancy(configure)
        .AddInterceptors(
            // with UseTransactionPooling(): PostgreSqlTransactionSessionInterceptor
            new TenantConnectionInterceptor(tenants),
            new TenantSaveChangesInterceptor(tenants));
}
```

With `AddDbContext((sp, o) => ...)`, `sp` is the scope of the current request, because EF builds options per scope by default. The tenant provider and therefore the interceptors see the right tenant. The registration lives in the core package because it is the same for every provider (chapter 46). For the bridge in the other direction, an interceptor can read services from the *internal* provider via `eventData.Context.GetService<T>()`; the connection interceptor gets the setting name from the singleton options that way (chapter 18).

> **DbContext pooling**
>
> With `AddDbContextPool`, contexts are reused and receive their options only once, built with the root provider. The interceptor instances are created once and serve all later requests. A tenant provider that keeps its tenant per scope then always returns that of the first request. For pooling, the tenant provider must therefore be a singleton that determines the tenant anew on every access, for example via `IHttpContextAccessor`. Chapter 40 shows this variant, chapter 8 the rule behind it, and the same applies to `AddDbContextFactory`, whose factory is a singleton by default.

# 8 State, lifetimes and thread safety

Version dependency: **stable**

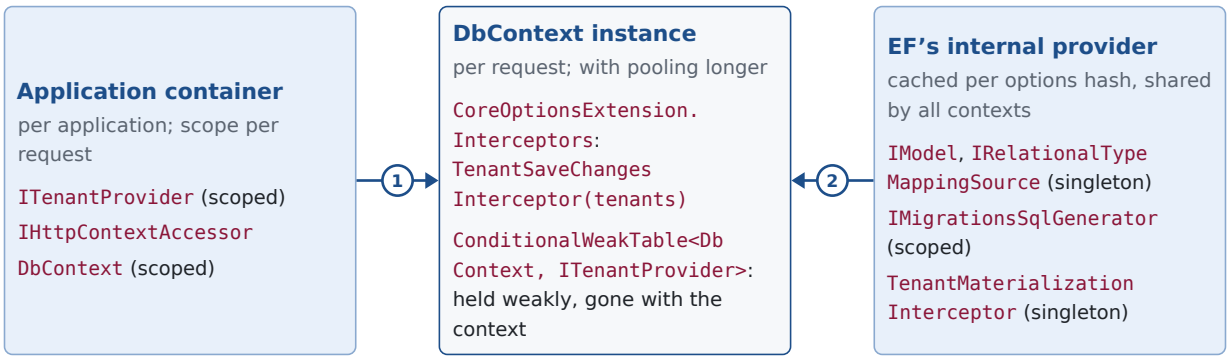
Almost every hard-to-find plugin bug has the same cause: state in the wrong place. A tenant in a singleton, a value in the hash that does not belong there, a scoped service held by a singleton. The previous chapters contain the individual rules; this chapter merges them into one and answers one question: **which kind of state may be stored where in an EF Core plugin?**

## Two containers, two meanings of lifetime

A plugin lives in two containers (chapters 2 and 7). The same words mean different things there:

| Lifetime | Application container | EF's internal provider |
|---|---|---|
| Singleton | Once per application | Once per internal provider, that is per configuration. Several configurations in one process yield several instances. |
| Scoped | Once per scope, in ASP.NET Core per request | Once per context instance. With pooling, the instance outlives the request and serves the next one. |
| Transient | New on every resolution | The same; rare for EF services (`ILazyLoader`) |
| `static` | Once per process, across all containers | The same |

Who builds which container decides the lifetime. The application builds the application container; the internal provider cannot resolve its services. EF builds the internal provider from the options, caches it by the hash of the options extensions (chapter 5) and shares it among all contexts with the same configuration. The lifetime of an EF service is set by EF (appendix A); `TryAdd` applies it (chapter 6).




1. `UseAcmeTenancy(sp)` resolves the tenant provider from the application container and hands it to the SaveChanges interceptor in the options of this context.
2. A singleton in the internal provider gets the context with every event and looks up the tenant provider of exactly that context. Nothing tenant-specific is stored in the internal provider or in the hash, so one internal provider serves all tenants without a leak and without a warning about many service providers.

## Which state belongs where

| State | Right place | Valid for |
|---|---|---|
| Static plugin configuration (annotation names, event IDs, defaults) | `const` or `static readonly`, immutable | Process |
| Cached reflection metadata (`MethodInfo`, `PropertyInfo`) | `static readonly` fields; they are immutable and therefore thread-safe | Process |
| Model-relevant configuration (setting name, policy name) | Immutable options extension, in the hash; singleton services read it through `ISingletonOptions` | Internal provider |
| Provider- and model-specific data (derived from the model or the options) | Annotation or runtime annotation on the model; intermediate results in a thread-safe singleton or through `GetOrAddRuntimeAnnotationValue` | Internal provider, model |
| DbContext-specific data | Scoped service in the internal provider or the context from the event data; reset with pooling (`IResettableService`) | Context instance |
| Connection-specific data (session settings) | The database session itself, set on every opening by a connection interceptor (chapter 18) | As long as the connection is open |
| Current tenant | `ITenantProvider` in the application container, read on every access; into the SQL only as a parameter or through the session | Request or job |
| Current user | Like the tenant: in the application container, per access (for example through `IHttpContextAccessor`) | Request |
| Other request-specific data (correlation ID, culture) | Application scope or `AsyncLocal`; in the SQL at most as a parameter or tag | Request or job |

Just as important is where state *never* belongs: values of a request never go into the options, the hash, the model, a singleton field or a translated query. All these places are reused across requests.

## Singleton means: one per internal provider

A singleton in the internal provider is not “global for the process”. Two contexts with a different setting name get two providers and therefore two instances of `TenancySingletonOptions`. Conversely, all contexts of one configuration share *one* instance, on all threads at the same time: parallel requests work with different contexts but with the same singletons.

A singleton must therefore be immutable after its initialization or cache in a thread-safe way. The sample shows four forms: the type mapping plugin is stateless, `TenancySingletonOptions` is initialized once and then only read, `TenancyLoggingDefinitions` creates its definitions with `LazyInitializer.EnsureInitialized` (chapter 19), and the materialization interceptor only remembers each context’s tenant provider, in a static, thread-safe `ConditionalWeakTable` that holds the context weakly (figure above). A single `DbContext`, on the other hand, is not thread-safe and is never used concurrently; a scoped service therefore only ever sees one caller at a time.

`ISingletonOptions` answers the question of how a singleton gets the configuration. `IDbContextOptions` is scoped and not available in a singleton (see failure case 3). EF calls `Initialize` once when it builds the provider, with the options of the first context. That is enough because the hash ensures that only contexts with the same model-relevant options get this provider. EF calls `Validate` when a context uses a provider that EF did not build for exactly these options, above all with `UseInternalServiceProvider`: then contexts with different settings may share a provider, and `Validate` has to reject that.

## Service provider and model cache

Both caches amplify every decision about state. The provider cache uses the options extensions as its key; the model is built per provider and context type (chapter 34). Chapter 5 covers the two directions of error: what is missing from the key although it influences the model leads to a wrong, shared model; what is in the key although it changes per request creates a provider including model per value (failure case 2 below). Instances of singleton interceptors are part of the key as well (chapter 17).

## Pooling, factories, jobs and open connections

The lifetimes above apply to `AddDbContext`. Other ways of creating contexts shift them:

| Situation | What changes | Rule |
|---|---|---|
| `AddDbContextPool` | Options and interceptors are created once from the root provider; the context instance and its scoped services serve many requests one after the other | Tenant provider reads per access; scoped services with state implement `IResettableService` (failure case 6 below) |
| `AddDbContextFactory` | Factory and options are singletons by default; the options callback gets the root provider | Register the factory as scoped or use a tenant provider that reads per access |
| Parallel requests | Different contexts, the same singletons | Singletons stateless or thread-safe |
| Background services | No `HttpContext`; the tenant comes from the job | Set the tenant before scope and context, for example through `AsyncLocal`; never fall back to “all tenants” |
| Long-open connections | The session state belongs to the connection, not to the context | Let EF open the connection; on a tenant change call `ApplyTenantSession()` (chapter 18); mind transaction pooling (chapter 36) |

## Six failure cases

Each of the following cases is reproduced as a test in the sample, the wrong variant next to the right one (`StateAndLifetimeTests`, case 5 in `MaterializationTests`, case 6 in `FailureCaseTests`).

| Case | Cause | Observable | Solution |
|---|---|---|---|
| 1. Current tenant in a singleton | A tenant provider remembers the first tenant, for example as an “optimization” | With pooling or a singleton factory, request B works entirely as tenant A: session, reads and writes. RLS does not help because all values match | Read the tenant on every access (`HttpContext`, `AsyncLocal`) |
| 2. Current tenant in the hash | The options extension carries the tenant into the provider key | One provider including model per tenant: memory and startup time grow, from the 21st on an exception | Tenant through interceptors and session; only model-relevant values in the hash |
| 3. Scoped service in a singleton | A singleton service injects `ICurrentDbContext` or `IDbContextOptions` | No error at registration, because EF does not validate scopes in the internal provider; the singleton gets `null` and fails later | Register the service as scoped, take the context from the event data, or read configuration through `ISingletonOptions` |
| 4. Options extension changed after registration | A mutable property of the extension is set later | The provider was built with the old value; new contexts silently keep working with it | Immutable extension, changes as a copy (`WithSettings`, chapter 5) |
| 5. Context state in a singleton interceptor | The interceptor remembers tenant or context of the first call | False alarms for other tenants, hidden leaks, race conditions; one instance per context creates one provider per context | Get data on every call through the context (chapter 17) |
| 6. Scoped state with pooling | A scoped plugin service collects data “per request”, such as counters or buffers | With pooling, the next request sees the data of the previous one, because context and scope are reused | Also register the service as `IResettableService` and clear it in `ResetState` |

Case 3 is the most treacherous because it starts without any warning. In the application container, ASP.NET Core reports such a “captive dependency” in the development environment when resolving; EF’s internal provider does not check it:

`tests/.../StateAndLifetimeTests.cs (shortened)`

```
// WRONG: a singleton that needs the current context
m.TryAddSingleton<ContextHolder, ContextHolder>();     // ContextHolder(ICurrentDbContext)

using var ctx = ContextWithHolder(ServiceLifetime.Singleton);
Assert.Null(ctx.GetService<ContextHolder>().Current);  // root scope: no context

// Right: scoped, one instance per context
Assert.Same(ctx, ctx.GetService<ContextHolder>().Current!.Context);
```

Case 4 can only be detected with `ISingletonOptions.Validate` where EF calls it, that is with a provider shared through `UseInternalServiceProvider`. The test `Shared_provider_rejects_contexts_with_different_tenancy_settings` shows that `TenancySingletonOptions` does exactly that. In the normal case, only immutability itself protects.

> **The rule in one sentence**
>
> What is the same for all contexts of a configuration belongs in options, model or singletons; what changes per context, in scoped services; what changes per request, in the application container, and from there per call into parameters or the session, never into a cache.

Part III

# The model

Metadata, Fluent API, conventions, validation and new column types.


› Chapter 9: Annotations and Fluent API

› Chapter 10: Conventions

› Chapter 11: Model validation

› Chapter 12: Type mappings and value converters

# 9 Annotations and Fluent API

Version dependency: **stable**

Annotations are EF's mechanism for extensible metadata: key-value pairs on every model element (model, entity type, property, index, ...). A plugin stores its configuration as annotations, and the entire remaining infrastructure (conventions, migrations, snapshots, compiled models) can work with them.

## Rules for annotations

| Rule | Reason |
|---|---|
| Own prefix (`Acme:Rls:`) | Avoids collisions with EF, providers and other plugins. |
| Simple values only: string, bool, numbers, enums | Snapshots and migrations write values as C# literals. Custom classes cannot be serialized. |
| Complex values as a string (e.g. JSON) | The differ compares strings reliably via `Equals`. A record with a `string[]` would be compared by reference and constantly appear "changed". |
| Keep explicit configuration, normalize it for migrations | For the differ, "annotation missing" and "annotation has default" differ and produce empty migrations. An explicit `force: false` still stays in the model; the annotation provider leaves the default out for migrations (see below). |
| One annotation per piece of configuration | Every policy has its own annotation, so the diff shows exactly which one changed. |

`Metadata/RlsAnnotationNames.cs, Metadata/RlsPolicy.cs (shortened, simplified)`

```
public static class RlsAnnotationNames
{
    public const string Prefix = "Acme:Rls:";
    public const string Enabled = Prefix + "Enabled";
    public const string Forced = Prefix + "Forced";
    public const string PolicyPrefix = Prefix + "Policy:";     // + policy name
    public const string QueryFilterKey = "Acme.Tenancy";
}

public sealed record RlsPolicy(string Name, PolicyCommand Command, bool Permissive,
    string[]? Roles, string? Using, string? WithCheck)
{
    public string Serialize()
        => JsonSerializer.Serialize(this, RlsJsonContext.Default.RlsPolicy);

    public static RlsPolicy Deserialize(string json) => /* ... */;
}
```

## The Fluent API

Fluent API methods are extension methods on EF's builders. Three design rules make them professional: the **non-generic** variant is the actual implementation (the model snapshot uses non-generic builders, chapter 23). The **generic** variant only delegates and preserves the type for

further chaining. And the class lives in the namespace `Microsoft.EntityFrameworkCore` so it is available everywhere without a `using`.

Builders/RlsEntityTypeBuilderExtensions.cs (shortened)

```
namespace Microsoft.EntityFrameworkCore;

public static class RlsEntityTypeBuilderExtensions
{
    public static EntityTypeBuilder HasRowLevelSecurity(
        this EntityTypeBuilder builder, bool force = false)
    {
        ArgumentNullException.ThrowIfNull(builder);
        builder.Metadata.SetAnnotation(RlsAnnotationNames.Enabled, true);

        // false too: an explicit decision beats the convention's FORCE default
        builder.Metadata.SetAnnotation(RlsAnnotationNames.Forced, force);

        return builder;
    }

    public static EntityTypeBuilder HasPolicy(
        this EntityTypeBuilder builder, string name, Action<PolicyBuilder> configure)
    {
        var policyBuilder = new PolicyBuilder(name);
        configure(policyBuilder);
        builder.Metadata.SetAnnotation(
            RlsAnnotationNames.PolicyPrefix + name, policyBuilder.Build().Serialize());
        return builder;
    }

    public static EntityTypeBuilder<TEntity> HasRowLevelSecurity<TEntity>(
        this EntityTypeBuilder<TEntity> builder, bool force = false) where TEntity : class
        => (EntityTypeBuilder<TEntity>)HasRowLevelSecurity(
            (EntityTypeBuilder)builder, force);

    // ... HasNoRowLevelSecurity, HasPolicy<TEntity> analogous
}
```

`HasRowLevelSecurity(force: false)` stores `false` and does not remove the annotation. Removed, it would be indistinguishable from “not configured”, and the isolation convention, which sets FORCE by default, would override the explicit decision. Two layers have different duties here: the model keeps every explicit configuration together with its configuration source, while the annotation provider normalizes for the migrations and treats `false` like a missing annotation: it does not pass it on to the relational model (chapter 21). That way the user’s decision wins, and a model without a change still produces no migration.

Nested configuration such as `HasPolicy` accepts an `Action<PolicyBuilder>`. The builder checks the database’s rules already while the model is built. A mistake like “INSERT policy with USING” therefore surfaces on the application’s first start, not when the migration runs in production:

Builders/PolicyBuilder.cs › Build

```
internal RlsPolicy Build()
{
    if (_using is null && _withCheck is null)
        throw new InvalidOperationException(
```

```
            $"Policy '{_name}' requires USING or WITH CHECK.");

    if (_command == PolicyCommand.Insert && _using is not null)
        throw new InvalidOperationException(
            $"INSERT policy '{_name}' only allows WITH CHECK.");

    if (_command is PolicyCommand.Select or PolicyCommand.Delete && _withCheck is not null)
        throw new InvalidOperationException(
            $"{_command} policy '{_name}' only allows USING.");

    return new RlsPolicy(_name, _command, _permissive,
        _roles.Count > 0 ? [.. _roles] : null, _using, _withCheck);
}
```

## Configuration sources

EF remembers for every piece of metadata *where* it came from. A source can override an equally strong or weaker one, but never a stronger one. This is the mechanism through which plugins set sensible defaults that users can always override:

| ConfigurationSource | Set by | Strength |
|---|---|---|
| `Explicit` | Fluent API in `OnModelCreating` (`Metadata.SetAnnotation`) | strongest |
| `DataAnnotation` | Attribute conventions (`fromDataAnnotation: true`) | medium |
| `Convention` | Conventions via `IConvention*Builder` | weakest |

For custom conventions the plugin therefore also offers overloads on the convention builder. They return `null` when a stronger source decided otherwise:

Builders/RlsEntityTypeBuilderExtensions.cs › convention variant

```
public static IConventionEntityTypeBuilder? HasRowLevelSecurity(
    this IConventionEntityTypeBuilder builder,
    bool enabled,
    bool fromDataAnnotation = false)
    => builder.HasAnnotation(RlsAnnotationNames.Enabled, enabled, fromDataAnnotation)
           is null
        ? null
        : builder;
```

The check of the source lives in the builder, not in the metadata. A convention that writes directly to the metadata bypasses it:

Convention (WRONG)

```
// In ProcessModelFinalizing: directly on the metadata instead of through entityType.Builder
entityType.SetAnnotation(RlsAnnotationNames.Enabled, true);
```

**What happens if a convention bypasses the configuration source?**

**Observable.** The user switches RLS off for an entity with `HasNoRowLevelSecurity()`, and after model building it is on again. The next migration enables RLS for a table that was explicitly excluded.

**Cause.** `SetAnnotation` on the convention metadata overwrites any existing value. Only the convention

builder compares the sources and returns `null` when a stronger one has decided.

**Solution.** In conventions always write through `entityType.Builder` or use the plugin's convention overloads.

**Test:** `FailureCaseTests.Convention_writing_to_the_metadata_…` and the counter-check through the builder.

# 10 Conventions

Version dependency: **stable**

Conventions change the model automatically. EF itself consists largely of conventions: discovering keys, finding relationships, deriving table names. A plugin adds its own to make configuration unnecessary.

## Convention types and timing

A convention implements one or more interfaces. Each stands for an event during model building. The most important ones:

| Interface | Event | Typical use |
|---|---|---|
| `IEntityTypeAddedConvention` | Entity type was added | Evaluate attributes, base configuration |
| `IPropertyAddedConvention` | Property was added | Column types, naming conventions |
| `IForeignKeyAddedConvention` | Relationship created | Indexes, delete behavior |
| `IEntityTypeBaseTypeChanged Convention` | Inheritance changed | Move configuration |
| `IModelFinalizingConvention` | Model is being finalized (still mutable) | Everything that depends on the finished model |
| `IModelFinalizedConvention` | Model is finished and validated by EF (read-only) | Validation |

The most common convention mistake is the wrong point in time:

Convention (WRONG)

```
public void ProcessPropertyAdded(IConventionPropertyBuilder propertyBuilder,
    IConventionContext<IConventionPropertyBuilder> context)
{
    // Remember the column name right away, for example for the policy expression
    propertyBuilder.HasAnnotation("Acme:Column", propertyBuilder.Metadata.GetColumnName());
}
```

**What happens if a convention runs too early?**

**Observable.** If the user then configures `HasColumnName("tenant")` or a naming plugin renames the column, the remembered name stays `TenantId`. The policy refers to a column that does not exist, and the migration only fails in the database.

**Cause.** Reactive conventions (`*Added`, `*Changed`) run while the user is still configuring. A column name may still change afterwards, an entity type may still get a base type.

**Solution.** Read whatever depends on the finished model in an `IModelFinalizingConvention`, through `GetColumnName(table)`. A convention that must stay reactive reacts to *all* relevant events.

**Test:** `FailureCaseTests.Convention_reading_the_column_name_too_early_…` and the counter-check at `ModelFinalizing`.

## Plugging in conventions

The plugin interface `IConventionSetPlugin` receives the convention set and adds its own conventions. `ConventionSet.Add` registers a convention in all lists whose interfaces it implements, each time at the end:

Conventions/TenancyConventionSetPlugin.cs

```
public class TenancyConventionSetPlugin : IConventionSetPlugin
{
    private readonly ProviderConventionSetBuilderDependencies _dependencies;
    private readonly IDbContextOptions _options;
    private readonly ISqlGenerationHelper _sqlGenerationHelper;
    private readonly TenancyLoggingDefinitions _loggingDefinitions;

    public TenancyConventionSetPlugin(
        ProviderConventionSetBuilderDependencies dependencies,
        IDbContextOptions options,
        ISqlGenerationHelper sqlGenerationHelper,
        TenancyLoggingDefinitions loggingDefinitions)
    {
        // ... ArgumentNullException.ThrowIfNull for all parameters ...
        _dependencies = dependencies;
        _options = options;
        _sqlGenerationHelper = sqlGenerationHelper;
        _loggingDefinitions = loggingDefinitions;
    }

    public virtual ConventionSet ModifyConventions(ConventionSet conventionSet)
    {
        var settings = _options.FindExtension<TenancyOptionsExtension>()?.Settings;
        if (settings is null)
            return conventionSet;

        // chapter "Value generation"
        conventionSet.Add(new TenantIdKeyValueGenerationConvention());
        conventionSet.Add(new NoTenantIsolationAttributeConvention(_dependencies));
        conventionSet.Add(new TenantIsolationConvention(
            settings, _sqlGenerationHelper, _dependencies.Logger, _loggingDefinitions));
        conventionSet.Add(new TenancyModelValidationConvention(
            _dependencies.Logger, _loggingDefinitions));

        return conventionSet;
    }
}
```

`ProviderConventionSetBuilderDependencies` is the parameter object that EF’s own conventions receive too. Among other things it contains the model logger, the validation logger and the type mapping source. Other services such as `ISqlGenerationHelper` can be injected as well, because the plugin itself is resolved from the internal provider.

## The isolation convention

The sample’s central convention runs at `ModelFinalizing` because column names are only final then. It finds the tenant property (marked with `IsTenantProperty()` or `TenantId` for `ITenant`

Owned, chapter 44), enables RLS with a policy, creates an index, sets a runtime annotation for the SaveChanges interceptor and optionally a query filter:

Conventions/TenantIsolationConvention.cs › ProcessModelFinalizing (simplified)

```
public virtual void ProcessModelFinalizing(
    IConventionModelBuilder modelBuilder,
    IConventionContext<IConventionModelBuilder> context)
{
    // TPT, TPC, entity splitting, owned types in their own table and implicit join
    // tables would leave tenant data without a tenant column: reject them first
    TenantTableMappingValidator.Validate(modelBuilder.Metadata, e => e.IsRlsOptedOut());

    foreach (var entityType in modelBuilder.Metadata.GetEntityTypes())
    {
        if (entityType.BaseType is not null       // TPH: the root type's table
            || entityType.IsOwned())              // validated: owner's table or JSON
            continue;

        // Marked property or "TenantId" with ITenantOwned (core package,
        // chapter "Property-level Fluent API and runtime annotations")
        if (entityType.FindTenantPropertyCandidate() is not IConventionProperty property)
            continue;      // (ITenantOwned without a mapped TenantId: exception, shortened)

        if (entityType.IsRlsOptedOut())
        {
            _logger.TenantIsolationOptedOut(_loggingDefinitions, entityType);  // warning
            continue;
        }

        // No table (view, ToSqlQuery, keyless) -> no RLS
        if (StoreObjectIdentifier.Create(entityType, StoreObjectType.Table)
            is not { } table)
            continue;

        // Runtime annotation: this is how the SaveChanges interceptor finds the property
        entityType.Builder.HasAnnotation(
            TenancyAnnotationNames.TenantProperty, property.Name);

        ApplyRowLevelSecurity(entityType, property, table);
        if (_settings.CreateTenantIndex) EnsureTenantIndex(entityType, property);
        if (_settings.ApplyQueryFilter) ApplyQueryFilter(entityType, property);

        _logger.TenantIsolationApplied(_loggingDefinitions, entityType);       // debug
    }
}
```

Before the convention annotates anything, it checks the tenant property. Both relational provider packages follow the same contract: a property marked with `IsTenantProperty()` wins, otherwise an `ITenantOwned` type needs a mapped property `TenantId` of type `TenantId`. If an `ITenantOwned` type has no such property, because `Ignore(e => e.TenantId)` removed it or because an explicit interface implementation is not a mapped property, the model throws an `InvalidOperationException`. That also applies to an entity excluded with `[NoTenantIsolation]`, since the interface promises a tenant column. Skipping the type silently would cost it the policy, the index and, because the runtime annotation would be missing, the check in `SaveChanges`. Types without `ITenantOwned` and without a marked property are not tenant data and stay untouched. The Cosmos package rejects a missing tenant property the same way (chapter 48).

Conventions/TenantIsolationConvention.cs › ApplyRowLevelSecurity

```
protected virtual void ApplyRowLevelSecurity(
    IConventionEntityType entityType,
    IConventionProperty property,
    StoreObjectIdentifier table)
{
    // Column name for exactly this table (may differ with table splitting)
    var column = _sqlGenerationHelper.DelimitIdentifier(property.GetColumnName(table)!);
    var condition =
        $"{column} = nullif(current_setting('{_settings.SettingName}', true), '')::uuid";

    var policy = new RlsPolicy(_settings.PolicyName, PolicyCommand.All, Permissive: true,
        Roles: null, Using: condition, WithCheck: condition);

    // Convention builder: does NOT override explicit configuration
    var builder = entityType.Builder;
    builder.HasAnnotation(RlsAnnotationNames.Enabled, true);
    if (_settings.ForceRowLevelSecurity)
        builder.HasAnnotation(RlsAnnotationNames.Forced, true);
    builder.HasAnnotation(RlsAnnotationNames.PolicyPrefix + _settings.PolicyName,
        policy.Serialize());
}
```

**Validate SQL you generate yourself**

The setting name is written into the policy expression without escaping. Unlike SQL that the developer writes in `Using(...)` themselves, here the *plugin* assembles SQL and is responsible for keeping invalid input out. That is why the options builder validates the name with a strict regular expression (`[a-z_][a-z0-9_]*\.[a-z_][a-z0-9_]*`).

## Which table mappings are protected

RLS protects tables, not entities. The policy compares the tenant column of the *same* table with the session setting. A table that holds tenant data but lacks this column cannot be protected by this policy. One entity can occupy several tables, and several entities can share one table. That is why `TenantTableMappingValidator` from the relational package checks, before any annotation, that every table with tenant data contains the tenant column. Both provider packages call it, so the contract is the same for PostgreSQL and SQL Server:

| Mapping | Tables with tenant data | Behavior |
|---|---|---|
| One table per entity | the table, with tenant column | protected |
| TPH (default) | one table for the whole hierarchy | protected, policy on the root type |
| Tenant-owned derived type below a root without a tenant | the same table, rows of other types without a tenant | rejected |
| TPT | one table per derived type *without* tenant column | rejected |
| TPC | one table per concrete type, the abstract base has none | rejected (not implemented) |
| Owned type in the owner's table (`OwnsOne`) | columns of the owner's table | protected |

| Mapping | Tables with tenant data | Behavior |
|---|---|---|
| Owned type as JSON (`ToJson`) | one column of the owner's table | protected |
| Owned type in its own table (`OwnsMany`, `ToTable`) | own table without tenant column | rejected |
| Table splitting | the same rows for all types | protected if all tenant-owned types use the same column |
| Entity splitting (`SplitToTable`) | another table without tenant column | rejected |
| Implicit many-to-many table with a tenant-owned side | join table without tenant column | rejected |
| Join entity with a tenant property (`UsingEntity`) | join table with tenant column | protected |
| View, `ToSqlQuery`, keyless without a table | no table of its own | no policy; the base tables carry it |
| Temporal table (SQL Server, `IsTemporal`) | the current table and its history table, which has no policy | rejected (SQL Server package) |

The evidence: `TableMappingTests` checks the rows from TPH to the join entity on the model, `SqlServerTemporalTableTests` the temporal table, `MappingIsolationTests` the protected rows against PostgreSQL, and `SqlServerTableMappingTests` the model and the DDL for SQL Server; the comparison on page 194 shows what is verified per provider.

Rejected means: the model throws an `InvalidOperationException` on the first access to `Model`, naming the table, the type and the way out. This happens before `EnsureCreated`, before any migration and before the first query. Seeming support that writes a policy onto a table without a matching column would be worse than the rejection. TPC could be protected in principle, since every concrete table inherits the tenant column. The plugin, however, supports exactly one table per hierarchy, and a half-done implementation must not pretend to be secure. The ways out are TPH or independent entities.

Temporal tables fail for the same reason, although the history table has the tenant column. A SQL Server security policy protects exactly the tables it names, and the plugin names the current table only. Queries with `FOR SYSTEM_TIME`, which EF issues for `TemporalAll()` and its siblings, read the history table without the current table's predicate and so return old rows of every tenant. The SQL Server convention therefore rejects a tenant-owned entity mapped with `IsTemporal()`; temporal tables without a tenant and excluded entities remain allowed. Protecting the history table as well would require predicates of its own on it, which the plugin does not generate.

Deliberate exceptions remain possible: an entity excluded with `[NoTenantIsolation]` or `HasNoRowLevelSecurity()` is not checked by the table validator, and the convention logs the exception as a warning. That way the model tells a decision from an oversight. For the implicit join table the PostgreSQL package offers `UsingEntity(j => j.HasNoRowLevelSecurity())`; the SQL Server package only knows the attribute and needs a join class for it. A tenant property on the join entity is almost always better:

Application

```
modelBuilder.Entity<Basket>()
    .HasMany(b => b.Labels).WithMany(l => l.Baskets)
    .UsingEntity("BasketLabel",
```

```
        j => j.Property<TenantId>("TenantId").IsTenantProperty());   // shadow property
```

The SaveChanges interceptor fills the shadow property like any other tenant property. `Mapping IsolationTests` checks both against a real PostgreSQL with a restricted role: a TPC model fails before a single table exists. In a model with TPH, an owned type, a JSON column and this join entity, every table has RLS, and without a tenant no row is visible in any table.

## Structural changes: index and query filter

Conventions may not only set annotations but change the model structurally. The index is only created if no index or key already starts with the tenant column. The query filter is built as an expression tree and registered as a **named filter**, new in EF Core 10. Named filters coexist with the user's own filters and can be switched off selectively with `IgnoreQueryFilters(["Acme.Tenancy"])`.

Conventions/TenantIsolationConvention.cs (excerpt)

```
protected virtual void EnsureTenantIndex(
    IConventionEntityType entityType, IConventionProperty property)
{
    var covered = entityType.GetIndexes().Any(i => i.Properties[0] == property)
                || entityType.GetKeys().Any(k => k.Properties[0] == property);
    if (!covered)
        entityType.Builder.HasIndex(new[] { property });
}

protected virtual void ApplyQueryFilter(
    IConventionEntityType entityType, IConventionProperty property)
{
    // e => EF.Property<TenantId>(e, "...") == EF.Functions.CurrentTenant()
    // EF.Property works for any property name and for shadow properties too
    var parameter = Expression.Parameter(entityType.ClrType, "e");
    var body = Expression.Equal(
        Expression.Call(typeof(EF), nameof(EF.Property), [typeof(TenantId)],
            parameter, Expression.Constant(property.Name)),
        Expression.Call(CurrentTenantMethod, Expression.Property(null, FunctionsProperty)));

    entityType.Builder.HasQueryFilter(
        RlsAnnotationNames.QueryFilterKey, Expression.Lambda(body, parameter));
}
```

Notably, the filter uses the plugin's own function `EF.Functions.CurrentTenant()`. It is therefore translated to SQL by the translator from chapter 14. The plugin's building blocks interlock.

**Combined with soft delete.** Before EF Core 10 an entity type had exactly one query filter: a second `HasQueryFilter` replaced the first, and `IgnoreQueryFilters()` switched off all of them at once, the tenant filter included. With named filters the application's own filters stay independent of the plugin's:

tests/.../QueryTranslationTests.cs (excerpt)

```
m.Entity<Order>().HasQueryFilter("SoftDelete", o => !EF.Property<bool>(o, "IsDeleted"));

// WHERE NOT (o."IsDeleted") AND o."TenantId" = nullif(current_setting(...), '')::uuid
var both = ctx.Orders.ToQueryString();
```

```
// Deleted rows become visible, the tenant filter stays
var withDeleted = ctx.Orders.IgnoreQueryFilters(["SoftDelete"]).ToQueryString();

// Only the tenant filter is off (with RLS the database still filters)
var allTenants = ctx.Orders.IgnoreQueryFilters(["Acme.Tenancy"]).ToQueryString();
```

**Switch off filters by name only**

An `IgnoreQueryFilters()` without names also switches off the tenant filter. Where an application shows deleted records, for example in a recycle bin, it names its own filter: `IgnoreQueryFilters(["SoftDelete"])`. Switching off `Acme.Tenancy` belongs to support or reporting roles only: with RLS the database still filters, without it (Cosmos DB, chapter 48) the query then sees every tenant.

## Final names, whoever sets them

The isolation convention reads the column name only at `ModelFinalizing` and only through `GetColumnName(table)`. Names that other conventions or plugins change are final by then. The sample checks this against EFCore.NamingConventions, which renames tables and columns to snake_case, in both activation orders:

Usage

```
options.UseNpgsql(connectionString).UseAcmeTenancy().UseSnakeCaseNamingConvention();
```

Generated migration SQL (excerpt, tests/.../NamingConventionTests.cs)

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON orders AS PERMISSIVE FOR ALL
    USING (tenant_id = nullif(current_setting('app.tenant_id', true), '')::uuid)
    WITH CHECK (tenant_id = nullif(current_setting('app.tenant_id', true), '')::uuid);
CREATE INDEX ix_orders_tenant_id ON orders (tenant_id);
```

**Never assemble names yourself**

A plugin that assembles names itself (from `Name` properties or by string concatenation) breaks as soon as a naming plugin is active. Always ask the model for the final name, and as late as possible.

## Attribute conventions

Data annotations are evaluated with an attribute convention. EF provides the base class `TypeAttributeConventionBase<TAttribute>`, which finds the attribute and only asks for the reaction. The value is set with `fromDataAnnotation: true` so the Fluent API can override it:

Conventions/NoTenantIsolationAttributeConvention.cs

```
public class NoTenantIsolationAttributeConvention
    : TypeAttributeConventionBase<NoTenantIsolationAttribute>
{
    public NoTenantIsolationAttributeConvention(
        ProviderConventionSetBuilderDependencies dependencies)
        : base(dependencies)
    {
    }
```

```
    protected override void ProcessEntityTypeAdded(
        IConventionEntityTypeBuilder entityTypeBuilder,
        NoTenantIsolationAttribute attribute,
        IConventionContext<IConventionEntityTypeBuilder> context)
        => entityTypeBuilder.HasRowLevelSecurity(enabled: false, fromDataAnnotation: true);
}
```

**Tip**

The attribute itself lives in the *core package*, the convention in the provider package. Entity classes can use the attribute without depending on PostgreSQL. This is the same split as with EF's own attributes in `System.ComponentModel.DataAnnotations`.

# 11 Model validation

Version dependency: **stable**

A plugin should detect misconfiguration before it leads to wrong SQL or data leaks. EF validates models through `IModelValidator`, but there is only one, and it belongs to the provider. For plugins, an `IModelFinalizedConvention` is the better place: it runs on the finished model and collides with nobody.

The timing is more precise than the name suggests. EF calls the finalized conventions only *after* the `IModelValidator`, when it creates the lean runtime model from the finished model. That conversion is itself a finalized convention (`RuntimeModelConvention`), and because `ConventionSet.Add` appends custom conventions at the end, the plugin's validation already receives the runtime model. It therefore only reads annotations that are kept there (chapter 44). With a compiled model it no longer runs at runtime at all, but already when the model is generated with `dotnet ef dbcontext optimize`.

Conventions/TenancyModelValidationConvention.cs (simplified)

```
public virtual IModel ProcessModelFinalized(IModel model)
{
    foreach (var entityType in model.GetEntityTypes())
    {
        if (!entityType.HasRlsConfiguration())
            continue;

        if (entityType.BaseType is not null)
            throw new InvalidOperationException(
                $"RLS on '{entityType.DisplayName()}' is invalid: only allowed on the root "
                + $"type ('{entityType.GetRootType().DisplayName()}').");

        if (StoreObjectIdentifier.Create(entityType, StoreObjectType.Table) is null)
            throw new InvalidOperationException("... not mapped to a table ...");

        if (!entityType.IsRlsEnabled() && entityType.GetRlsPolicies().Count > 0)
            _logger.PoliciesWithoutRowLevelSecurity(_loggingDefinitions, entityType);
    }
    return model;
}
```

The validation distinguishes two kinds of problems. **Structural errors** that are guaranteed to lead to wrong behavior throw an exception. **Questionable configuration** that may be intended produces a warning through the logging system. Users can turn such warnings into errors selectively (chapter 19).

# 12 Type mappings and value converters

Version dependency: **medium**

A type mapping connects a CLR type with a database type. It determines the column type in migrations, how parameters are set, what literals look like in SQL, how values are compared and copied and how they are stored in JSON columns. For custom types a plugin has two options:

| Approach | Advantages | Disadvantages |
|---|---|---|
| Value converter (user configures `HasConversion`) | No plugin needed | Must be configured for every property or via `ConfigureConventions` |
| `IRelationalTypeMapping` `SourcePlugin` | Type works everywhere without configuration, own literals, full control | Provider-specific (store type) |

## The type and its converter

The strongly typed `TenantId` and its converter live in the core package. The converter has a shared instance; the note on compiled models at the end of the chapter explains why.

TenantId.cs, TenantIdConverter.cs (shortened, simplified)

```
public readonly record struct TenantId(Guid Value)
{
    public static TenantId Empty => default;
    public bool IsEmpty => Value == Guid.Empty;              // translated to SQL
    public static TenantId New() => new(Guid.CreateVersion7());
}

public sealed class TenantIdConverter : ValueConverter<TenantId, Guid>
{
    public static TenantIdConverter Instance { get; } = new();

    public TenantIdConverter()
        : base(id => id.Value, value => new TenantId(value))
    {
    }
}
```

## The type mapping

```
public class TenantIdTypeMapping : RelationalTypeMapping
{
    // Required for compiled models: public static Default property
    public static TenantIdTypeMapping Default { get; } = new();

    public TenantIdTypeMapping()
        : base(new RelationalTypeMappingParameters(
            new CoreTypeMappingParameters(
                typeof(TenantId),
                converter: TenantIdConverter.Instance,
```

```
                jsonValueReaderWriter: TenantIdJsonReaderWriter.Instance),
            storeType: "uuid",
            dbType: System.Data.DbType.Guid))
    {
    }

    protected TenantIdTypeMapping(RelationalTypeMappingParameters parameters)
        : base(parameters)
    {
    }

    // EF clones mappings, e.g. for facets or changed comparers
    protected override RelationalTypeMapping Clone(
        RelationalTypeMappingParameters parameters)
        => new TenantIdTypeMapping(parameters);

    protected override string GenerateNonNullSqlLiteral(object value)
    {
        var guid = value switch
        {
            TenantId id => id.Value,
            Guid g => g,
            _ => throw new ArgumentException($"Unexpected type {value.GetType()}."),
        };
        return $"'{guid}'::uuid";
    }
}
```

Every custom mapping needs the protected constructor taking `RelationalTypeMappingParameters` and a `Clone` override that creates its own type. Without it, a cloned mapping loses its behavior, for example literal generation.

## The type mapping plugin

`RelationalTypeMappingSource` asks all plugins *before* falling back to the provider's mappings. A plugin returns `null` when it is not responsible:

Storage/TenancyTypeMappingSourcePlugin.cs (simplified)

```
public class TenancyTypeMappingSourcePlugin : IRelationalTypeMappingSourcePlugin
{
    public virtual RelationalTypeMapping? FindMapping(
        in RelationalTypeMappingInfo mappingInfo)
    {
        if (mappingInfo.ClrType != typeof(TenantId))
            return null;

        // Explicitly different column type (HasColumnType("text")): not responsible
        var storeTypeBase = mappingInfo.StoreTypeNameBase;
        if (storeTypeBase is not null
            && !string.Equals(storeTypeBase, "uuid", StringComparison.OrdinalIgnoreCase))
            return null;

        return TenantIdTypeMapping.Default;
    }
}
```

> **Note**
>
> Without a mapping, EF would not recognize `TenantId` as a scalar type during model building and would abort with an error. With the plugin, the type works as a property, as a key, in queries, as a parameter and in JSON columns, without a single line of configuration.

## JSON support

For JSON columns a mapping needs a `JsonValueReaderWriter`. It has three duties: read, write, and describe how it is created in generated code. The pattern follows EF's own `JsonGuidReaderWriter`:

TenantIdJsonReaderWriter.cs

```
public sealed class TenantIdJsonReaderWriter : JsonValueReaderWriter<TenantId>
{
    private static readonly PropertyInfo InstanceProperty =
        typeof(TenantIdJsonReaderWriter).GetProperty(nameof(Instance))!;

    public static TenantIdJsonReaderWriter Instance { get; } = new();

    private TenantIdJsonReaderWriter()
    {
    }

    public override TenantId FromJsonTyped(
        ref Utf8JsonReaderManager manager, object? existingObject = null)
        => new(manager.CurrentReader.GetGuid());

    public override void ToJsonTyped(Utf8JsonWriter writer, TenantId value)
        => writer.WriteStringValue(value.Value);

    // For compiled models and precompiled queries
    public override Expression ConstructorExpression
        => Expression.Property(null, InstanceProperty);
}
```

> **Compiled models need Default and Instance**
>
> When generating for NativeAOT (`dotnet ef dbcontext optimize --nativeaot`), the compiled model generator (chapter 24) emits code of the form `TenantIdTypeMapping.Default.Clone(...)` for every type mapping. It requires a **public static property** named `Default` on a **public** type and aborts otherwise. It only takes over converters, comparers and readers/writers without extra code when they are *the same instance* as in the default mapping. That is why the mapping uses the static instances rather than `new ...()`. Without NativeAOT, EF 10 resolves type mappings at startup instead; a plugin should still meet the requirements, because users decide on NativeAOT, not the plugin.

Part IV

# Queries

How LINQ becomes SQL, how a plugin translates its own functions and members, and how rows become objects again.


› Chapter 13: The query pipeline

› Chapter 14: Translating methods and members

› Chapter 15: Preventing client-side evaluation

› Chapter 16: Further query extension points

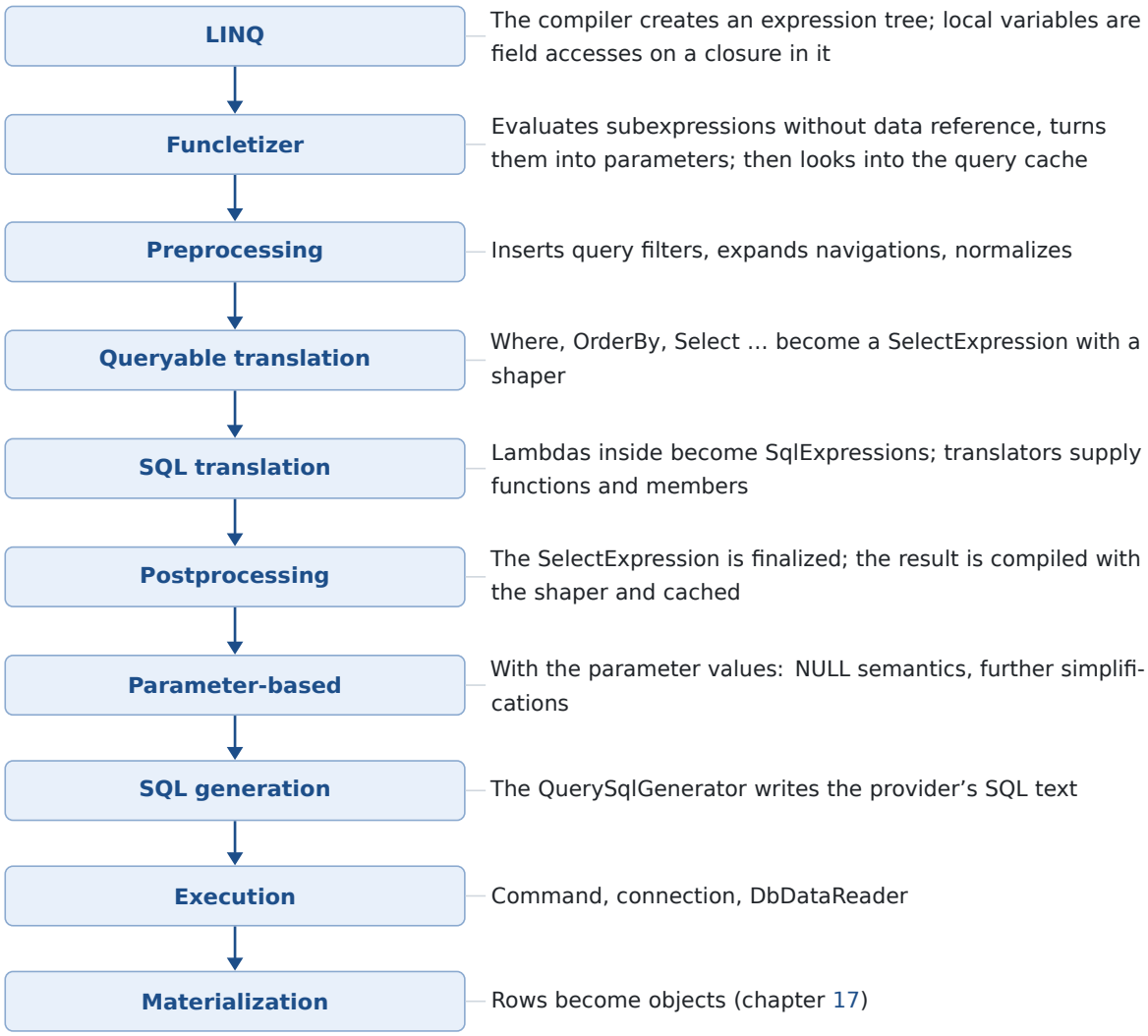
› Chapter 17: Materialization: from rows to objects

# 13 The query pipeline

Version dependency: **medium**

A LINQ query passes through a series of stages in EF Core before SQL is produced, and through another one before objects come back. Every query extension in this book sits at exactly one of these stages. This chapter is the map: it follows a concrete query through the pipeline, shows who is responsible at each stage and where a plugin can intervene, and helps to *locate* a desired change first. Which mechanism is then the right one is decided by the “Queries” section of the decision guide (page 19); how it works is explained in the respective chapter.

The stages at a glance:




## A query on its way

The sample query runs with the tenant query filter enabled (`ApplyQueryFilter()`, chapter 10). The name comes from a request and may be missing:

Application

```
string? name = filter.Name;                        // may be null

var customers = await db.Customers
    .Where(c => c.Name == name)
    .OrderBy(c => c.Id)
    .ToListAsync();
```

What happens to the sample query at the individual stages is shown by EF's debug logs (`CoreEventId.QueryCompilationStarting` and `QueryExecutionPlanned`) and `ToQueryString()`.

**Funcletizer.** The local value `name` has no data reference. EF does not inline it as a constant but turns it into a parameter. Only this tree is the key for the query cache: the same query with a different name hits the same entry.

Log: Compiling query expression

```
DbSet<Customer>()
    .Where(c => c.Name == @name)
    .OrderBy(c => c.Id)
```

**Preprocessing.** The plugin's named query filter is added, with `EF.Functions.CurrentTenant()`. That this call is still a call here and was not executed in .NET beforehand is thanks to the plugin's evaluatable expression filter (chapter 15).

**Queryable and SQL translation.** Both stages interlock: for every queryable operator, EF translates its lambda into SQL expressions. `c.Name == @name` becomes a binary SQL comparison, and for `CurrentTenant()` the plugin's method translator supplies a function expression. The result is a `SelectExpression`, the relational representation of the query, still independent of the provider's SQL syntax:

Log: Generated query execution expression (excerpt)

```
SELECT c.Id, c.Name, c.TenantId
FROM Customers AS c
WHERE (c.TenantId == CAST(nullif(current_setting('app.tenant_id', TRUE), '') AS uuid))
    && (c.Name == @name)
ORDER BY c.Id ASC
```

**Postprocessing and compilation.** The `SelectExpression` is finalized and compiled into a delegate together with the shaper that will later build the objects. From preprocessing up to here, everything runs once per query shape.

**Parameter-based processing.** Only at execution time does EF know the parameter values. The `RelationalParameterBasedSqlProcessor` evaluates the NULL semantics. If `name` is null, `c.Name = NULL` can never be true for a required column, and the whole filter collapses to `FALSE`. With a value, the comparison stays.

**SQL generation.** The provider's generator writes the text: Npgsql turns the `CAST` into the PostgreSQL notation `::uuid` and quotes identifiers.

ToQueryString() with name = "Contoso" and with name = null

```
-- @name='Contoso'
SELECT c."Id", c."Name", c."TenantId"
FROM "Customers" AS c
```

```
WHERE c."TenantId" = nullif(current_setting('app.tenant_id', TRUE), '')::uuid
    AND c."Name" = @name
ORDER BY c."Id"

SELECT c."Id", c."Name", c."TenantId"
FROM "Customers" AS c
WHERE FALSE
ORDER BY c."Id"
```

**Execution and materialization.** The command runs, and the shaper builds a `Customer` from every row; the tenant ID comes back through the value converter. Chapter 17 describes this way back.

## How often each stage runs

The pipeline has three points in time, and every mechanism belongs to exactly one of them. For the sample query, executed four times (twice with `"Contoso"`, once with null, once with `"Fabrikam"`), this gives:

| Point in time | Stages | In the sample |
|---|---|---|
| Once per query shape | Preprocessing through postprocessing; query interceptor, translators, compiling the shaper | once |
| Once per parameter constellation | Parameter-based processing and SQL generation, cached by what influenced the decisions (here: null or not null) | twice |
| On every execution | Funcletizer (extracts the parameter values), command, command interceptor, materialization | four times |

This leads to the most important rule for query extensions: values that change per execution (parameters, tenant, time) must appear in the SQL as a parameter or database function, never as a constant inserted at translation time. That is exactly why the plugin translates `CurrentTenant()` into `current_setting(...)` and does not insert the current tenant. With precompiled queries (chapter 16) the first point in time even moves into the build.

## The map

The following table assigns each stage the responsible EF component, the intervention a plugin has there, and the chapter that covers it. The grey labels follow the legend of the decision guide in chapter 4.

| Stage | Responsible | Intervention | Ch. |
|---|---|---|---|
| Funcletizer | `ExpressionTreeFuncletizer` (internal), asks `IEvaluatable ExpressionFilter` | `IEvaluatableExpression FilterPlugin` public · multiple · Singleton | 15 |
| Query cache | `ICompiledQueryCache` (internal, singleton) | No intervention | – |
| Before preprocessing | `QueryCompilationContext` at the start of compilation | `IQueryExpressionInterceptor` public · multiple · Singleton | 16, 30 |

| Stage | Responsible | Intervention | Ch. |
|---|---|---|---|
| Preprocessing | `IQueryTranslationPreprocessorFactory`; navigation expansion internal | Query filter in the model; otherwise replace a single service<br>public · single · Scoped | 10 |
| Queryable translation | `IQueryableMethodTranslatingExpressionVisitorFactory`; Npgsql's implementation internal | Replace a single service<br>provider base · single · Scoped | – |
| SQL translation | `IRelationalSqlTranslatingExpressionVisitorFactory`, `ISqlExpressionFactory`, translator providers | Method, member, aggregate translator plugins<br>public · multiple · Scoped | 14, 43 |
| Postprocessing | `IQueryTranslationPostprocessorFactory`; Npgsql's implementation internal | Replace a single service<br>provider base · single · Scoped | 16 |
| Shaper | `IShapedQueryCompilingExpressionVisitorFactory` | Through type mappings, constructor binding and `IMaterializationInterceptor` | 17 |
| Parameter-based | `IRelationalParameterBasedSqlProcessorFactory`, with `SqlNullabilityProcessor` inside; Npgsql internal | Replace a single service, needed for custom nodes<br>provider base · single · Scoped | 42 |
| SQL generation | `IQuerySqlGeneratorFactory`; `NpgsqlQuerySqlGenerator` internal | Replace a single service, needed for custom nodes<br>provider base · single · Singleton | 42 |
| Execution | Relational commands and connection | `IDbCommandInterceptor`<br>public · multiple · Scoped | 18 |
| Materialization | Compiled shaper, materializer | `IMaterializationInterceptor`<br>public · multiple · Singleton | 17 |

**Where provider-internal API begins**

Up to and including SQL translation, a plugin gets by with public plugin interfaces. The base classes of the later stages are public in EF (`RelationalQueryTranslationPostprocessor`, `RelationalParameterBasedSqlProcessor`, `QuerySqlGenerator`), but every provider derives its own classes from them, with Npgsql all of them in `Npgsql.EntityFrameworkCore.PostgreSQL.Query.Internal`. Replacing one of these single services means deriving from the provider's class to keep its behavior, and thus working against internal API (chapter 27).

## Six ways to intervene in a query

The mechanisms of the following chapters are easily confused because they all "change the query". They differ in what they see and when they run:

| Mechanism | Works on | Right for |
|---|---|---|
| Expression rewriting (`IQueryExpressionInterceptor`) | The LINQ tree after the funcletizer: parameters instead of values, no SQL yet | Adding conditions, replacing calls, rejecting operations; once per query shape |

| Mechanism | Works on | Right for |
|---|---|---|
| Method, member, aggregate translation | A single call, member or aggregate; builds from existing SQL building blocks | New functions in LINQ; `null` for everything that does not fit |
| Custom `SqlExpression` | A new node type in the `SelectExpression` | Syntax that is not a function call, or custom NULL semantics; must be known in three places: translator, parameter-based processing, generator |
| `RelationalParameterBasedSqlProcessor` | The finished `SelectExpression` *with* parameter values | Decisions that depend on a parameter's value, above all NULL semantics; cached per constellation |
| `QuerySqlGenerator` | The final tree, only writes text | The notation of a node; no longer makes decisions about content |
| Materialization (`IMaterializationInterceptor`) | The rows after execution | Checking, completing, replacing objects; does not change SQL |

Two confusions are particularly common. Rewriting conditions in the generator bypasses the NULL optimization that ran before and sees no parameter values. And trying to "translate" something with a query interceptor can only replace LINQ with other LINQ; only a translator creates an SQL node that does not exist yet.

## Locate first, then choose

A desired change to a query can almost always be assigned to a stage by one question. Only then is it worth choosing the mechanism in the decision guide (page 19).

| What should change? | Stage | Continue in |
|---|---|---|
| A value without data reference is evaluated in .NET too early | Funcletizer | chapter 15 |
| *Which* query is asked: an additional condition, a replaced call | Before preprocessing | Query filter (chapter 10) or chapter 16 |
| How a .NET call, member or aggregate looks in SQL | SQL translation | chapters 14, 43 |
| SQL needs a form that does not exist as a node | SQL translation through SQL generation | chapter 42 |
| The form depends on a parameter's value | Parameter-based | chapter 42 |
| Only the text: comment, hint, notation | SQL generation or execution | `TagWith`, chapter 18 |
| The returned objects | Materialization | chapter 17 |

> **Tip**
>
> Three tools are enough for locating: the debug log `QueryCompilationStarting` shows the tree after the funcletizer, `QueryExecutionPlanned` the `SelectExpression` including the shaper, and `ToQueryString()` the final SQL (appendix D). Where the difference between two of them shows, that is the stage.

# 14 Translating methods and members

Version dependency: **medium**

## The EF.Functions pattern

Database functions without a .NET counterpart are declared as extension methods on `DbFunctions`. They are never executed; the body only throws an exception in case someone calls them outside of LINQ. The declaration lives in the core package, the translation in the provider package:

TenancyDbFunctionsExtensions.cs (core package)

```
namespace Microsoft.EntityFrameworkCore;

public static class TenancyDbFunctionsExtensions
{
    public static TenantId CurrentTenant(this DbFunctions _)
        => throw new InvalidOperationException(
            $"{nameof(CurrentTenant)} can only be used in LINQ to Entities queries.");
}
```

## A method call translator

An `IMethodCallTranslator` receives every method call EF does not know itself. It checks whether it is responsible and then builds a SQL expression tree using `ISqlExpressionFactory`:

Query/CurrentTenantTranslator.cs (simplified)

```
public class CurrentTenantTranslator : IMethodCallTranslator
{
    private static readonly MethodInfo CurrentTenantMethod =
        typeof(TenancyDbFunctionsExtensions).GetRuntimeMethod(
            nameof(TenancyDbFunctionsExtensions.CurrentTenant), [typeof(DbFunctions)])!;

    private readonly ISqlExpressionFactory _sql;
    private readonly IRelationalTypeMappingSource _typeMappingSource;
    private readonly ITenancySingletonOptions _options;

    public CurrentTenantTranslator(
        ISqlExpressionFactory sql,
        IRelationalTypeMappingSource typeMappingSource,
        ITenancySingletonOptions options)
    {
        // ... ArgumentNullException.ThrowIfNull for all parameters ...
        _sql = sql;
        _typeMappingSource = typeMappingSource;
        _options = options;
    }

    public virtual SqlExpression? Translate(
        SqlExpression? instance, MethodInfo method,
        IReadOnlyList<SqlExpression> arguments,
        IDiagnosticsLogger<DbLoggerCategory.Query> logger)
```

```
    {
        if (method != CurrentTenantMethod)
            return null;                          // not responsible -> next translator

        var text = _typeMappingSource.FindMapping(typeof(string))!;
        var boolean = _typeMappingSource.FindMapping(typeof(bool))!;

        // Returns NULL although no argument is NULL: no argument propagates
        var currentSetting = _sql.Function(
            "current_setting",
            [
                _sql.Constant(_options.Settings.SettingName, text),
                _sql.Constant(true, boolean),
            ],
            nullable: true,
            argumentsPropagateNullability: [false, false],
            typeof(string), text);

        // Deliberately never NULL for EF, matching the CLR type TenantId (see below)
        var nullIfEmpty = _sql.Function(
            "nullif",
            [currentSetting, _sql.Constant(string.Empty, text)],
            nullable: false,
            argumentsPropagateNullability: [false, false],
            typeof(string), text);

        return _sql.Convert(nullIfEmpty, typeof(TenantId), TenantIdTypeMapping.Default);
    }
}
```

This turns `o.TenantId == EF.Functions.CurrentTenant()` into:

Generated SQL (excerpt)

```
WHERE o."TenantId" = nullif(current_setting('app.tenant_id', TRUE), '')::uuid
```

| Detail | Meaning |
|---|---|
| `return null` | Contract for “not responsible”. Never throw for foreign methods; the next translator gets its turn. |
| `arguments` | For extension methods, `arguments[0]` is the `this` parameter (here `DbFunctions`). |
| `nullable` | Can the result be NULL? EF uses this to optimize `IS NULL` checks and to produce C# NULL semantics. `CurrentTenant` deliberately says `false`: the CLR type `TenantId` is not nullable, and without a tenant every comparison should be NULL, so that no row matches. |
| `argumentsPropagateNullability` | “The result is NULL exactly when this argument is NULL.” If there are such arguments, EF derives the nullability from them alone. Functions that produce NULL themselves (`nullif`, `current_setting(…, true)`) therefore use `false` everywhere; `reverse(x)` or `upper(x)` use `true`. |
| Type mappings | Every SQL expression needs a mapping. It determines literals, parameters and casts. `Convert` with the TenantId mapping produces a cast that Npgsql writes as `::uuid`. |
| `Constant(...)` | Constants become literals via the mapping and are escaped correctly, including the setting name. |

The nullability details are the most treacherous part of a translator, because a mistake only shows in certain queries. An example is a function `EF.Functions.EmptyAsNull(c.Name)` that turns an empty string into NULL and returns `string?`:

Translator (WRONG)

```
_sql.Function("nullif", [arguments[1], _sql.Constant("", arguments[1].TypeMapping)],
    nullable: true,                                  // right: can be NULL ...
    argumentsPropagateNullability: [true, false],    // ... but this overrides it for EF
    typeof(string), arguments[1].TypeMapping);
```

**What happens if a translator declares the wrong nullability?**

**Observable.** `Where(c => EF.Functions.EmptyAsNull(c.Name) == null)` becomes `WHERE FALSE`: the query never finds customers with an empty name, without an error or a warning. With a nullable column as the argument the result would be right; the mistake only shows with required columns.

**Cause.** If a function has propagating arguments, EF's nullability processor determines its nullability from these alone. If the argument is not nullable (a required column, a constant), the function counts as never NULL, whatever `nullable` says.

**Solution.** An argument propagates only if "the argument is NULL" is the *only* thing that makes the result NULL. Functions that produce NULL themselves mark no argument. And `nullable` follows the CLR type: if it is nullable, EF may expect NULL; if it is not, as with `TenantId`, `false` is the honest statement.

**Test:** `FailureCaseTests.Nullable_function_…` with both variants; `QueryTranslationTests.Inequality_with_CurrentTenant_…` pins the safe direction.

## A member translator

For properties and fields there is `IMemberTranslator`. The sample translates `tenantId.IsEmpty`. The constant gets the column's type mapping so that the literal comes out in the right format:

Query/TenantIdMemberTranslator.cs

```
public class TenantIdMemberTranslator : IMemberTranslator
{
    private static readonly PropertyInfo IsEmptyProperty =
        typeof(TenantId).GetProperty(nameof(TenantId.IsEmpty))!;

    private readonly ISqlExpressionFactory _sql;

    public TenantIdMemberTranslator(ISqlExpressionFactory sql)
    {
        ArgumentNullException.ThrowIfNull(sql);

        _sql = sql;
    }

    public virtual SqlExpression? Translate(
        SqlExpression? instance, MemberInfo member, Type returnType,
        IDiagnosticsLogger<DbLoggerCategory.Query> logger)
    {
        if (instance is null || !Equals(member, IsEmptyProperty))
            return null;

        return _sql.Equal(instance, _sql.Constant(TenantId.Empty, instance.TypeMapping));
```

```
    }
}
```

## Translator plugins

Translators are not registered directly, but through plugin classes that return a list. The plugins are **scoped** (appendix A) and can therefore inject scoped services such as `ISqlExpressionFactory`:

Query/TenancyTranslatorPlugins.cs (simplified)

```
public class TenancyMethodCallTranslatorPlugin : IMethodCallTranslatorPlugin
{
    public TenancyMethodCallTranslatorPlugin(
        ISqlExpressionFactory sql,
        IRelationalTypeMappingSource typeMappingSource,
        ITenancySingletonOptions options)
    {
        Translators = [new CurrentTenantTranslator(sql, typeMappingSource, options)];
    }

    public virtual IEnumerable<IMethodCallTranslator> Translators { get; }
}

public class TenancyMemberTranslatorPlugin : IMemberTranslatorPlugin
{
    public TenancyMemberTranslatorPlugin(ISqlExpressionFactory sql)
    {
        Translators = [new TenantIdMemberTranslator(sql)];
    }

    public virtual IEnumerable<IMemberTranslator> Translators { get; }
}
```

> **Tip**
>
> `query.ToQueryString()` produces a query's SQL without executing it and without a database connection. It is the most important tool for developing and testing translators (chapter 26).

# 15 Preventing client-side evaluation

Version dependency: **stable**

The first stage of the pipeline, the funcletizer, evaluates every sub-expression in .NET up front that has no reference to query data. `o.Total > limit * 2` thus becomes a parameter. For `EF.Functions.CurrentTenant()` this would be fatal: the expression has no data reference, so EF would execute the method, which throws its exception.

EF knows only a few exceptions. In the source of `EvaluatableExpressionFilter` (EF Core 10) they are `DateTime.Now`, `DateTime.UtcNow`, `DateTime.Today`, their `DateTimeOffset` counterparts, `Guid.NewGuid()`, `Random.Next(...)` and all methods declared in EF's own class `DbFunctionsExtensions`. Custom EF.Functions methods live in a different class and are **not** covered. After that, EF asks all registered plugins:

Core package: TenancyDbFunctionsAttribute.cs, Query/TenancyEvaluatable…Plugin.cs

```
[AttributeUsage(AttributeTargets.Class, Inherited = false)]
public sealed class TenancyDbFunctionsAttribute : Attribute;

public class TenancyEvaluatableExpressionFilterPlugin : IEvaluatableExpressionFilterPlugin
{
    public virtual bool IsEvaluatableExpression(Expression expression)
        => expression is not MethodCallExpression { Method.DeclaringType: { } type }
           || !type.IsDefined(typeof(TenancyDbFunctionsAttribute), inherit: false);
}

[TenancyDbFunctions]
public static class TenancyDbFunctionsExtensions { /* CurrentTenant() */ }
```

The plugin lives in the **core package** and recognizes function classes by an attribute. It thus covers every provider package of the tenancy plugin, without the core having to know their classes. The standalone sample plugin from chapters 42 and 43 does not depend on the core package and therefore brings a filter of its own, which recognizes its function class by type.

Whether a custom EF.Functions method needs this plugin depends on whether it has arguments referencing data. `EF.Functions.Soundex(c.Name)` is harmless because `c.Name` points to a column. Methods without arguments or only with constants (`CurrentTenant()`, `Random()`, `ServerTime()`) always need it:

Custom function class (WRONG)

```
public static class ShopDbFunctionsExtensions          // without [TenancyDbFunctions]
{
    public static DateTime ServerTime(this DbFunctions _)
        => throw new InvalidOperationException("Only in LINQ queries.");
}
```

**What happens if the evaluatable filter is missing?**

**Observable.** Every query with the function throws `InvalidOperationException`: "An exception was thrown while attempting to evaluate a LINQ query parameter expression", with the message from the method body inside. It looks as if the method were called outside of LINQ, although the translator is

registered and correct.

**Cause.** The funcletizer runs before any translator and evaluates expressions without data reference in .NET. EF only knows exceptions for its own class `DbFunctionsExtensions`.

**Solution.** An `IEvaluatableExpressionFilterPlugin`, in the sample through the attribute on the function class.

**Test:** `FailureCaseTests.Function_without_data_reference_…`, with and without the attribute.

# 16 Further query extension points

Version dependency: **medium**

The sample needs method and member translators. Complex plugins use further points; here is an overview with classification:

| Extension point | Purpose | Notes |
|---|---|---|
| `IAggregateMethodCallTranslatorPlugin` | Custom aggregate functions in `GroupBy` (e.g. `bool_and`, statistics functions) | Scoped, multiple registrations; implemented in chapter 43 |
| `IQueryExpressionInterceptor` | Rewrite the LINQ expression tree before translation | Singleton interceptor; register it in the internal provider as `IInterceptor` (chapter 30); ideal for query rewrites, tags, hints |
| `IMaterializationInterceptor` | Create, initialize or check objects during materialization | Singleton interceptor; must be stateless and thread-safe; implemented in chapter 17 |
| `IMethodCallTranslatorProvider` etc. | Replace translator infrastructure | Single provider services; only in exceptional cases |
| `IQuerySqlGeneratorFactory` | SQL generation for custom SqlExpression types | Singleton, provider service; implemented in chapter 42 |
| `IRelationalParameterBasedSqlProcessorFactory` | Nullability processing of custom expression nodes | Scoped, provider service; implemented in chapter 42 |
| `IQueryTranslationPostprocessorFactory` | Adjust the expression tree after translation | Provider service; deep intervention, tightly bound to EF internals |

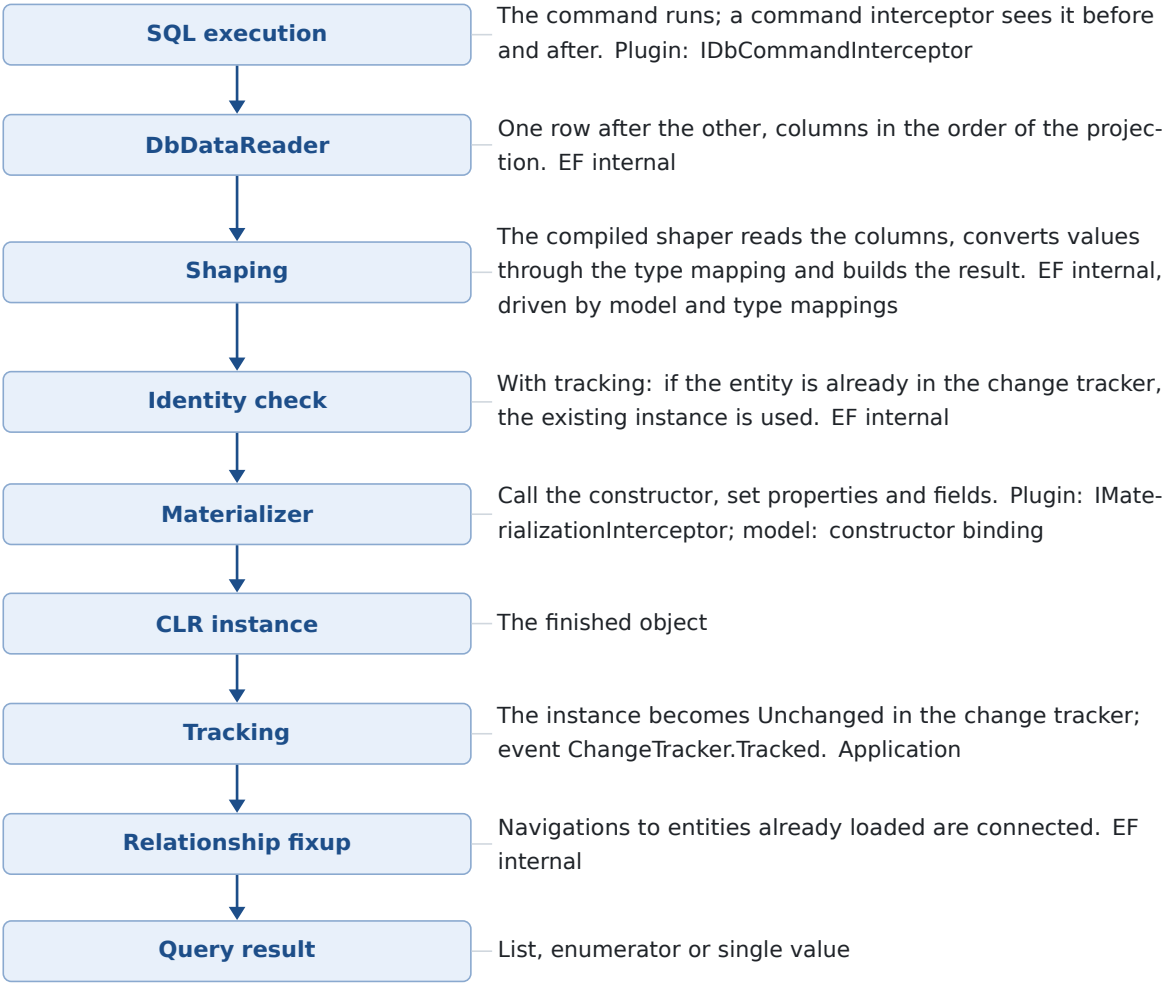
> **Precompiled queries**
>
> With NativeAOT and precompiled queries (since EF Core 9), EF translates queries at build time. Translators then run in the build process, not at run time. They may therefore only depend on options and model, never on runtime state. The sample meets this: the setting name is an option; the tenant itself is only evaluated in the database session.

# 17 Materialization: from rows to objects

Version dependency: **stable**

The previous chapters of this part describe the way there: LINQ becomes SQL. Chapter 32 describes the way from entities to INSERT, UPDATE and DELETE. This chapter closes the circle and describes the way back: how do the rows the database returns become objects, and where can a plugin intervene?

## The way back at a glance




The identity check comes *before* the materializer: an entity the context already tracks is not created a second time. The following overview shows where a plugin can intervene and what EF keeps to itself:

| Stage | Intervention for plugins | EF internal |
|---|---|---|
| Execution | `IDbCommandInterceptor` can change the command or supply its own `DbDataReader` (chapter 18) | Connection, execution strategy, retries |

| Stage | Intervention for plugins | EF internal |
|---|---|---|
| Shaping | Indirectly: type mapping and value converter determine how values are read and converted (chapter 12) | Building and compiling the shaper |
| Materializer | `IMaterializationInterceptor` with four hooks; constructor binding through the model | How EF combines constructor and values |
| Tracking | The change tracker's `Tracked` and `StateChanged` events (application) | Identity map, states, snapshots for change detection |
| Fixup | No hook | Navigation fixer |

## Shaping: the compiled way back

Translating a query produces not only SQL but also a **shaper**: an expression tree that describes how a row of the `DbDataReader` becomes the result. EF compiles it into a delegate and caches it together with the SQL per query shape (chapter 13). Per row, only this delegate runs, without looking anything up in the model.

This has an important consequence: everything that shapes the shaper is decided at translation time, not at execution time. That includes the type mapping's reader method, the value converter, the chosen constructor and the registered materialization interceptors. For `TenantId`, the shaper reads the provider value with `GetGuid` and calls the compiled conversion delegate of the `TenantIdConverter` (`ConvertFromProviderTyped`). Values in JSON columns are read through the mapping's `JsonValueReaderWriter`. Apart from the converter itself, no plugin code runs for this; that is why a value converter is cheaper for pure value conversion than an interceptor that is called for every entity.

A projection such as `Select(o => o.Total)` produces a shaper that only reads that column. No entity is created and no materializer runs.

## The materializer: instance, constructor, fields

For an entity, the shaper first creates an instance and then sets the remaining values. Which constructor it uses is determined by the model (constructor binding):

| Rule | Behavior in EF Core 10 |
|---|---|
| Candidates | All declared instance constructors, whatever their accessibility. |
| Bindable | Every parameter matches a mapped property by name and type, or a service. EF does not set properties bound through the constructor a second time. |
| Services as parameters | `DbContext` (also the derived type), `ILazyLoader`, the lazy-loading delegate `Action<object, string>` and `IEntityType`; more through custom `IParameterBindingFactory` plugins. |
| Selection | EF goes through the bindable constructors in order. A constructor replaces the current favorite if it has more service parameters or fewer property parameters; a tie in both stops model building. A parameterless constructor therefore wins against property constructors, but not against one with a `DbContext` parameter. If constructors mix services and properties, the order of declaration can decide. |

| Rule | Behavior in EF Core 10 |
|---|---|
| Other properties | EF writes them through the backing field if there is one. Logic in the setter therefore does *not* run when loading. |
| Shadow properties | Have no field; with tracking their values live in the change tracker. |
| Owned types | Are entity types and are materialized individually. |
| Complex types | Belong to the entity and are materialized with it, without a pass of their own. |

The selection is implemented in `ConstructorBindingFactory` (EF Core 10.0.0). `ConstructorBindingTests` records both cases: without services the parameterless constructor wins, next to a constructor with a `DbContext` it loses.

That setters do not run when loading is intentional: loading is not creating. Checks that should apply when creating or changing belong in the domain's constructors and methods. What holds when loading is guaranteed by the database.

## Tracking, identity resolution and fixup

What happens after the materializer depends on the query's tracking behavior:

| Mode | Identity | Fixup |
|---|---|---|
| Tracking (default) | Before the materializer, EF looks up the key in the change tracker. On a hit it returns the existing instance unchanged: local changes stay, new values from the database are discarded. | New instances become Unchanged; EF connects navigations to all tracked entities, including those from earlier queries. |
| `AsNoTracking` | Every row yields a new instance, even the same entity several times. | Only within the result: the shaper connects what was loaded with `Include`. |
| `AsNoTrackingWithIdentityResolution` | An identity map for this query only: the same entity is materialized once. | As without tracking; afterwards nobody tracks the instances. |

**Relationship fixup** is EF internal. A plugin cannot influence it; an application can observe it through `ChangeTracker.Tracked` and `StateChanged`.

> **IIdentityResolutionInterceptor is not a materialization hook**
>
> The name suggests otherwise, but `IIdentityResolutionInterceptor` (scoped, multiple) never runs for queries. It applies when `Attach`, `Update` or `Add` want to attach an instance with a key the context already tracks, and decides what happens to the tracked instance. EF ships `IgnoringIdentityResolutionInterceptor` and `UpdatingIdentityResolutionInterceptor` for this. Queries resolve identity without it: with tracking through the change tracker, otherwise as in the table above.

## IMaterializationInterceptor in detail

`IMaterializationInterceptor` has four hooks that EF calls for every entity in this order. All of them have a default implementation; a plugin only overrides what it needs.

| Method | When | Options |
|---|---|---|
| `CreatingInstance` | Before the constructor | Read the row's values, including shadow properties; supply your own instance with `SuppressWithResult` |
| `CreatedInstance` | After the constructor | Values set through the constructor are present; return a different instance |
| `InitializingInstance` | Before the remaining values are set | Take over setting them with `Suppress` |
| `InitializedInstance` | All values set | Check, complete or return a different instance |

Every method receives `MaterializationInterceptionData`: the context, the `IEntityType`, the query's tracking behavior and `GetPropertyValue` for the row's values. Because the type is an `IEntityType`, complex types do not pass through the interceptor individually, while owned types do. The struct's constructor is EF internal; tests therefore do not call the hooks but the logic behind them (see below).

> **Lifetime: a singleton, compiled into the shaper**
>
> The interceptor is an `ISingletonInterceptor`. EF builds it into the shaper at translation time, and the shaper is cached per internal service provider. One instance therefore serves all contexts and all threads at the same time; it must be stateless and thread-safe. It gets the data of a request on every call through `materializationData.Context`.
>
> Singleton interceptor instances are also part of the internal provider's cache key: a new instance per context through `AddInterceptors` creates a new internal provider including model for every context (consequences in chapter 5). A plugin therefore registers it in `ApplyServices`; an application always passes the same instance.

## Example: checking tenants while loading

Chapter 18 shows a real gap: if the application passes a connection that is already open, the connection interceptor does not run, and a query silently returns the rows of the tenant whose value is still in the session. The same happens if the application accidentally connects with a role that bypasses RLS. The plugin offers an optional check for this that sits exactly in the right place: every tenant-owned entity a query materializes must belong to the context's tenant.

Application

```
options.UseNpgsql(connectionString)
       .UseAcmeTenancy(sp, tenancy => tenancy.VerifyMaterializedTenants());
```

Interceptors/TenantMaterializationInterceptor.cs (core package, shortened, simplified)

```
public class TenantMaterializationInterceptor : IMaterializationInterceptor
{
    // Provider per context instance, held weakly: goes away with the context
    private static readonly ConditionalWeakTable<DbContext, ITenantProvider> Providers =
        new();

    public virtual InterceptionResult<object> CreatingInstance(
        MaterializationInterceptionData materializationData,
        InterceptionResult<object> result)
    {
```

```
        Verify(materializationData.Context, materializationData.EntityType,
            materializationData.GetPropertyValue);

        return result;
    }

    internal static void Verify(
        DbContext context, IEntityType entityType, Func<string, object?> readValue)
    {
        // Runtime annotation of the isolation convention; without it, no isolation
        if (entityType.GetTenantPropertyName() is not { } propertyName)
            return;

        // The tenant is read for every row: this instance is shared by all contexts, every
        // context can have a different tenant, and a pooled context changes it between leases
        var current = FindTenantProvider(context).CurrentTenantId;
        var value = (TenantId)readValue(propertyName)!;

        if (value != current)
            throw new TenancyException("... isolation did not take effect ...");
    }

    // Once per context through its interceptors (like ApplyTenantSession) to the
    // ITenantProvider of the SaveChanges interceptor, from the table afterwards
    internal static ITenantProvider FindTenantProvider(DbContext context)
        => Providers.GetValue(context, LookUpTenantProvider);
}
```

Infrastructure/TenancyOptionsExtension.cs › ApplyServices (addition)

```
// Optional materialization check (chapter "Materialization: from rows to objects"). A
// singleton interceptor belongs in the internal provider: one instance per provider,
// never a new instance per context. The setting is part of the hash.
if (Settings.VerifyMaterializedTenants)
    new EntityFrameworkServicesBuilder(services)
        .TryAdd<ISingletonInterceptor, TenantMaterializationInterceptor>();
```

| Decision | Reason |
|---|---|
| `CreatingInstance` instead of `InitializedInstance` | The check reads the value from the row. A foreign row never becomes an object, and the tenant property may be a shadow property. |
| Runtime annotation | Which entity is isolated and what its tenant property is called has been on the entity type since model building (chapter 44). No guessing at run time. |
| Tenant per call | Through the context to this context's `ITenantProvider`; the provider is cached per context, the tenant is read and compared per entity. That still costs per row, which is why the check is optional. |
| No tenant | Without a tenant, RLS returns no rows. If one arrives anyway, RLS is not in effect and the check fires. |
| In the core package | The check knows neither SQL nor provider types (chapter 46); all three provider packages activate it with one line. |

The integration tests reproduce the gap from chapter 18: a context for tenant A over a connection whose session still carries tenant B. Without the check, the query returns B's rows; with the check, it

throws a `TenancyException`. Own rows pass with and without tracking and through `FromSql`.

## The failure case: request data in a singleton

The obvious first design passes the tenant to the interceptor's constructor and creates it per context like the other interceptors. That fails because of the cache key: every context gets an internal provider of its own. The obvious second design shares one instance and remembers the tenant on the first call to save the lookup per row:

tests/...IntegrationTests/MaterializationTests.cs (WRONG, shortened)

```
private sealed class RememberingTenantInterceptor : IMaterializationInterceptor
{
    private TenantId? _tenant;                      // per-request state in a singleton

    public InterceptionResult<object> CreatingInstance(
        MaterializationInterceptionData materializationData,
        InterceptionResult<object> result)
    {
        // "Optimization": look up the tenant only once
        _tenant ??= _tenantOf(materializationData.Context);

        if (materializationData.GetPropertyValue<TenantId>(nameof(Order.TenantId)) != _tenant)
            throw new TenancyException("Row of another tenant.");

        return result;
    }
}
```

The test shows both consequences. The second request for tenant B gets an error for its own rows, because the interceptor remembered tenant A. Worse: a context for B over a connection whose session still carries A returns A's rows, and the remembered tenant A lets them through. The check does not prevent the leak, it hides it. Under load there is also a race condition, because all threads write the same field.

> **Tip**
>
> A singleton interceptor has only two sources of data: what belongs to the configuration (through the constructor or singleton options, the same for all contexts) and what the call brings along (`Context`, entity type, the row's values). Everything else belongs in the application or in a scoped service the interceptor reaches through the context (chapter 8).

The right optimization does not remember the tenant but the way to it. Finding the `ITenantProvider` for every row through `GetService<IDbContextOptions>`, the extension and a LINQ search in the interceptors costs the same thing a thousand times for a thousand rows. The sample therefore keeps the provider in a static `ConditionalWeakTable<DbContext, ITenantProvider>`. It holds its keys weakly: a context that is released disappears together with its entry, without the interceptor learning about it. With pooling the entry stays correct across leases, because a context keeps its options and therefore its interceptors for its whole life; the tenant itself is still read on every row. If the lookup fails, the table stores nothing, and the configuration error stays visible on every call. The unit tests in `MaterializationTests` check repeated rows in one context, contexts with different tenants and a pooled context across two leases.

## Limits

| Limit | Consequence |
|---|---|
| Entities only | Projections of individual columns or into DTOs do not pass through the materializer. An integration test shows that `Select(o => o.Total)` returns values of another tenant unchecked; only the database can protect here. |
| No tracked entities | If an entity is already tracked, it is not materialized again; the interceptor only sees the first query. |
| No filter | The interceptor cannot skip rows, only throw or replace the instance. Filtering is the job of query filters or the database. |
| No SQL | It sees the result, not the query; `ExecuteUpdate`, `ExecuteDelete` and scalar queries materialize nothing. |
| Hot path | It runs for every entity of every query (chapter 39); expensive work multiplies with the size of the result. |
| An exception aborts the query | All other rows are lost as well. That is right for a security check, not for a convenience feature. |

## When materialization, when something else?

The interceptor is the right place for cross-cutting tasks that concern *every* loaded entity, across all queries, with and without tracking, even with raw SQL. For almost everything else there is a simpler and cheaper way:

| Goal | Better | Why |
|---|---|---|
| Store a value differently than it is presented (format, type, encryption) | Value converter | Works in both directions and in parameters, directly in the shaper instead of through an interceptor |
| Immutable objects, invariants | Constructor binding | EF calls the constructor itself |
| Only some columns or a DTO | Projection with `Select` | No entity, no tracking, less data |
| The same setting for many types (converter, field access) | Convention | Once during model building instead of for every row |
| Derived values, business rules | Domain logic (method, computed property) | Loading is not a business event |
| React when an application loads entities | `ChangeTracker.Tracked` | Public API without a plugin; only with tracking |
| Check or complete every loaded entity, even without tracking | `IMaterialization Interceptor` | The only hook that sees every way to the entity |

Part V

# Run time

Intervening when saving, connecting and executing, and reporting events properly.


› Chapter 18: Interceptors

› Chapter 19: Diagnostics and logging

# 18 Interceptors

Version dependency: **stable**

Interceptors are hooks into EF's run time: before and after commands, connections, transactions, SaveChanges and materialization. They can observe, change values, suppress operations or replace results.

| Interface | Events | Lifetime |
|---|---|---|
| `ISaveChangesInterceptor` | SavingChanges, SavedChanges, SaveChangesFailed, ThrowingConcurrencyException | Scoped |
| `IDbConnectionInterceptor` | Opening, closing, creating the connection | Scoped |
| `IDbCommandInterceptor` | Creating and executing commands, data readers | Scoped |
| `IDbTransactionInterceptor` | Begin, commit, rollback, savepoints | Scoped |
| `IIdentityResolutionInterceptor` | Conflicts when attaching already tracked entities; never for queries (chapter 17) | Scoped |
| `IMaterializationInterceptor` | Creating and initializing entities (chapter 17) | **Singleton** |
| `IQueryExpressionInterceptor` | LINQ expression before compilation | **Singleton** |

The two singleton interceptors implement `ISingletonInterceptor`. They are used once per internal provider, not per context, and must therefore be stateless and thread-safe. Registered in the internal provider, however, EF only finds the materialization interceptor under `ISingletonInterceptor`; it calls a query expression interceptor only when it is registered as `IInterceptor` (chapter 30).

## Two ways to register

| Way | Properties | When |
|---|---|---|
| `optionsBuilder.AddInterceptors(...)` | Instances from the application; can use app services; differ per context | Interceptors with app dependencies (tenant, user, clock) |
| `EntityFrameworkServicesBuilder.TryAdd<IInterceptor, X>()` in `ApplyServices` | Resolved from the internal provider ("injected interceptors"); have access to EF services, not to app services | Purely technical interceptors without app dependencies |

EF combines both ways into one list. The SaveChanges and the connection interceptor need the application's tenant and therefore come the first way (chapter 7). The optional interceptors for materialization, bulk operations and tracing are registered by the plugin in the internal provider.

## The SaveChanges interceptor

It sets the tenant of new entities and prevents data of another tenant from being written or reassigned. This complements RLS (defense in depth) and produces clearer errors than the database. Chapter 30 shows which paths bypass it.

Interceptors/TenantSaveChangesInterceptor.cs (1/2)

```
public class TenantSaveChangesInterceptor : SaveChangesInterceptor
{
    private readonly ITenantProvider _tenants;

    public TenantSaveChangesInterceptor(ITenantProvider tenants)
    {
        ArgumentNullException.ThrowIfNull(tenants);

        _tenants = tenants;
    }

    public override InterceptionResult<int> SavingChanges(
        DbContextEventData eventData, InterceptionResult<int> result)
    {
        if (eventData.Context is { } context) ApplyTenant(context);
        return result;
    }

    public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData, InterceptionResult<int> result,
        CancellationToken cancellationToken = default)
    {
        if (eventData.Context is { } context) ApplyTenant(context);
        return ValueTask.FromResult(result);
    }
```

Interceptors/TenantSaveChangesInterceptor.cs (2/2, simplified)

```
    internal void ApplyTenant(DbContext context)
    {
        // Which entities are affected is stored in the runtime annotation
        // (chapter "Property-level Fluent API and runtime annotations")
        var changes = context.ChangeTracker.Entries()
            .Where(e => e.State is EntityState.Added or EntityState.Modified
                                or EntityState.Deleted)
            .Select(e => (Entry: e, Name: e.Metadata.GetTenantPropertyName()))
            .Where(x => x.Name is not null)
            .Select(x => (x.Entry, Property: x.Entry.Property(x.Name!)))
            .ToList();
        if (changes.Count == 0) return;

        var current = _tenants.CurrentTenantId
            ?? throw new TenancyException("No active tenant ...");

        foreach (var (entry, property) in changes)
        {
            var currentValue = (TenantId)property.CurrentValue!;
            var originalValue = (TenantId)property.OriginalValue!;
            switch (entry.State)
            {
                case EntityState.Added when currentValue.IsEmpty:
                    property.CurrentValue = current;
                    break;
                case EntityState.Added when currentValue != current:
                    throw Mismatch(entry, currentValue, current);
                // Only an actual change of the value counts, not the flag
```

```
                case EntityState.Modified
                    when property.IsModified && currentValue != originalValue:
                    throw new TenancyException("The tenant property is immutable.");
                case EntityState.Modified or EntityState.Deleted:
                    if (entry.State == EntityState.Modified)
                        property.IsModified = false;    // do not write the column
                    // Empty = unknown (shadow property): the database decides
                    if (!originalValue.IsEmpty && originalValue != current)
                        throw Mismatch(entry, originalValue, current);
                    break;
            }
        }
    }
}
```

Anyone who only works with asynchronous code easily overrides only one variant:

`Interceptor (WRONG)`

```
public class TenantSaveChangesInterceptor : SaveChangesInterceptor
{
    public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData, InterceptionResult<int> result,
        CancellationToken cancellationToken = default) { /* apply the tenant */ }

    // SavingChanges is missing
}
```

**What happens if only the asynchronous variant is implemented?**

**Observable.** With `SaveChangesAsync` everything works. A synchronous `SaveChanges`, for example in a console tool or an older code path, saves without a tenant; RLS rejects the row with `WITH CHECK`, a plugin without database protection silently writes it wrongly.

**Cause.** Depending on the call, EF calls exactly one variant. The base class `SaveChangesInterceptor` implements the other one as an empty method, so there is no error. The same applies to connection and command interceptors.

**Solution.** Always override both variants and call the same logic, like `ApplyTenant` in the sample.

**Test:** `FailureCaseTests.Interceptor_with_only_the_async_variant_…`.

## Tracking flag, change of value and database protection

Three things look alike when saving and mean different things:

- **The tracking flag.** `DbContext.Update` and `Attach` of an untracked entity set `IsModified` for *every* property, the tenant property included. Original and current value are the same: the value the entity brings along. Up to 1.0 the interceptor already rejected this flag as a change of tenant, and an ordinary update from an API failed.
- **The actual change of value.** A row is only reassigned when the value differs from the original value. The interceptor rejects that, even when the new value is the active tenant. For an unchanged value it clears the flag, so that the `UPDATE` does not write the tenant column.
- **The state of the database.** For a tracked entity the original value comes from the database. For a detached entity it is only a claim of the caller. A forged entity with the own tenant and the ID of a foreign row therefore passes every check in the interceptor. Only the policy decides:

> the `UPDATE` does not see the foreign row, changes zero rows, and EF reports a `DbUpdateConcurrencyException`. An empty original value (a shadow property the entity cannot carry) counts as unknown, not as foreign; here, too, the database decides.

The interceptor therefore only promises what it can know without accessing the database: no change of tenant in the change tracker, no values known to be foreign, no change without an active tenant. Whether a row really belongs to the tenant is checked by RLS alone. `SaveChangesInterceptorTests` covers the cases without a database, `DetachedUpdateTests` against PostgreSQL with a restricted role: `Update` with an unchanged tenant, synchronous and asynchronous, the forged foreign row ID and the reassignment of a tracked row.

The logic lives in an `internal` method `ApplyTenant`. Tests can call it directly through `InternalsVisibleTo` without needing a database (chapter 26).

## The connection interceptor

It sets the session setting that the policies and `CurrentTenant()` evaluate every time a connection is opened. The flow (intercept opening, sync and async, fetch the tenant) is the same for every database and therefore lives in an abstract base in the relational package (chapter 46). The PostgreSQL package only supplies the command. It gets the setting name from the *internal* provider via the context from the event data (`context.GetService<T>()`), and the tenant from the *app* container:

Interceptors/TenantSessionInterceptor.cs (relational), Runtime/TenantConnectionInterceptor.cs (simplified)

```
// Relational package
public abstract class TenantSessionInterceptor : DbConnectionInterceptor
{
    protected TenantSessionInterceptor(ITenantProvider tenants)
    {
        ArgumentNullException.ThrowIfNull(tenants);

        Tenants = tenants;
    }

    protected ITenantProvider Tenants { get; }

    public override async Task ConnectionOpenedAsync(
        DbConnection connection, ConnectionEndEventData eventData,
        CancellationToken cancellationToken = default)
    {
        await using var command =
            CreateSessionCommand(connection, eventData.Context, Tenants.CurrentTenantId);
        await command.ExecuteNonQueryAsync(cancellationToken);
    }

    // ConnectionOpened (synchronous) analogous

    protected abstract DbCommand CreateSessionCommand(
        DbConnection connection, DbContext? context, TenantId? tenant);
}

// PostgreSQL package
public class TenantConnectionInterceptor : TenantSessionInterceptor
{
```

```
    public TenantConnectionInterceptor(ITenantProvider tenants)
        : base(tenants)
    {
    }

    protected override DbCommand CreateSessionCommand(
        DbConnection connection, DbContext? context, TenantId? tenant)
    {
        var settingName = context?.GetService<ITenancySingletonOptions>()
                              .Settings.SettingName
                          ?? TenancySettings.Default.SettingName;

        var command = connection.CreateCommand();
        command.CommandText = "SELECT set_config(@name, @value, false)";
        AddParameter(command, "name", settingName);
        // Always set, even empty: no stale value remains
        AddParameter(command, "value", tenant?.ToString() ?? string.Empty);
        return command;
    }
}
```

> **Empty string instead of NULL**
>
> Npgsql resets the session state when a connection returns to the pool. After that, `current_setting('app.tenant_id', true)` returns an empty string instead of `NULL`. A `''::uuid` would abort every query. Policy and translator therefore use `nullif(..., '')`: without a tenant the value is `NULL`, the comparison is never true and no rows are visible. That is the safe direction.

By default EF opens and closes the connection for every query. Every opening costs an extra round trip here. With many small queries it pays off to open the connection once per request explicitly (chapter 39).

## Where the connection interceptor does not run

The interceptor runs when EF opens a connection, and only then. Two situations bypass it, and both are easy to create by accident:

| Situation | Consequence | Remedy |
|---|---|---|
| An already open connection is passed in (`UseNpgsql(connection)`, `SetDbConnection`) | EF never opens it, so the interceptor never runs. The session keeps whatever value it has: none, or that of a previous tenant | Pass connections closed and let EF open them; otherwise call `Database.ApplyTenantSession()` |
| The connection stays open (`OpenConnection`, an open transaction) while the tenant changes | The new tenant only takes effect when the connection is opened again | One context, one tenant; if that is not possible, call `Database.ApplyTenantSession()` after the change |

`tests/...IntegrationTests/ConnectionLifetimeTests.cs (shortened, simplified)`

```
// Already open, with a session value left over from earlier work (tenant B)
await using var connection = new NpgsqlConnection(appConnectionString);
await connection.OpenAsync();
// ... SELECT set_config('app.tenant_id', <tenant B>, false) ...
```

```
// Meant for tenant A, but EF does not open the connection: the interceptor never runs
await using var db = CreateAppContext(connection, tenantA);
var orders = await db.Orders.ToListAsync();       // the rows of tenant B!
```

For these cases the plugin offers the explicit counterpart to the interceptor. `Database.ApplyTenantSession()` writes the current tenant into the session if the connection is open, and does nothing if it is closed, because EF will then open it and the interceptor takes over. It reuses the interceptor's own command, so it works for every provider package:

Usage

```
await using var db = CreateAppContext(connection, tenantA);   // connection already open
await db.Database.ApplyTenantSessionAsync();                   // session now: tenant A
var orders = await db.Orders.ToListAsync();                    // the rows of tenant A
```

Interceptors/TenantSessionInterceptor.cs (excerpt)

```
public virtual async Task<bool> ApplyToOpenConnectionAsync(
    DbContext context, CancellationToken cancellationToken = default)
{
    var connection = context.Database.GetDbConnection();
    if (connection.State != ConnectionState.Open)
        return false;                     // EF opens it later; the interceptor takes over

    await using var command =
        CreateSessionCommand(connection, context, Tenants.CurrentTenantId);
    // Inside an explicit transaction the command must be enlisted in it
    command.Transaction = context.Database.CurrentTransaction?.GetDbTransaction();
    await command.ExecuteNonQueryAsync(cancellationToken);
    return true;
}
```

The extension method `ApplyTenantSession` in the relational package finds the interceptor among the context's interceptors and fails with a clear message if there is none. Two details matter: the method needs the context, not just the connection, because the provider reads its settings from the internal service provider; that is why `CreateSessionCommand` receives the `DbContext` instead of the event data. And the command must join a running transaction: PostgreSQL is lenient here, SQL Server rejects a command that is not enlisted. The SQL Server tests check exactly this case; without the line they fail.

In `ConnectionOpened` this assignment is missing on purpose. A local transaction requires an open connection, and EF opens the connection before it calls `BeginTransaction`; when the connection is opened there is no transaction yet. An ambient `TransactionScope`, in turn, is joined by Npgsql and SqlClient themselves when the connection opens, and every command on the connection runs in it without a `DbTransaction` being assigned. `TransactionContextTests` and the SQL Server tests check all three paths: the connection that `BeginTransactionAsync` opens, `ApplyTenantSession` in an explicit transaction, and reading and writing under a `TransactionScope`.

> **One context, one tenant**
>
> Isolation follows the connection, not the context. A context used for several tenants, or a connection that outlives a request, carries the tenant of the moment it was opened. `ConnectionLifetimeTests` shows both cases against a real database, together with the two remedies: a closed connection that EF opens itself, and `ApplyTenantSession()`. A plugin whose hooks cannot cover every path should offer such an

> explicit API instead of only documenting the gap.

The gap can also be detected while loading: with `VerifyMaterializedTenants()` the plugin checks every loaded entity against the context's tenant and throws instead of returning another tenant's rows (chapter 17).

## Dapper and raw ADO.NET

Reports and bulk exports often run through Dapper or raw ADO.NET on the context's connection (`Database.GetDbConnection()`). Row level security applies to these commands as well, because it lives in the database, not in EF. What matters is only whether the session value is set, and that depends on who opens the connection:

| Who opens the connection | Result |
|---|---|
| EF, for example `OpenConnectionAsync()` or an open transaction | The interceptor has set the session; Dapper sees the rows of the current tenant |
| Dapper or ADO.NET itself, also after EF closed the connection again after a query | No session value: the policies match no rows (fail closed) until `ApplyTenantSessionAsync()` sets it |
| With transaction pooling (`UseTransactionPooling`) | The tenant exists only inside EF's transaction; Dapper must run in it |

The second row is the trap: because EF closes the connection again after every query, a connection that just carried the tenant is closed afterwards. Dapper opens a closed connection itself, and the interceptor does not run. The safe pattern opens the connection through EF for the whole block:

Usage (Dapper)

```
await db.Database.OpenConnectionAsync();                // EF opens: session set
var connection = db.Database.GetDbConnection();
var orders = await connection.QueryAsync<OrderDto>(
    "SELECT \"Id\", \"Total\" FROM \"Orders\"");             // RLS: the current tenant only
await db.Database.CloseConnectionAsync();

// PgBouncer in transaction mode: the tenant exists only inside EF's transaction
await using var tx = await db.Database.BeginTransactionAsync();
var rows = await connection.QueryAsync<OrderDto>(sql, transaction: tx.GetDbTransaction());
```

tests/.../RawAdoNetTests.cs (excerpt)

```
var tenant = await CreateTenantWithOrdersAsync(2);
await using var db = _fixture.CreateAppContext(
    new MutableTenantProvider { CurrentTenantId = tenant });
await db.Orders.CountAsync();                 // EF opened and closed the connection again

// Opened directly, as Dapper does with a closed connection: EF is not involved
var connection = db.Database.GetDbConnection();
await connection.OpenAsync();
Assert.Equal(0, await CountOrdersAsync(connection));      // fail closed

await db.Database.ApplyTenantSessionAsync();
Assert.Equal(2, await CountOrdersAsync(connection));
```

`RawAdoNetTests` checks all three rows against PostgreSQL with a plain command, which is all Dapper sends. The result is never a leak: without a session value the query sees nothing. It is a wrong empty result, and that is why the pattern belongs in the application’s guidelines.

# 19 Diagnostics and logging

Version dependency: **stable**

A professional plugin reports events through EF's logging system. Messages then end up in the same channels users configure anyway (`ILogger`, `LogTo`, `DiagnosticSource`), and warnings can be ignored, logged or turned into errors via `ConfigureWarnings`.

## Event IDs

Diagnostics/TenancyEventId.cs (shortened)

```
public static class TenancyEventId
{
    public const int TenancyBaseId = 1_150_000;     // own, documented range

    private enum Id
    {
        TenantIsolationApplied = TenancyBaseId,
        TenantIsolationOptedOut,
        PoliciesWithoutRowLevelSecurity,
    }

    private static readonly string ModelPrefix = "Acme.EntityFrameworkCore.Tenancy.Model.";

    public static readonly EventId TenantIsolationApplied =
        Make(Id.TenantIsolationApplied);
    public static readonly EventId TenantIsolationOptedOut =
        Make(Id.TenantIsolationOptedOut);
    // ...

    private static EventId Make(Id id) => new((int)id, ModelPrefix + id);
}
```

EF and providers occupy low ranges. A plugin chooses its own high range and documents it. Names follow EF's scheme "Assembly.Category.Event" so that prefix filters work.

## Event definitions

An `EventDefinition<T>` bundles event ID, log level, message template and the warning behavior from the options. EF caches its definitions per internal provider; the plugin does the same with its own class registered as a singleton (chapter 6):

Diagnostics/TenancyLoggingDefinitions.cs (excerpt)

```
public class TenancyLoggingDefinitions
{
    private EventDefinition<string>? _tenantIsolationOptedOut;

    public virtual EventDefinition<string> LogTenantIsolationOptedOut(
        IDiagnosticsLogger logger)
        => LazyInitializer.EnsureInitialized(ref _tenantIsolationOptedOut, () =>
            Create(logger, TenancyEventId.TenantIsolationOptedOut, LogLevel.Warning,
```

```
                nameof(TenancyEventId.TenantIsolationOptedOut),
                "Entity '{entityType}' implements ITenantOwned but is excluded from "
                + "tenant isolation."));

    private static EventDefinition<string> Create(
        IDiagnosticsLogger logger, EventId eventId, LogLevel level,
        string name, string message)
        => new(
            logger.Options,                      // contains the warning behavior
            eventId,
            level,
            "TenancyEventId." + name,            // appears in warning-as-error messages
            l => LoggerMessage.Define<string>(l, eventId, message));
}
```

## The logger extension

The actual log method follows the pattern of EF's `CoreLoggerExtensions` exactly: classic logging first, then the event data for DiagnosticSource and `LogTo`:

Diagnostics/TenancyLoggerExtensions.cs (excerpt, simplified)

```
private static void LogEntityType(
    IDiagnosticsLogger<DbLoggerCategory.Model> diagnostics,
    EventDefinition<string> definition,
    IReadOnlyEntityType entityType)
{
    // 1. ILogger; throws with ConfigureWarnings(w => w.Throw(...))
    if (diagnostics.ShouldLog(definition))
        definition.Log(diagnostics, entityType.DisplayName());

    // 2. DiagnosticSource and LogTo (simple logging)
    if (diagnostics.NeedsEventData(definition, out var diagnosticSource, out var simpleLog))
    {
        var eventData = new EntityTypeEventData(definition, GenerateMessage, entityType);
        diagnostics.DispatchEventData(definition, eventData, diagnosticSource, simpleLog);
    }
}
```

Users can then turn the warning into an error:

Usage

```
options.UseNpgsql(cs)
       .UseAcmeTenancy()
       .ConfigureWarnings(w => w.Throw(TenancyEventId.TenantIsolationOptedOut));

// -> InvalidOperationException while building the model:
//    "An error was generated for warning '...TenantIsolationOptedOut': Entity
//     'AuditEntry' ... This exception can be suppressed or logged by passing event ID
//     'TenancyEventId.TenantIsolationOptedOut' to the 'ConfigureWarnings' method ..."
```

> **Tip**
>
> Security-relevant deviations belong in the log as a **warning**, not as a debug message. Teams with strict requirements escalate them to errors via `ConfigureWarnings`; others at least see them at startup. The

> plugin does not have to make this decision for everyone.

## OpenTelemetry: the tenant on the trace

Distributed tracing with OpenTelemetry builds on `System.Diagnostics.Activity`. EF Core starts no activities itself. The spans of database commands come either from the OpenTelemetry instrumentation for EF Core, which listens to EF's `DiagnosticSource` events, or from the database driver: Npgsql and SqlClient have activity sources of their own. Both usually lack the tenant, the first question when a query is slow or fails in production. With `EnrichOpenTelemetry()` the plugin adds it as the tag `tenant.id` (default: off):

Usage

```
options.UseNpgsql(cs).UseAcmeTenancy(sp, t => t.EnrichOpenTelemetry());
// SQL Server: UseSqlServer(cs, sql => sql.UseAcmeTenancy(sp).EnrichOpenTelemetry())
```

A command interceptor tags `Activity.Current` before every command. It is registered in the internal service provider as `IInterceptor`, so it is scoped with one instance per context, needs no application services and reads the tenant from the context of the command:

Interceptors/TenantActivityEnrichmentInterceptor.cs (excerpt)

```
protected virtual void Enrich(CommandEventData eventData)
{
    // The cheap checks first: without a recorded activity there is nothing to do
    if (Activity.Current is not { IsAllDataRequested: true } activity
        || eventData.Context is not { } context)
        return;

    // The tenant provider of this context travels with its SaveChanges interceptor
    var tenant = context.GetService<IDbContextOptions>()
        .FindExtension<CoreOptionsExtension>()?.Interceptors?
        .OfType<TenantSaveChangesInterceptor>().FirstOrDefault()?.Tenants.CurrentTenantId;

    // Without a tenant no tag: an empty or placeholder value would be misleading
    if (tenant is { } id)
        activity.SetTag(TagName, id.Value.ToString("D"));
}
```

Which activity receives the tag depends on the instrumentation:

| Instrumentation | Tagged activity |
|---|---|
| OpenTelemetry instrumentation for EF Core | The command's own span: EF 10 raises its `Command Executing` event, which starts the span, before it calls the interceptors |
| Only the driver (Npgsql, SqlClient) | The enclosing span, typically the HTTP request: the driver starts its span inside `ExecuteReader`, after the interceptor |
| No listener | None; the interceptor returns after one check, without allocations |

`ActivityEnrichmentTests` checks the tag against PostgreSQL with an `ActivityListener`: set when enabled, missing when disabled and missing without a tenant.

> **Tenant IDs in telemetry**
>
> A tenant ID is an identifier, not secret data, but traces often leave the application: to a monitoring service, sometimes to another company. Whether tenant IDs may go there is a data protection decision of the operator, which is why the option is off by default.

Part VI

# Migrations and design time

Generating schema changes, extending the dotnet ef tooling and supporting compiled models.


› Chapter 20: How migrations work

› Chapter 21: Declarative SQL from annotations

› Chapter 22: Custom migration operations

› Chapter 23: Design-time services

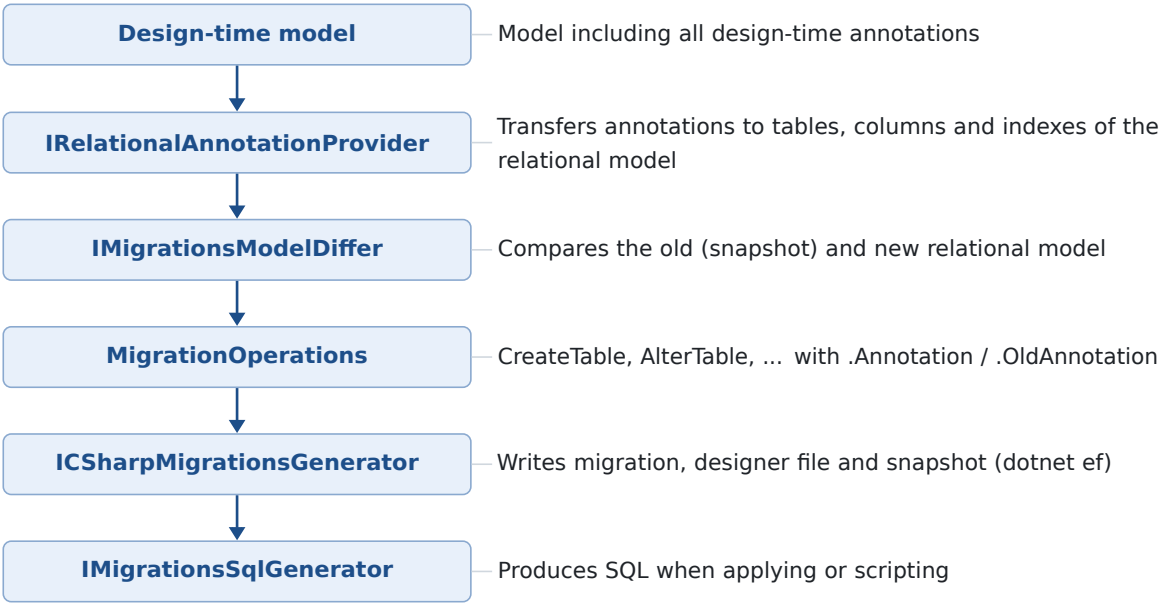
› Chapter 24: Compiled models, trimming and NativeAOT

› Chapter 25: Scaffolding (reverse engineering)

# 20 How migrations work

Version dependency: **stable**

Migrations compare two models and produce operations and SQL from them. A plugin can hook in at several points; to choose the right one you need to know the flow:




There are two basic strategies for plugins. **Declarative**: the plugin describes the desired state as annotations; EF detects changes by itself, and the plugin only has to produce SQL. **Imperative**: the plugin offers its own operations that users write into migrations by hand. Good plugins use both: declarative for everything that belongs to the model, imperative for everything else.

> **Note**
>
> `Database.EnsureCreated()` uses the same pipeline: it computes the diff from the empty state to the design-time model and executes the SQL of the migrations generator. A plugin's custom migration SQL therefore takes effect there too, which simplifies integration tests considerably (chapter 26).

# 21 Declarative SQL from annotations

Version dependency: **volatile**

## The annotation provider

The differ compares the *relational* model (tables, columns), not the entity types. Which annotations a table carries is decided by the `IRelationalAnnotationProvider`. There is only one (a singleton per internal service provider), and it belongs to the provider; the plugin therefore derives from the Npgsql implementation:

Metadata/Internal/TenancyAnnotationProvider.cs

```
public class TenancyAnnotationProvider : NpgsqlAnnotationProvider
{
    public TenancyAnnotationProvider(RelationalAnnotationProviderDependencies dependencies)
        : base(dependencies)
    {
    }

    public override IEnumerable<IAnnotation> For(ITable table, bool designTime)
    {
        foreach (var annotation in base.For(table, designTime))
            yield return annotation;

        if (!designTime)          // migrations only: the runtime model stays lean
            yield break;

        // TPH/table splitting: the type with RLS configuration, not simply the first
        var entityType = table.EntityTypeMappings
            .Select(m => (IEntityType)m.TypeBase)
            .FirstOrDefault(e => e.HasRlsConfiguration())
            ?? (IEntityType)table.EntityTypeMappings.First().TypeBase;
        foreach (var annotation in entityType.GetAnnotations())
        {
            if (!annotation.Name.StartsWith(RlsAnnotationNames.Prefix,
                                            StringComparison.Ordinal))
                continue;
            // Opt-out marker and explicit force: false stay in the model (they beat the
            // conventions), but "false" and "missing" must be the same table
            if (annotation.Name is RlsAnnotationNames.Enabled or RlsAnnotationNames.Forced
                && annotation.Value is false)
                continue;
            yield return annotation;
        }
    }
}
```

From here EF takes over: when a policy changes, the differ produces an `AlterTableOperation` with new and old annotations. In the generated migration it looks like this, without any custom code generator:

Generated migration (excerpt)

```
migrationBuilder.AlterTable(name: "Orders")
    .Annotation("Acme:Rls:Policy:tenant_isolation", "{...new...}")
    .OldAnnotation("Acme:Rls:Policy:tenant_isolation", "{...old...}");
```

Because the annotation provider only passes the annotations on for `designTime`, they are missing from the runtime model that `context.Model` returns.

Startup check of a plugin (WRONG)

```
// Does every isolated table have RLS? Read from the runtime model
var table = context.Model.GetRelationalModel().FindTable("Orders", null);
var enabled = table?[RlsAnnotationNames.Enabled];          // always null
```

**What happens if plugin code reads the runtime model instead of the design-time model?**

**Observable.** Code that reads the plugin's schema annotations from the relational runtime model finds none: a check reports nothing, a self-generated script contains no policies. EF protects the differ itself: `GetDifferences` with the runtime model fails with a hint to `IDesignTimeModel`.

**Cause.** The runtime model is lean. The annotation provider passes annotations for tables on only at design time, so that they do not burden the run time.

**Solution.** For everything related to the schema, use `context.GetService<IDesignTimeModel>().Model`.

**Test:** `FailureCaseTests.Table_annotations_of_the_runtime_model_…` and `Differ_rejects_the_runtime_model_….`

## The SQL generator

There is also only one `IMigrationsSqlGenerator`. The plugin derives from the Npgsql generator and extends `CreateTable` and `AlterTable`:

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (excerpt)

```
protected override void Generate(
    CreateTableOperation operation, IModel? model,
    MigrationCommandListBuilder builder, bool terminate = true)
{
    // Unterminated SQL cannot take further statements (as with Npgsql comments)
    if (!terminate && operation.HasRlsConfiguration())
        throw new ArgumentException("RLS cannot be emitted in an unterminated CREATE TABLE.");

    base.Generate(operation, model, builder, terminate);
    if (terminate)
        GenerateRowLevelSecurity(operation.Name, operation.Schema, previous: null,
            operation, builder);
}

protected override void Generate(
    AlterTableOperation operation, IModel? model, MigrationCommandListBuilder builder)
{
    base.Generate(operation, model, builder);
    GenerateRowLevelSecurity(operation.Name, operation.Schema, operation.OldTable,
        operation, builder);
```

```
}
```

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (shortened, simplified)

```
protected virtual void GenerateRowLevelSecurity(string tableName, string? schema,
    IReadOnlyAnnotatable? previous, IReadOnlyAnnotatable current,
    MigrationCommandListBuilder builder)
{
    var table = Delimit(tableName, schema);
    var oldPolicies = previous?.GetRlsPolicies() ?? new Dictionary<string, string>();
    var newPolicies = current.GetRlsPolicies();

    // 1. Drop removed or changed policies
    foreach (var (name, json) in oldPolicies)
        if (!newPolicies.TryGetValue(name, out var n) || n != json)
            Statement(builder, $"DROP POLICY {Delimit(name)} ON {table}");

    // 2. RLS state (ENABLE/DISABLE, FORCE/NO FORCE)
    if ((previous?.IsRlsEnabled() ?? false) != current.IsRlsEnabled())
        Statement(builder, $"ALTER TABLE {table} "
            + (current.IsRlsEnabled() ? "ENABLE" : "DISABLE") + " ROW LEVEL SECURITY");
    // ... FORCE analogous

    // 3. Create new or changed policies
    foreach (var (name, json) in newPolicies)
        if (!oldPolicies.TryGetValue(name, out var o) || o != json)
            Statement(builder, CreatePolicySql(table, RlsPolicy.Deserialize(json)));
}

protected virtual void Statement(MigrationCommandListBuilder builder, string sql)
{
    builder.Append(sql).AppendLine(Dependencies.SqlGenerationHelper.StatementTerminator);
    EndStatement(builder);
}
```

| Detail | Reason |
|---|---|
| DROP + CREATE instead of `ALTER POLICY` | `ALTER POLICY` can change neither `FOR` nor PERMISSIVE/RESTRICTIVE. DDL is transactional in PostgreSQL; no visible gap occurs. |
| `EndStatement(builder)` | Ends the command like the base class does, honoring transaction suppression. |
| Own `Delimit` helpers | Npgsql's `DelimitIdentifier` helpers are private. Always escape identifiers via `Dependencies.SqlGenerationHelper`, never by string concatenation. |
| DropTable, RenameTable | Nothing to do: PostgreSQL removes or moves policies together with the table. |

# 22 Custom migration operations

Version dependency: **medium**

Some things do not belong in the model but in migrations: roles, grants, extensions, data backfills. For these the plugin offers its own operations. They consist of three parts: the operation class, an extension method on `MigrationBuilder` and the SQL generation in the generator.

Migrations/Operations/TenancyOperations.cs (excerpt)

```
public class CreateApplicationRoleOperation : MigrationOperation
{
    public required string Name { get; init; }
    public bool CanLogin { get; init; }       // passwords do NOT belong in migrations
}
```

Migrations/Operations/TenancyMigrationBuilderExtensions.cs (excerpt)

```
namespace Microsoft.EntityFrameworkCore.Migrations;      // always imported in migrations

public static class TenancyMigrationBuilderExtensions
{
    public static OperationBuilder<CreateApplicationRoleOperation> CreateApplicationRole(
        this MigrationBuilder migrationBuilder, string name, bool canLogin = false)
    {
        var operation = new CreateApplicationRoleOperation
        {
            Name = TenancyIdentifiers.ValidateIdentifier(name, "role name"),
            CanLogin = canLogin,
        };
        migrationBuilder.Operations.Add(operation);
        return new OperationBuilder<CreateApplicationRoleOperation>(operation);
    }
}
```

The generator dispatches operations through the central `Generate(MigrationOperation, ...)` method. Unknown types go to the base class:

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (excerpt, simplified)

```
protected override void Generate(
    MigrationOperation operation, IModel? model, MigrationCommandListBuilder builder)
{
    switch (operation)
    {
        case CreateApplicationRoleOperation createRole: Generate(createRole, builder); break;
        case DropApplicationRoleOperation dropRole:     Generate(dropRole, builder); break;
        case GrantTablePrivilegesOperation grant:       Generate(grant, builder); break;
        default: base.Generate(operation, model, builder); break;
    }
}

protected virtual void Generate(CreateApplicationRoleOperation operation,
    MigrationCommandListBuilder builder)
{
    var literal = Literal(operation.Name);           // 'shop_app'  (for the comparison)
```

```
    var identifier = Delimit(operation.Name);    // shop_app (CREATE ROLE, "..." if needed)
    var login = operation.CanLogin ? "LOGIN" : "NOLOGIN";

    builder
        .AppendLine("DO $$")
        .AppendLine("BEGIN")
        .AppendLine("    IF NOT EXISTS (SELECT FROM pg_catalog.pg_roles")
        .AppendLine($"                   WHERE rolname = {literal}) THEN")
        .AppendLine($"        CREATE ROLE {identifier} {login} NOSUPERUSER NOBYPASSRLS;")
        .AppendLine("    END IF;")
        .AppendLine("END")
        .Append("$$");
    builder.AppendLine(Dependencies.SqlGenerationHelper.StatementTerminator);
    EndStatement(builder);
}
```

**Literal or identifier**

The role name appears twice, but in two forms: as a **string literal** in the comparison (`'shop_app'`, produced by the string type mapping) and as an **identifier** in `CREATE ROLE` (`shop_app`, in double quotes if it needs them). Mixing them up or building them by string concatenation produces invalid SQL or an injection hole. The extension method additionally validates the name.

**Tip**

Write operations idempotently (`IF NOT EXISTS`, `DROP ... IF EXISTS`). Roles exist cluster-wide and outlive databases; a migration that fails on its second run against a new database is a common annoyance.

In a migration, the operations are used like EF's own:

`Migrations/20261021_Initial.cs (hand-edited)`

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    // BEFORE the tables (chapter "Advanced migrations")
    migrationBuilder.CreateApplicationRole("shop_support");
    // ... operations generated by EF (tables, RLS via annotations) ...
    migrationBuilder.CreateApplicationRole("shop_app", canLogin: true);
    migrationBuilder.GrantTablePrivileges("shop_app");
}

protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropApplicationRole("shop_app");
    // ...
}
```

**When the differ should produce custom operations**

If the *differ* should produce custom operations automatically, you need a derived `IMigrationsModelDiffer` and an `ICSharpMigrationOperationGenerator` that writes the operation as C#. Both are single services with a lot of internal API. The declarative route via annotations (chapter 21) is almost always the better choice.

# 23 Design-time services

Version dependency: **volatile**

The `dotnet ef` tools (adding migrations, writing snapshots, scaffolding, compiled models) use their own service provider with design-time services. A plugin extends it through `IDesignTimeServices`.

## Loaded through the startup project

An assembly attribute points to the class. The tooling, however, reads it only from the startup project and from the migrations assembly, not from referenced packages; an attribute in the plugin assembly itself would have no effect. The plugin therefore documents one line that the application adds once to its startup project. The second argument restricts loading to contexts with the matching provider. The consumer test (page 105) does exactly that:

eng/consumer-test/template/DesignTimeServices.cs

```
[assembly: DesignTimeServicesReference(
    "Acme.EntityFrameworkCore.Tenancy.PostgreSQL.Design.TenancyDesignTimeServices, "
        + "Acme.EntityFrameworkCore.Tenancy.PostgreSQL",
    "Npgsql.EntityFrameworkCore.PostgreSQL")]
```

Design/TenancyDesignTimeServices.cs (excerpt)

```
public class TenancyDesignTimeServices : IDesignTimeServices
{
    public void ConfigureDesignTimeServices(IServiceCollection services)
    {
        // Remove all registrations (EF default, Npgsql), then add ours:
        // independent of the order in which the tooling loads services
        services.RemoveAll<IAnnotationCodeGenerator>();
        services.AddSingleton<IAnnotationCodeGenerator, TenancyAnnotationCodeGenerator>();

        // ... likewise ICSharpRuntimeAnnotationCodeGenerator (compiled models) and
        // IDatabaseModelFactory (scaffolding)
    }
}
```

## Readable snapshots: the annotation code generator

By default EF writes unknown annotations as `HasAnnotation(...)` into the snapshot, for policies with JSON values. That works, but is hardly readable. A custom `IAnnotationCodeGenerator` translates them into the Fluent API:

ShopContextModelSnapshot.cs

```
// Before
b.HasAnnotation("Acme:Rls:Enabled", true);
b.HasAnnotation("Acme:Rls:Forced", true);
b.HasAnnotation("Acme:Rls:Policy:tenant_isolation", "{\"Name\":\"tenant_isolation\",...}");

// After
RlsEntityTypeBuilderExtensions.HasRowLevelSecurity(b, true);
```

```
RlsEntityTypeBuilderExtensions.HasPolicy(b, "tenant_isolation", p => p
    .Using("\"TenantId\" = nullif(current_setting('app.tenant_id', true), '')::uuid")
    .WithCheck("\"TenantId\" = nullif(current_setting('app.tenant_id', true), '')::uuid"));
```

EF Core 10 writes extension methods in the snapshot as static calls (`Extensions.Method(b, ...)`) instead of `b.Method(...)`. This avoids ambiguities when several providers or plugins define methods with the same name. Tests that check the generated code should expect this form.

Design/Internal/TenancyAnnotationCodeGenerator.cs (excerpt, simplified)

```
public override IReadOnlyList<MethodCallCodeFragment> GenerateFluentApiCalls(
    IEntityType entityType, IDictionary<string, IAnnotation> annotations)
{
    var calls = new List<MethodCallCodeFragment>();

    // CONTRACT: REMOVE translated annotations from the dictionary
    if (annotations.TryGetValue(RlsAnnotationNames.Enabled, out var enabled)
        && enabled.Value is true)
    {
        annotations.Remove(RlsAnnotationNames.Enabled);
        var forced = annotations.Remove(RlsAnnotationNames.Forced, out var f)
                     && f.Value is true;
        calls.Add(new MethodCallCodeFragment(HasRowLevelSecurityMethod, forced));
    }

    foreach (var key in annotations.Keys
                 .Where(k => k.StartsWith(RlsAnnotationNames.PolicyPrefix,
                                          StringComparison.Ordinal))
                 .ToList())                          // copy: the dictionary changes
    {
        var policy = RlsPolicy.Deserialize((string)annotations[key].Value!);
        annotations.Remove(key);
        calls.Add(new MethodCallCodeFragment(HasPolicyMethod, policy.Name,
            new NestedClosureCodeFragment("p", BuildPolicyChain(policy))));
    }

    calls.AddRange(base.GenerateFluentApiCalls(entityType, annotations));
    return calls;
}
```

| Pitfall | Solution |
|---|---|
| Translated annotation stays in the dictionary | Appears twice in the snapshot. Always remove it; whatever remains, EF writes as `HasAnnotation`. |
| Generic method in the `MethodInfo` | The snapshot uses the non-generic `EntityTypeBuilder`. Pick the non-generic overload via `GetRuntimeMethod` with exact parameter types. |
| `params` argument as an array | `MethodCallCodeFragment` takes `params object?[]`. A `string[]` would become the argument list through array covariance: wrap it in `new object?[] { roles }`. |
| Enums in the snapshot | Rendered as `PolicyCommand.Select`. The enum therefore lives in the always-imported namespace `Microsoft.EntityFrameworkCore`. |
| Lambda arguments | `NestedClosureCodeFragment("p", chain)` produces `p => p.Using(...)`; chains are built with `fragment.Chain(...)`. |

The generator only affects the snapshot and designer files. The actual migrations keep `.Annotation(...)` on `AlterTable`; that is the intended mechanism for changes.

# 24 Compiled models, trimming and NativeAOT

Version dependency: **medium**

`dotnet ef dbcontext optimize` produces a **compiled model**: C# code that builds the runtime model directly instead of computing it at startup through conventions. This shortens startup for large models considerably and is a prerequisite for NativeAOT. A plugin must be prepared for it in four places:

| Building block | Requirement | In the sample |
|---|---|---|
| Type mappings | Public type with a public static property `Default` (used for NativeAOT); converters/comparers as shared instances | `TenantIdTypeMapping.Default`, `TenantIdConverter.Instance`, `TenantIdValueComparer` |
| JSON reader/writer | Static `Instance` and `ConstructorExpression` | `TenantIdJsonReaderWriter` |
| Value generators | Configured with `HasValueGeneratorFactory(Type)`; factory with a public parameterless constructor | `TenantIdValueGeneratorFactory` (chapter 31) |
| Annotations | Simple values are taken over by EF automatically; an `ICSharpRuntimeAnnotationCodeGenerator` filters design-time annotations and translates complex values | Filters RLS policies, keeps the runtime annotation (chapter 44) |

The last point is often misunderstood. EF takes annotations with simple values (strings, numbers, bool, enums) into the compiled model **automatically**; the source of `CSharpRuntimeAnnotationCodeGenerator` shows this, writing them via `GenerateSimpleAnnotations`. A custom generator is therefore not needed to *keep* annotations, but to *filter out* pure design-time annotations that nobody reads at run time, and to translate annotations with complex values into code. Chapter 44 shows the filtering in the sample; the policies are stored there as JSON strings, that is, as simple values.

## Trimming and AOT

All packages of the plugin set `<IsAotCompatible>true</IsAotCompatible>`. The compiler then reports code that uses reflection without annotations. The JSON serialization of the policies uses a source generator instead of reflection:

Metadata/RlsPolicy.cs (excerpt)

```
[JsonSourceGenerationOptions(UseStringEnumConverter = true)]
[JsonSerializable(typeof(RlsPolicy))]
internal partial class RlsJsonContext : JsonSerializerContext;
```

Where dynamic code is unavoidable, the warning is suppressed with a justification instead of being ignored. The isolation convention builds the query filter as an expression tree, which needs dynamic code. Conventions only run while the model is built, however, and NativeAOT requires a compiled model, in which they no longer run; the suppression states exactly this reason. The warnings only show reliably in a full build; incremental builds keep quiet about them (upgrade playbook in chapter 27).

> **Note**
>
> Conventions, snapshots and migrations only run at design time or during model building; with a compiled model and precompiled queries they disappear at run time. For AOT, the critical parts are therefore type mappings, converters and interceptors; with precompiled queries, translators already run at build time.

## What is actually checked as a consumer

`IsAotCompatible` runs analyzers but proves nothing at run time. A unit test that generates compiled model code and checks text in it (`CompiledModelTests`) does not compile or load the model either. `eng/consumer-test/run.sh` therefore checks the plugin the way an application gets it, in a temporary directory that is deleted at the end:

| Step | Check |
|---|---|
| Packages | `dotnet pack` of the four libraries (core, relational layer, PostgreSQL, SQL Server) into a local source; a fresh project references the packages, not the projects. Its own NuGet cache, package source mapping with `Acme.*` only from the local source and a SHA-512 comparison prove that exactly the packages just built are resolved. |
| Tooling | `dotnet-ef` as a local tool. The design-time services are loaded by the attribute in the startup project. Migration and SQL script contain RLS, `FORCE`, policy and index (PostgreSQL) and security policy and session key (SQL Server). `has-pending-model-changes` then reports no change. |
| Runtime model | `TenantId` becomes `uuid`, the runtime annotation names the property, `CurrentTenant()` and the query filter are translated. |
| Compiled model | `dotnet ef dbcontext optimize`, compile, load with `UseModel`, the same checks. |
| NativeAOT (`--nativeaot`) | Publish with `PublishAot` and precompiled queries, then run against PostgreSQL in Docker: two tenants write through the interceptors, one reads, RLS returns only its rows. No trim or AOT warning may come from an Acme assembly. |

CI runs the flow in a job of its own with the newest stable 10.0.x patch for the tooling and prints the versions. The test found things the unit tests could not show:

- **Query filters and compiled models exclude each other.** EF Core 10 rejects `optimize`: “Compiled model can’t be generated, because query filters are not supported.” Whoever enables the plugin’s optional filter gives up compiled model and NativeAOT; protection then comes from RLS alone. The test expects this error explicitly, so that a later EF version that supports filters stands out.
- **Value comparers through reflection.** The type mapping for `TenantId` brought no comparers of its own. The compiled model then only refers to `TenantIdTypeMapping.Default`, and EF creates the comparer on first tracking with `MakeGenericMethod`, which made the NativeAOT application fail. Since then both mappings carry shared instances from `TenantId ValueComparer`, like converter and JSON reader.
- **Limits of EF itself.** Precompiled queries and NativeAOT are experimental in EF Core 10 (warning EF9100). `PublishAot` must not be in the project, because EF otherwise refuses to build the model at design time already. The package `Microsoft.EntityFrameworkCore.Tasks` generates model and queries on publish; an additional, manually generated model gets in its

way. Precompilation rejects `o.Total > 0` on a `decimal` property, `0m` works. EF, EF Relational and Npgsql report trim and AOT warnings themselves (IL2104, IL3053).

Not checked are SQL Server under NativeAOT (the package is marked for it, a run with a real database is missing) and the Cosmos package, which the consumer test does not pack.

# 25 Scaffolding (reverse engineering)

Version dependency: **medium**

`dotnet ef dbcontext scaffold` reads an existing database schema and generates entities and a context. A plugin that manages its own schema objects can recognize them during this too. The route goes through two design-time services:

| Service | Task |
|---|---|
| `IDatabaseModelFactory` | Reads the schema into a `DatabaseModel`. A derived Npgsql factory queries `pg_policies` and `pg_class.relrowsecurity` and attaches them as annotations to `DatabaseTable`. |
| `IAnnotationCodeGenerator` | Writes these annotations as Fluent API into the generated `OnModelCreating`, with the same logic as for the snapshot. |

The sample implements both: a custom database model factory reads RLS state and policies from the database, and the annotation code generator from chapter 23 writes them into the scaffolded context. Chapter 45 shows the implementation in detail.

Part VII

# Quality and delivery

Testing, versioning, packaging, and the checklist from A to Z.


› Chapter 26: Test strategy

› Chapter 27: Versioning and compatibility

› Chapter 28: Packaging and documentation

› Chapter 29: The checklist from A to Z

# 26 Test strategy

Version dependency: **medium**

Plugins can be tested to a surprising extent without a database. Model building, type mappings, SQL translation, migration SQL and snapshots only need a connection string, no connection. The sample's test strategy has four levels:

| Level | Checks | Database |
|---|---|---|
| Model tests | Annotations, conventions, validation, options, caching | no |
| SQL tests | Translations via `ToQueryString`, migration SQL via differ and generator | no |
| Snapshot tests | Code generator as a string and as a round trip (Roslyn) | no |
| Integration tests | Real behavior: RLS, roles, interceptors (Testcontainers) | yes (Docker) |

## Test helpers

Tests build many contexts with different `OnModelCreating`. Because EF caches the model per internal provider, every test must get its own provider; that is what `EnableServiceProviderCaching(false)` is for:

tests/.../TestModel.cs (shortened, simplified)

```
public static TestContext Create(
    Action<TenancyOptionsBuilder>? tenancy = null,
    Action<ModelBuilder>? model = null,
    bool cacheServiceProvider = false,
    Action<DbContextOptionsBuilder>? configure = null)
{
    var builder = new DbContextOptionsBuilder<TestContext>()
        .UseNpgsql("Host=localhost;Database=unused")       // no connection needed
        .UseAcmeTenancy(tenancy)
        .EnableServiceProviderCaching(cacheServiceProvider);
    configure?.Invoke(builder);
    return new TestContext(builder.Options, model);
}

public static IReadOnlyList<string> MigrationSql(DbContext? source, DbContext target)
{
    var operations = target.GetService<IMigrationsModelDiffer>().GetDifferences(
        source?.GetService<IDesignTimeModel>().Model.GetRelationalModel(),
        target.GetService<IDesignTimeModel>().Model.GetRelationalModel());

    return target.GetService<IMigrationsSqlGenerator>()
        .Generate(operations, target.GetService<IDesignTimeModel>().Model)
        .Select(c => c.CommandText.Trim()).ToList();
}
```

> **Use the design-time model**
>
> Always use the **design-time model** for migration tests (`IDesignTimeModel`). `context.Model` is the lean runtime model; annotations that the annotation provider only passes on at design time are missing from

it (failure case in chapter 21).

**Why not SQLite or InMemory?**

Tests against SQLite or the InMemory provider are tempting because they need no Docker. For a plugin they prove little: InMemory is not relational, so type mappings, translators and migrations never run; SQLite has a different SQL dialect, other type mappings and no Row Level Security. A provider-specific plugin is tested with its own provider: without a database (connection string only) for everything up to the SQL text, and against the real database for behavior. Applications that use the plugin are a different matter (next section).

## Test mode for applications

Applications often have hundreds of business logic tests on SQLite or InMemory. They are not about isolation but about their own logic, and they must not break just because the entities implement `ITenantOwned`. The PostgreSQL package refuses SQLite on purpose, and switching that check off would not help: the package replaces provider services with Npgsql subclasses, maps `TenantId` to `uuid` and filters with `current_setting()`. The core therefore has a test mode of its own that contains only the application-side half of the plugin:

tests/.../TestingModeTests.cs (excerpt)

```
private sealed class AppContext : DbContext
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
        => modelBuilder.HasTenantQueryFilters(this);
}

new DbContextOptionsBuilder<AppContext>()
    .UseSqlite(connection)
    .UseAcmeTenancyForTesting(Services(tenants))
    .Options

ctx.Orders.Add(new Order { Total = 1 });      // TenantId left empty on purpose
await ctx.SaveChangesAsync();
```

`UseAcmeTenancyForTesting(sp)` maps `TenantId` as a `Guid` for every provider (through a value converter selector, as EF does for enums), adds the SaveChanges interceptor of the core and records the tenant property for it. `HasTenantQueryFilters(this)` filters with the tenant as a query parameter, the same filter as for Cosmos DB (chapter 48). Row level security, migration SQL, the session and the translators are missing. `TestingModeTests` runs the mode on SQLite and InMemory.

**A test mode, not a production mode**

`UseAcmeTenancyForTesting` refuses PostgreSQL, SQL Server and Cosmos DB, so that it cannot replace the real isolation by accident. Whether the isolation holds is shown by the plugin's own integration tests and by the application's integration tests against the real database (Testcontainers); the test mode only keeps the business logic tests running.

## What is tested

| Test | Ensures |
|---|---|
| Caching with different options | Two contexts with different settings get different models (hash mistake from chapter 5). |
| Invalid setting names | Validation including an injection attempt. |
| Type mapping without configuration | `TenantId` becomes `uuid`; literal format; a different column type is not claimed. |
| `CurrentTenant()` via `ToQueryString` | Translation, and that nothing is evaluated client-side (chapter 15). |
| Explicit beats convention | An own policy with the same name is kept; the Fluent API overrides the attribute. |
| Warning as error | The opt-out warning becomes an error via `ConfigureWarnings`. |
| Validation | RLS on a derived type and invalid policies throw while building the model. |
| No phantom migration | An unchanged model produces not a single operation. |
| Migration SQL | Initial, change (DROP + CREATE), opt-out, custom operations. |
| SaveChanges logic | Setting, foreign tenant, reassignment, no tenant. |
| Snapshot | Fluent API instead of annotations; round trip without differences. |
| RLS in PostgreSQL | Isolation, no rows without a tenant, WITH CHECK applies. |
| Edge cases (chapters 30–35) | ExecuteUpdate and raw SQL against RLS, value generation, JSON columns, compiled model, idempotent script. |
| Composability | Decorator around the migrations generator and the composition matrix of two plugins (chapter 6), EFCore.NamingConventions in both orders (chapter 10). |
| Annotation format | Serialized policies stay identical; tolerant reading (chapter 27). |
| Context lifetimes | DbContext factory, background jobs, open connections (chapters 18, 40). |
| Cross-tenant access | Reporting role reads all tenants but cannot write; the app role cannot switch RLS off (chapter 36). |

## The snapshot round trip

The most valuable test for design-time code: generate the snapshot, compile it with Roslyn, build the model from it and diff it against the original. Any discrepancy between code generator and Fluent API would otherwise produce a phantom migration for every user:

tests/.../SnapshotTests.cs (excerpt)

```
[Fact]
public void Snapshot_roundtrip_produces_no_differences()
{
    using var ctx = CreateContext();
    var snapshot = Compile(GenerateSnapshot(ctx));

    // As in EF's migrator: finalize and initialize
    var model = snapshot.Model;
    if (model is IMutableModel mutable) model = mutable.FinalizeModel();
    model = ctx.GetService<IModelRuntimeInitializer>()
        .Initialize(model, designTime: true, validationLogger: null);

    var differences = ctx.GetService<IMigrationsModelDiffer>().GetDifferences(
```

```
        model.GetRelationalModel(), ctx.DesignModel().GetRelationalModel());

    Assert.Empty(differences);
}
```

## Integration tests with Testcontainers

The real security behavior can only be verified against the real database, here PostgreSQL. The fixture starts a container, creates the schema with `EnsureCreated` (including RLS, chapter 20), executes the custom migration operations for real and connects the tests with a restricted role:

tests/...IntegrationTests/PostgresFixture.cs (excerpt)

```
await owner.Database.EnsureCreatedAsync();          // RLS, policies, index

var commands = owner.GetService<IMigrationsSqlGenerator>().Generate(
[
    new CreateApplicationRoleOperation { Name = AppRole, CanLogin = true },
    new GrantTablePrivilegesOperation { Role = AppRole },
]);
await owner.GetService<IMigrationCommandExecutor>().ExecuteNonQueryAsync(
    commands, owner.GetService<IRelationalConnection>(),
    new MigrationExecutionState(), commitTransaction: true);
```

tests/...IntegrationTests/RowLevelSecurityTests.cs (excerpt, simplified)

```
[Fact]
public async Task Database_rejects_rows_of_other_tenant()
{
    // Bypass the interceptor: direct SQL. The policy's WITH CHECK must apply.
    var tenants = new MutableTenantProvider { CurrentTenantId = TenantId.New() };
    await using var db = _fixture.CreateAppContext(tenants);

    var ex = await Assert.ThrowsAsync<PostgresException>(() => db.Database.ExecuteSqlAsync(
        $"INSERT INTO \"Orders\" (\"TenantId\", \"Total\") VALUES ({Guid.NewGuid()}, 1)"));

    Assert.Equal(PostgresErrorCodes.InsufficientPrivilege, ex.SqlState);
}
```

> **Tip**
>
> For providers and very deep plugins, EF Core provides the **specification tests** (`Microsoft.EntityFrameworkCore.Relational.Specification.Tests`). This is the same test suite EF and the providers themselves are tested against. The NetTopologySuite plugin in the EF repository shows how a plugin uses it.

## Combinations of migrations and isolation

Single features are well covered; the errors a review found were in combinations: a changed session key together with a dropped table, a temporal table, an ignored tenant property. A complete cartesian product of all migration operations and all settings would be large and mostly redundant. The tests therefore cover a small matrix, chosen by one question: where do two features touch the same database object? On SQL Server that is the shared predicate function and the schema-bound

policy. Every migration case is generated by EF's real differ, runs Up and Down in one transaction against SQL Server and then reads and writes as the app login for two tenants:

| Combination | Test | Checks |
|---|---|---|
| Session key or policy schema changed, a protected table dropped | `SqlServerSessionSettingsTests` | Order of the SQL, Up and Down, isolation of the remaining table |
| Session key changed, table renamed | `SqlServerCombinedMigrationTests` | Order of the SQL, Up and Down, policy on the new name |
| Policy schema changed, column renamed, table renamed and moved | as above | Up and Down, function and policy only in the new schema |
| Migration fails after policy and function were dropped | as above | Rollback: table, policy and function with the old key are unchanged |
| Temporal table (SQL Server only), ignored or unmapped tenant property | model tests of both relational packages | Rejected while the model is built |

Still not covered by a test: combinations on PostgreSQL, where policies follow a rename and there is no shared function, so the problem class does not arise; a changed setting name of the PostgreSQL package together with other table changes; several contexts migrating the same database concurrently; and the Cosmos package, which has no migrations. These combinations are not claimed to work.

# 27 Versioning and compatibility

Version dependency: **medium**

## Package version and supported EF version

The sample versions plugin and handbook together: the package version equals the book edition, packages 1.2.0 belong to edition 1.2. That way every code excerpt belongs to exactly one package state. Which EF version a state supports is therefore not part of the version number but of the packages' dependency range and the compatibility table in the README: plugin 1.x supports EF Core 10. Moving to a new EF major version is a breaking change for users and gets a new major version, 2.0 for EF Core 11, together with a new book edition. Many plugins tie their major version to EF Core's instead (10.x for EF Core 10); that shows the matching combination in the number. Both ways work as long as the dependencies are limited to one EF major version. A property keeps the range in one place so that CI can override it for the patch channel (section "Patches and previews in CI"):

Directory.Packages.props (excerpt)

```
<PropertyGroup>
  <EFCoreVersion Condition="'$(EFCoreVersion)' == ''">[10.0.0,11.0.0)</EFCoreVersion>
  <CentralPackageFloatingVersionsEnabled>true</CentralPackageFloatingVersionsEnabled>
</PropertyGroup>
<PackageVersion Include="Microsoft.EntityFrameworkCore" Version="$(EFCoreVersion)" />
<PackageVersion Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="$(EFCoreVersion)" />
```

## Dealing with internal APIs

Plugins rarely get by without internal APIs: the sample derives from `NpgsqlAnnotationProvider` and `NpgsqlAnnotationCodeGenerator` and uses `NpgsqlOptionsExtension`; in total, the PostgreSQL package touches 10 internal types, the SQL Server package 2, the Cosmos package 1, the core and the relational layer none (inventory in the upgrade playbook). Such types can change even in minor versions. EF reports their use with warning **EF1001**.

| Measure | Effect |
|---|---|
| Look for a public alternative | Every internal API avoided is one risk less. |
| Bundle internal access | A few classes encapsulate it; a break affects one place. |
| Set `<NoWarn>EF1001</NoWarn>` deliberately | Project-wide and documented, not scattered per line. |
| Limit the version range | Prevents users from getting untested combinations. |
| CI against the newest patches and previews | Shows breaks before users report them; the channels are described in section "Patches and previews in CI". |
| Round trip and SQL tests | Detect behavior changes the compiler does not see. |

> **Multi-targeting**
>
> Supporting several EF major versions at once is possible through multi-targeting, but doubles test effort and conditional compilation. For most plugins, one release branch per EF major version is simpler.

## Evolving annotation formats

Everything a plugin writes into annotations ends up in its users' model snapshots, often long before the next upgrade. If a new plugin version serializes the same configuration differently (an additional JSON property, a different order, another enum spelling), the differ compares two different strings. Every user then gets a migration that drops and recreates every policy without any actual change. The serialized format is therefore part of the public API, just like the methods.

| Rule | How |
|---|---|
| Pin the format with a test | A golden-value test compares the serialized form character by character. |
| Read tolerantly | Missing properties become defaults, unknown ones are ignored (the default behavior of System.Text.Json). |
| Only add, and do not write defaults | New optional properties with `[JsonIgnore(Condition = WhenWritingDefault)]` keep existing annotations identical. |
| Real format changes only with a major version | Document them, ideally together with the EF major upgrade, when users expect migrations anyway. |

tests/.../AnnotationFormatTests.cs (excerpt)

```
[Fact]
public void Serialized_policy_format_is_stable()
{
    var policy = new RlsPolicy("support_read", PolicyCommand.Select, Permissive: false,
        Roles: ["shop_support"], Using: "true", WithCheck: null);

    // Golden value: changing it is a breaking change for existing snapshots
    Assert.Equal(
        """{"Name":"support_read","Command":"Select","Permissive":false"""
        + ""","Roles":["shop_support"],"Using":"true","WithCheck":null}""",
        policy.Serialize());
}
```

Two further tests check tolerant reading in both directions. The snapshot round trip from chapter 26 complements them: the round trip shows that the current code generator and Fluent API agree, the golden value shows that the next version still agrees with the snapshots users already have.

## Failure classes in an upgrade

An upgrade to a new EF major version rarely breaks a plugin in just one place, and not every break announces itself. Knowing the failure classes tells you which test finds them, and above all which one does not:

| Failure class | Typical example | Detected in the sample by |
|---|---|---|
| Compile-time break | A provider class's constructor has a new parameter, an internal type has moved | Build; `PublicAPI` files for the plugin's own API (chapter 41) |
| Runtime break | A service changes its lifetime, a registration fails | `ServiceRegistrationTests`, `DesignTimeServicesTests`, `OptionsTests`, `CompositionTests` |

| Failure class | Typical example | Detected in the sample by |
|---|---|---|
| Silent behavioral change | An order, a precedence or a nullability rule of EF changes | `FailureCaseTests`, `CompositionMatrixTests`, `StateAndLifetimeTests`, `SaveChangesInterceptorTests`; with a database `MaterializationTests`, `ConnectionLifetimeTests` |
| SQL change | The provider writes an expression differently or optimizes a condition away | `QueryTranslationTests`, `DeepQueryTests`, `DebuggingTests` |
| Model change | A convention runs earlier, a default configuration changes | `ConventionTests`, `TenantPropertyTests`, `ValidationTests`, `TypeMappingTests`, `JsonTests`, `ValueGenerationTests`, `NamingConventionTests` |
| Snapshot/migration change | The code generator or the differ behaves differently; every user would get a phantom migration | `SnapshotTests` (round trip), `MigrationSqlTests`, `MigrationScriptTests`, `AnnotationFormatTests`, `CompiledModelTests`, consumer test |
| Performance regression | A hot path allocates more or becomes slower | The benchmark project (chapter 39) |

The third row is the most dangerous. Many of the sample's tests do not check the plugin but the behavior of EF that the plugin and this book rely on. In an upgrade they are more valuable than any other test, because they report exactly what no compiler sees.

## The upgrade playbook

The workflow is described for the move from EF Core 10 to EF Core 11, but it applies to every major version: the numbers are interchangeable, the order is not. Beforehand, the previous line is frozen in a release branch; it keeps getting patches for its EF version while `main` moves to the new one.

Step 1: branch and versions

```
git switch -c release/1.x main && git push -u origin release/1.x
git switch main

global.json                "version": "11.0.100"
Directory.Build.props      <TargetFramework>net11.0</TargetFramework>
                           <VersionPrefix>2.0.0</VersionPrefix>
Directory.Packages.props   <EFCoreVersion ...>[11.0.0,12.0.0)</EFCoreVersion>
```

Which target framework an EF version requires follows no fixed rule: EF Core 9 targets `net8.0`, EF Core 10 `net10.0`, the release candidates of EF Core 11 `net11.0` (EF Core releases and planning). Before upgrading, look into the packages of EF *and* the provider: which frameworks do they offer, which SDK do they need? In the sample EF Core 11 requires `net11.0`, so target framework, SDK and packages change together; a new EF package could not even be restored on the old target framework.

| No. | Step | Approach | In the sample |
|---|---|---|---|
| 1 | Update packages | Versions from step 1, plus companion packages released with EF (`Microsoft.Extensions.*`); pinned companion packages already block the restore | `EFCoreVersion` |
| 2 | Classify compiler errors | Count by error code: moved types (CS0234, CS0246), changed constructors and dependencies (CS1729, CS7036), removed overrides (CS0115) | Command below |
| 3 | Investigate internal API | Compare the EF1001 inventory (below) against the sources of both versions; every place individually, including those that still compile; start with the volatile chapters (page 118) | Core: 0, PostgreSQL: 10, SQL Server: 2, Cosmos DB: 1 internal types (functions sample: 5) |
| 4 | Compare registrations and lifetimes | Compare EF's service tables of both versions; the plugin's own `Replace` calls fix lifetimes | `ServiceRegistrationTests` |
| 5 | Check derived provider classes | The compiler reports removed methods, but not new virtual overloads the base now calls instead: read the base class in the diff | `MigrationSqlTests`, `DeepQueryTests` |
| 6 | Check model and conventions | Model tests; also compare `ToDebugString` of the design-time model of both versions (appendix D) | `ConventionTests` etc. |
| 7 | Check annotation formats | The plugin's own format must not change; EF's own annotations the plugin reads may | `AnnotationFormatTests` |
| 8 | Test query SQL | All `ToQueryString` tests; explain every deviation instead of just adjusting the expectation | `QueryTranslationTests` etc. |
| 9 | Compare migration SQL | The generated SQL of the tests; for sample applications, compare the script of `dotnet ef migrations script` before and after the upgrade | `MigrationSqlTests`, `MigrationScriptTests` |
| 10 | Snapshot round trip | Generate the snapshot, compile it, compare; a new migration on an existing project must be empty | `SnapshotTests` |
| 11 | Compiled models | Generate, compile, load with `UseModel`, including the runtime annotations | `CompiledModelTests`, `run.sh` (page 105) |
| 12 | Trimming and NativeAOT | Read new IL warnings (IL2xxx, IL3xxx) in the build and start a NativeAOT application; the experimental API for precompiled queries (EF9100) changes particularly often | `IsAotCompatible`, `run.sh --nativeaot` |
| 13 | Integration tests | Against real databases, with the provider versions of the new line | Integration and SQL Server tests |
| 14 | Compare the plugin's public API | The public API analyzer breaks the build on every deviation; enter intended breaks deliberately | `PublicAPI.*.txt` |

| No. | Step | Approach | In the sample |
|---|---|---|---|
| 15 | Compare benchmarks | The same benchmark on both lines; compare ratio and allocations, not absolute times | Benchmark project |
| 16 | Document breaking changes | CHANGELOG, compatibility table (plugin 2.x for EF Core 11), notes for users, support period of the old line | `CHANGELOG.md` |

Two commands are enough for steps 2 and 3. The second lifts the project-wide suppression of EF1001 for this build only and thus shows every place where the plugin touches internal API:

Steps 2 and 3

```
# Compiler errors by code
dotnet build -c Release 2>&1 | grep -oE "error [A-Z]+[0-9]+" | sort | uniq -c | sort -rn

# Inventory of internal API: NoWarn without EF1001 (EF9100 stays suppressed)
dotnet build -c Release --no-incremental -p:NoWarn=EF9100 2>&1 \
  | grep -oE "EF1001: [A-Za-z.]+" | sort | uniq -c | sort -rn
```

How much a chapter depends on the EF version is shown at its start. In a major upgrade, check the **volatile** chapters first: their code derives from or uses classes in `.Internal` namespaces (EF1001), and breaks there often only show at run time. The medium ones follow, the stable ones come last.

| Rating | Meaning | Chapters |
|---|---|---|
| **volatile** | Depends heavily on `.Internal` namespaces (EF1001); check first in every major upgrade | 21, 23, 32, 42, 44, 45 |
| **medium** | Provider base classes or public API that changes often | 2, 5, 6, 12, 13, 14, 16, 22, 24, 25, 26, 27, 30, 33, 43, 47, 48 |
| **stable** | Concepts and public API | all others |

> **Explain deviations, do not test them away**
>
> If an SQL or behavior test fails after the upgrade, it is tempting to just adjust the expectation. Every deviation must be explained first, though: is it an improvement in EF, a break in the plugin, or changed semantics that affect users? Only the answer decides whether the test, the code or the documentation changes.

## Patches and previews in CI

Version ranges solve only part of the problem. NuGet takes the *lowest* version of a range that satisfies all dependencies; the regular CI therefore tests against the beginning of the line, while users get newer patches. The sample's CI therefore works with three channels:

| Channel | Trigger and packages | Purpose | On failure |
|---|---|---|---|
| Regular (`ci.yml`) | Every push and pull request; lowest version of the range | The line the plugin is built against | Required check |

| Channel | Trigger and packages | Purpose | On failure |
|---|---|---|---|
| Patches (`compatibility.yml`) | Weekly and manually; `-p:EFCoreVersion=10.0.*`, stable packages only | Newest patches of the plugin's own line | Investigate, patch release |
| Next major version | Separate job, scheduled; previews and RCs | See breaks months before the release | Not a required check |

The patch channel is active in the sample. Its first run already showed a real finding: the plugin itself ran with EF Core 10.0.14, but test and sample projects could not be restored because they had pinned companion packages to 10.0.0 and a Roslyn version of their own. Since then, these packages follow `EFCoreVersion`.

A channel for the next major version is deliberately not active in the sample: it works with prereleases. Whoever sets one up follows four rules so that regular releases are never coupled to preview packages:

| Rule | Why |
|---|---|
| Switch everything through global properties of this job: SDK, target framework and `EFCoreVersion` | The defaults in the project files stay stable; a preview package can never get into a regular build |
| Change SDK, target framework and packages together | With the old target framework the restore already fails; such a job can never turn green |
| Never pack or publish | Releases come only from regular builds of the release branches |
| Not a required check | Previews break regularly; the job is an early warning that opens an issue, not a gate for pull requests |

Job for the next EF version (sketch, not in the repository)

```
ef-next:
  runs-on: ubuntu-latest
  continue-on-error: true                # early warning, not a required check
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-dotnet@v4
      with: { dotnet-version: 11.0.x, dotnet-quality: preview }
    - run: dotnet new globaljson --sdk-version "$(dotnet --version)" --force
    - name: Build against the next EF Core version (never packed or published)
      run: dotnet build -c Release -p:TargetFramework=net11.0 -p:EFCoreVersion=11.0.0-*
    - run: >
        dotnet test tests/Acme.EntityFrameworkCore.Tenancy.Tests -c Release --no-build
        -p:TargetFramework=net11.0
```

Third-party test dependencies such as EFCore.NamingConventions often ship later for the new version and can block this job's restore. Exclude such tests in the preview channel instead of switching the channel off.

## NuGet advisories and transitive dependencies

A plugin ships not only its own code but also its dependency graph. NuGet audit reports known vulnerabilities as warnings NU1901 to NU1904 (NuGet: Auditing package dependencies). What

matters is *whom* a finding affects. `dotnet nuget why` finds out, showing the path from the direct dependency to the affected package. The sample had four cases:

| Package | Origin | Affects | Decision |
|---|---|---|---|
| `System.Security.Cryptography.Pkcs` 9.0.4 | EF Core SqlServer → `Microsoft.Data.SqlClient` 6.1 | Consumers of the SQL Server package | No patch and no SqlClient version (not even 7.x) brings 9.0.21 yet. Hence a direct dependency on 9.0.22 in the SQL Server package; it ends up in the nuspec and fixes the application's graph. |
| `Cryptography.Xml`/`Pkcs` 9.0.0 | EF Core Design 10.0.0 → MSBuild 17.14, Workspaces.MSBuild 4.14 | Tooling only: tests, sample | Newer Design patches need other MSBuild packages but would pull EF itself along and end the test of the minimum version. Hence `Xml` 9.0.22 in test and sample projects only. |
| `SSH.NET` 2024.2.0 | Testcontainers 4.6 | Tests only | Testcontainers 4.16 within the same major version; brings 2026.0.0. |
| `SQLitePCLRaw.lib.e_sqlite3` 2.1.11 | EF Core Sqlite 10.0.0 → `SQLitePCLRaw.bundle_e_sqlite3` | Tests only | Direct dependency on 2.1.14 in the test project only. |

Every direct dependency on what is really a transitive package is listed in `Directory.Packages.props` with its origin, the advisory and the criterion for when it can go again. Nothing is suppressed: neither `NoWarn` nor a disabled audit makes a dependency safe, they only hide the finding. Both CI channels also run `dotnet list package --vulnerable --include-transitive` and fail on any finding. The regular run still builds against EF Core 10.0.0, the compatibility run against the newest 10.0.x patch; the fixes are minimum versions and apply in both. For tooling there is an interaction: `Microsoft.EntityFrameworkCore.Tasks` treats the audit warnings as errors on publish, so NativeAOT fails with EF Design 10.0.0 (page 105).

## Upgrade checklist

- ☐ Create a release branch for the old line; move `main` to the new version.
- ☐ Change SDK, target framework, `EFCoreVersion`, companion packages and `VersionPrefix` together.
- ☐ Count compiler errors by code and work through them class by class.
- ☐ Create the EF1001 inventory and check every place against the new sources, including those that compile.
- ☐ Check lifetimes against EF's service table; read derived provider classes for new virtual overloads.
- ☐ Model, SQL, migration, snapshot and compiled model tests green; every deviation explained.
- ☐ Annotation formats unchanged (golden-value tests).
- ☐ New IL warnings for trimming and AOT read.
- ☐ Integration tests against real databases with the new providers.
- ☐ Public API diff checked; intended breaks entered.

☐ Benchmarks of both lines compared.

☐ CHANGELOG, compatibility table and support period of the old line documented.

# 28 Packaging and documentation

Version dependency: **stable**

Shared settings live in `Directory.Build.props`, package versions centrally in `Directory.Packages.props` (Central Package Management):

`Directory.Build.props (shortened)`

```
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  <NoWarn>$(NoWarn);EF1001</NoWarn>
  <Deterministic>true</Deterministic>
  <WarningsAsErrors>$(WarningsAsErrors);RS0016;RS0017</WarningsAsErrors>
</PropertyGroup>
<PropertyGroup>
  <PackageLicenseExpression>MIT</PackageLicenseExpression>
  <PackageReadmeFile>README.md</PackageReadmeFile>
  <VersionPrefix>1.2.0</VersionPrefix>       <!-- = book edition -->
  <IncludeSymbols>true</IncludeSymbols>
  <SymbolPackageFormat>snupkg</SymbolPackageFormat>
  <PublishRepositoryUrl>true</PublishRepositoryUrl>      <!-- SourceLink -->
  <EmbedUntrackedSources>true</EmbedUntrackedSources>
</PropertyGroup>
```

| Element | Why |
|---|---|
| `GenerateDocumentationFile` | XML docs in the package: users see explanations in the IDE. |
| `WarningsAsErrors` RS0016, RS0017 | A changed public API breaks the build (chapter 41). |
| `IsAotCompatible` | Analyzers report AOT and trimming problems at build time. |
| SourceLink, `.snupkg` | Users can debug into the plugin's code. Built into the SDK since .NET 8. |
| README in the package | Shown on nuget.org; minimal example and compatibility. |
| CHANGELOG | Mark breaking changes clearly, especially on EF major upgrades. |
| `InternalsVisibleTo` | Tests reach internal logic without public API. |
| Package names | Core without provider, relational layer as `.Relational`, provider packages with a suffix (`.PostgreSQL`, `.SqlServer`, `.Cosmos`). |

Besides the API docs, the documentation above all needs the things users do not see in the code: which single services are replaced (composability), which event IDs exist, which limitations apply (pooling) and which security assumptions hold (database roles).

# 29 The checklist from A to Z

Version dependency: **stable**

For every new plugin, in the order of development. Each item sums up a rule; the reasoning is in the chapters named under each heading.

## Architecture

Details: chapters 3, 4.

- ☐ Decide the level: helpers, model helpers, real plugin or provider?
- ☐ Separate core and provider packages; types for entity assemblies go into the core.
- ☐ For every extension point, look for a plugin interface first, replace only then.
- ☐ Check every mechanism against the simpler alternative with the decision guide.

## Options

Details: chapter 5.

- ☐ Extension immutable with copy constructor and virtual Clone.
- ☐ All model-relevant options in the hash, nothing that varies per context.
- ☐ Validate for preconditions (provider, combinations).
- ☐ Maintain LogFragment and PopulateDebugInfo.
- ☐ Use method in the namespace Microsoft.EntityFrameworkCore, generic and non-generic.

## Services

Details: chapters 6, 8.

- ☐ Registration through the services builder (lifetimes from EF’s table).
- ☐ Decorate single services where the public interface suffices.
- ☐ Otherwise Replace, and offer the logic publicly and virtually.
- ☐ ISingletonOptions for singleton services, registered twice.
- ☐ Classify every piece of state: request data never in options, hash, model, singleton or translated query.
- ☐ Reset scoped services with state for pooling (IResettableService).
- ☐ App dependencies via AddInterceptors, document pooling behavior.

## Model

Details: chapters 9, 10, 11.

- ☐ Annotations with a prefix, simple values only; keep explicit values (also defaults that overrule a convention), normalize them for migrations in the annotation provider.
- ☐ Implement the Fluent API non-generically, delegate generically.
- ☐ Convention builder overloads for ConfigurationSource.
- ☐ Conventions at ModelFinalizing when they depend on the finished model.
- ☐ Attributes in the core package, attribute conventions in the provider package.
- ☐ Validation as IModelFinalizedConvention: throw on errors, warn on questionable cases.
- ☐ Check table mappings against the support matrix and reject unsupported ones while the model is built (page 52).

## Type mappings

Details: chapter 12.

- ☐ Protected parameter constructor and Clone override.
- ☐ Literal generation for model and provider type.
- ☐ Public static Default property, shared converter instance.
- ☐ Shared value comparer instances, otherwise EF creates them through reflection (NativeAOT).
- ☐ JsonValueReaderWriter with Instance and ConstructorExpression.

## Queries

Details: chapters 13, 14, 15.

- ☐ EF.Functions methods in the core, translation in the provider package.
- ☐ Translators: null for foreign methods, correct nullability, set type mappings.
- ☐ IEvaluatableExpressionFilterPlugin for methods without data reference.
- ☐ Never insert run-time values at translation time (query cache, AOT).

### Run time

Details: chapters 18, 19.

- ☐ Implement interceptors synchronously and asynchronously.
- ☐ Tell the tracking flag from an actual change of value (page 84).
- ☐ Singleton interceptors stateless and thread-safe.
- ☐ Register singleton interceptors in ApplyServices or pass one shared instance, never a new one per context; get per-request data on every call through the context.
- ☐ Own event IDs in an own range, EventDefinitions cached.
- ☐ Security-relevant cases as warnings, escalatable via ConfigureWarnings.
- ☐ Where interceptors do not apply, offer an explicit API (e.g. ApplyTenantSession).

### Migrations & design time

Details: chapters 20, 21, 22, 23, 24, 25.

- ☐ Declarative via annotation provider (design time only) and SQL generator.
- ☐ Escape identifiers, literals via type mappings, validate self-generated SQL.
- ☐ Custom operations idempotent, extensions in the migrations namespace.
- ☐ Remove schema-bound objects before renames and recreate them with the final names afterwards (page 188).
- ☐ DesignTimeServicesReference with forProvider in the application’s startup project (not in the plugin); code generator removes annotations.
- ☐ Check compiled models: Default, Instance, runtime annotations.

### Quality

Details: chapters 26, 27, 28.

- ☐ Model, SQL, snapshot round trip and integration tests.
- ☐ Caching test with different options; test for no phantom migration.
- ☐ Version range per EF major, EF1001 deliberately; CI with a patch channel, a preview channel only as an early warning; upgrades following the playbook.
- ☐ XML docs, SourceLink, symbols, README, CHANGELOG, limitations documented.
- ☐ Check the packages as a consumer: tooling, compiled model, NativeAOT (page 105).
- ☐ Pin serialized annotation formats with golden-value tests.
- ☐ Test together with popular plugins (naming conventions, decorators) and check the coexistence with a second plugin in both orders (chapter 6).

## Advanced extension points (part VIII)

Details: chapters 30, 31, 32, 33, 34, 35.

- ☐ Document paths that bypass interceptors (ExecuteUpdate, raw SQL).
- ☐ Configure value generators with HasValueGeneratorFactory(Type) (compiled models).
- ☐ Replace IUpdateSqlGenerator only when the SQL needs a different shape; stateless (singleton) and exactly one per context.
- ☐ Check conventions for complex types and JSON columns.
- ☐ Document and test the order of custom migration operations.
- ☐ Consider idempotent scripts and suppressTransaction.

## Operation and integration (part IX)

Details: chapters 36, 37, 38, 39, 40, 41.

- ☐ Document operational assumptions (roles, pooling, transaction pooling).
- ☐ Document context lifetimes: factories, background jobs, open connections.
- ☐ Cross-tenant access only through separate roles, never through app switches.
- ☐ Document the threat model: what is protected and what is not (page 143).
- ☐ Tenant-safe relationships: composite foreign keys, unique per tenant.
- ☐ Retrofit existing tables in two migrations with a guard.
- ☐ Measure hot paths (interceptors), do not guess.
- ☐ Second-level caches: the tenant in every cache key, no caching without a tenant.
- ☐ Public API analyzer as errors, not as warnings.

## Deep extensions (part X)

Details: chapters 42, 43, 44, 45.

- ☐ Custom SQL nodes: VisitChildren, Update, Quote, Print, Equals/GetHashCode.
- ☐ Handle custom nodes in the SQL generator AND in the nullability processor.
- ☐ Aggregates: handle predicates, DISTINCT and empty sets deliberately.
- ☐ Choose runtime annotations deliberately, filter design-time annotations.
- ☐ Scaffolding: read the plugin’s schema objects from the database.

## Multiple providers (part XI)

Details: chapters 46, 47, 48.

- ☐ Provider-neutral building blocks into the core, differences into provider packages.
- ☐ Relational building blocks into a layer of their own, so that the core stays usable without SQL.
- ☐ Without row level security: document that the query filter is the only isolation, and how raw access bypasses it.

Part VIII

# Advanced extension points

Limits of the run-time hooks and further extension points: bulk operations, value generation, the update pipeline, complex types, models per runtime context and advanced migrations.


› Chapter 30: Bulk operations and the limits of interceptors

› Chapter 31: Value generation

› Chapter 32: The update pipeline

› Chapter 33: Complex types and JSON

› Chapter 34: Models per runtime context

› Chapter 35: Advanced migrations

# 30 Bulk operations and the limits of interceptors

Version dependency: **medium**

A SaveChanges interceptor only sees what goes through `SaveChanges`. `ExecuteUpdate` and `ExecuteDelete` write directly to the database, without change tracker and without SaveChanges. The same applies to raw SQL. A plugin that enforces its guarantees only through SaveChanges has a gap here:

| Path | SaveChanges interceptor | Connection interceptor | RLS |
|---|---|---|---|
| `SaveChanges` | yes | yes | yes |
| `ExecuteUpdate`, `ExecuteDelete` | **no** | yes | yes |
| `FromSql`, `SqlQuery`, `ExecuteSql` | **no** | yes | yes |
| ADO.NET or psql as the app role | **no** | **no** | yes (without tenant: no rows) |

In the sample, RLS catches every one of these paths because the database enforces the rules itself. An integration test proves it by deliberately bypassing the interceptor:

tests/...IntegrationTests/AdvancedTests.cs (excerpt)

```
[Fact]
public async Task ExecuteUpdate_bypasses_interceptor_but_not_rls()
{
    // ... create an order for "tenant" ...
    await using var attacker = ContextFor(tenant);

    // No SaveChanges -> the SaveChanges interceptor does NOT run.
    // The policy's WITH CHECK still prevents the reassignment.
    var ex = await Assert.ThrowsAsync<PostgresException>(() =>
        attacker.Orders.ExecuteUpdateAsync(
            s => s.SetProperty(o => o.TenantId, TenantId.New())));

    Assert.Equal(PostgresErrorCodes.InsufficientPrivilege, ex.SqlState);
}
```

**What a plugin must document**

Plugins without database-level protection (soft delete, audit fields, tenant filtering via query filter only) must document that bulk operations and raw SQL bypass their rules. To prevent it, an `IQueryExpressionInterceptor` can be used: `ExecuteUpdate` and `ExecuteDelete` go through the query pipeline, the interceptor sees their expression tree and can, for example, reject a `SetProperty` on protected columns. Raw SQL remains open regardless.

## Example: protecting the tenant column in bulk updates

The sample implements exactly this interceptor, as an option: `UseAcmeTenancy(t => t.ProtectBulkOperations())` for PostgreSQL, `sql.ProtectBulkOperations()` for SQL Server. EF Core 10 no

longer takes the setters of `ExecuteUpdate` as an expression tree but as an `Action<UpdateSetters Builder<T>>`. In the tree the interceptor sees, they then appear as an array of tuples: `Execute Update(source, new[] { new Tuple(e => e.TenantId, value), ... })`. The first element of each tuple is the property lambda, and that is what the visitor checks:

`Interceptors/TenantQueryExpressionInterceptor.cs (core package, shortened)`

```
protected override Expression VisitMethodCall(MethodCallExpression node)
{
    if (node.Method.DeclaringType == typeof(EntityFrameworkQueryableExtensions)
        && node.Method.Name is nameof(EntityFrameworkQueryableExtensions.ExecuteUpdate)
            or nameof(EntityFrameworkQueryableExtensions.ExecuteUpdateAsync)
        && node.Arguments.Count > 1
        && node.Arguments[1] is NewArrayExpression setters)
    {
        foreach (var setter in setters.Expressions)
        {
            if (setter is NewExpression { Arguments: [var selector, ..] }
                && Unquote(selector) is LambdaExpression lambda)
                Check(lambda);
        }
    }

    return base.VisitMethodCall(node);
}

private void Check(LambdaExpression selector)
{
    var parameter = selector.Parameters[0];
    var name = selector.Body switch
    {
        // e => e.TenantId
        MemberExpression member when member.Expression == parameter => member.Member.Name,
        // e => EF.Property<TenantId>(e, "TenantId"), also for shadow properties
        MethodCallExpression
        {
            Method.Name: nameof(EF.Property),
            Arguments: [var target, ConstantExpression { Value: string property }]
        } when target == parameter => property,
        _ => null,
    };
    // ...
}
```

If the property being set is the tenant property according to the runtime annotation, the interceptor throws a `TenancyException`, before any connection is opened. It runs only at compilation, so once per query shape before translation; executions served from the query cache cost nothing. Other columns and ordinary queries pass unchanged. RLS remains the actual boundary: the interceptor only stops reassigning rows early and with a clear message; it does not check `ExecuteDelete` or raw SQL.

The registration holds a trap. `IQueryExpressionInterceptor`, like `IMaterializationInterceptor`, derives from `ISingletonInterceptor`. Registered under that type in the internal provider, however, EF never calls it: EF takes query expression interceptors from the `IInterceptor` registrations and the options, materialization interceptors also from the `ISingletonInterceptor` registrations. The plugin therefore registers it as `IInterceptor`; stateless as it is, an instance per context does no

harm. `BulkOperationTests` records both registrations, the integration tests show an allowed and a rejected update against PostgreSQL.

# 31 Value generation

Version dependency: **stable**

EF generates key values either in the database (identity, sequences) or client-side via a `ValueGenerator`. For `Guid` keys EF does this by itself, for custom types like `TenantId` it does not. A plugin closes this gap with a generator, a factory and a convention:

ValueGeneration/TenantIdValueGenerator.cs (core package)

```
public class TenantIdValueGenerator : ValueGenerator<TenantId>
{
    // Final values, no temporary placeholders until saving
    public override bool GeneratesTemporaryValues => false;

    public override TenantId Next(EntityEntry entry) => TenantId.New();   // UUIDv7
}
```

ValueGeneration/TenantIdValueGeneratorFactory.cs (core package)

```
public class TenantIdValueGeneratorFactory : ValueGeneratorFactory
{
    // Stateless generator: EF may cache it and share it between contexts
    public override ValueGenerator Create(IProperty property, ITypeBase typeBase)
        => new TenantIdValueGenerator();
}
```

Conventions/TenantIdKeyValueGenerationConvention.cs (core package)

```
public virtual void ProcessModelFinalizing(
    IConventionModelBuilder modelBuilder,
    IConventionContext<IConventionModelBuilder> context)
{
    foreach (var entityType in modelBuilder.Metadata.GetEntityTypes())
    {
        if (entityType.BaseType is not null
            || entityType.FindPrimaryKey() is not { Properties: [var property] }
            || property.ClrType != typeof(TenantId))
            continue;

        // Convention source: an explicit ValueGeneratedNever() wins (-> null)
        if (property.Builder.ValueGenerated(ValueGenerated.OnAdd) is null)
            continue;

        property.Builder.HasValueGeneratorFactory(typeof(TenantIdValueGeneratorFactory));
    }
}
```

**A factory type, not a generator**

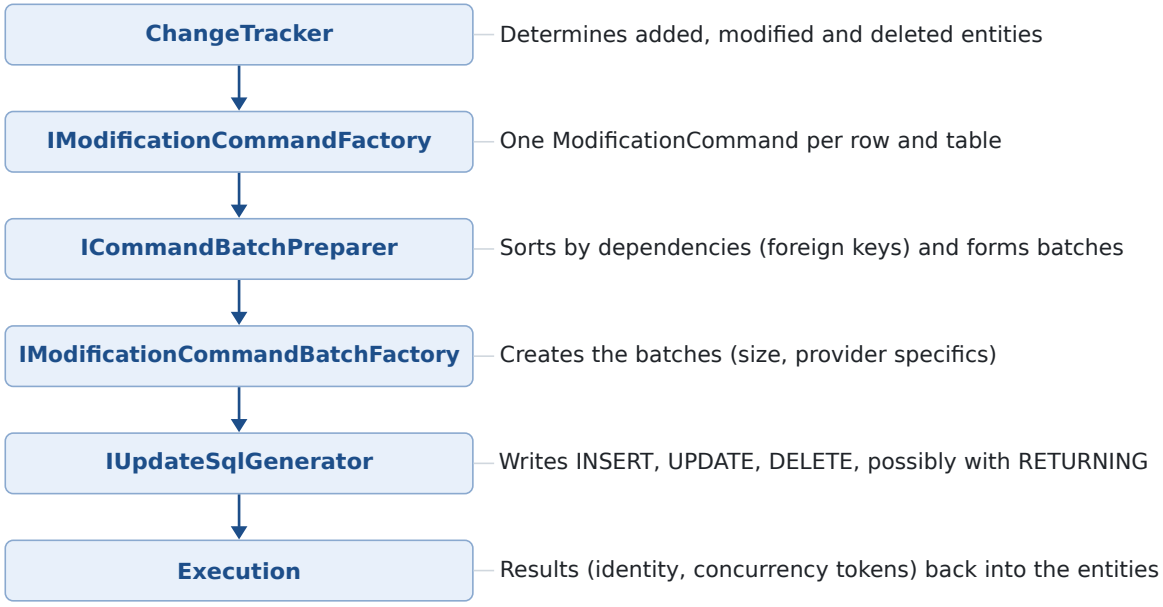
A value generator can be configured in several ways: `HasValueGenerator` with a type or a factory function, or `HasValueGeneratorFactory` with a factory type. EF Core 10's compiled model generator only accepts the last one and otherwise aborts with "The property ... has a value generator configured. Use 'HasValueGeneratorFactory' to configure the value generator factory type." Plugins should therefore use `HasValueGeneratorFactory(Type)` and give the factory a public parameterless constructor.

The convention lives in the core package because it has nothing to do with PostgreSQL; the provider package plugs it in via its convention set plugin. The compiled model test checks that the factory appears in the generated code.

# 32 The update pipeline

Version dependency: **volatile**

Chapter 13 describes the way from LINQ to SQL. Writing has its own pipeline, which plugins extend for bulk operations, database-side soft deletes or returned values:




| Service | Lifetime | Kind |
|---|---|---|
| `IModificationCommandFactory` | Singleton | single |
| `ICommandBatchPreparer` | Scoped | single |
| `IModificationCommandBatchFactory` | Scoped | single |
| `IUpdateSqlGenerator` | Singleton | single |

All of these are single provider services, with the consequences from chapter 6. Before replacing them, it pays to check the alternatives: a SaveChanges interceptor can rewrite states (delete becomes modify for soft delete), a command interceptor can add SQL, and for bulk changes there is `ExecuteUpdate`. A custom `IUpdateSqlGenerator` is only justified when the SQL itself needs a different shape, for example `INSERT ... ON CONFLICT` for upserts.

## Example: the database clock on UPDATE

A column `UpdatedAt` should hold the time of the last change, and the database's clock should decide it, not that of whichever application server wrote the row. For `INSERT` EF has the means: `HasDefaultValueSql("now()")`. For `UPDATE` there is none. `ValueGeneratedOnAddOrUpdate()` only tells EF that the database produces the value; EF leaves the column out of `SET`, reads it back with `RETURNING` and expects a trigger to have set it. The simpler means do not reach the goal:

- A **SaveChanges interceptor** can only set a value from the application: the application server's clock, not `now()` of the transaction.

- A **command interceptor** would have to edit finished SQL text, with parameter names and the `RETURNING` clause, in every batch: fragile.
- A **trigger** works, but it is a schema object outside the EF model that every migration must carry along.

What is missing is a different shape of the `UPDATE` itself, and only the `IUpdateSqlGenerator` writes it. The example is a teaching example in the test project, not part of the tenancy packages. A Fluent API method marks the property, the derived generator adds the assignment:

`tests/.../UpdatePipeline/DatabaseClockUpdateSqlGenerator.cs (shortened)`

```
public static PropertyBuilder<T> HasDatabaseClockOnUpdate<T>(this PropertyBuilder<T> builder)
{
    builder.Metadata.SetAnnotation(Annotation, true);
    return builder.HasDefaultValueSql("now()").ValueGeneratedOnAddOrUpdate();
}

public class DatabaseClockUpdateSqlGenerator : NpgsqlUpdateSqlGenerator   // internal (EF1001)
{
    protected override void AppendUpdateCommand(
        StringBuilder commandStringBuilder,
        string name,
        string? schema,
        IReadOnlyList<IColumnModification> writeOperations,
        IReadOnlyList<IColumnModification> readOperations,
        IReadOnlyList<IColumnModification> conditionOperations,
        bool appendReturningOneClause = false)
    {
        AppendUpdateCommandHeader(commandStringBuilder, name, schema, writeOperations);

        // EF's order, with the extra assignments after SET
        foreach (var column in readOperations.Where(IsDatabaseClock))
        {
            commandStringBuilder.Append(", ");
            SqlGenerationHelper.DelimitIdentifier(commandStringBuilder, column.ColumnName);
            commandStringBuilder.Append(" = now()");
        }

        AppendWhereClause(commandStringBuilder, conditionOperations);
        AppendReturningClause(commandStringBuilder, readOperations,
            appendReturningOneClause ? "1" : null);
        commandStringBuilder.AppendLine(SqlGenerationHelper.StatementTerminator);
    }
}
```

The override changes as little as possible. A generated-on-update column is a read operation for EF: it is in `RETURNING`, not in `SET`. The method writes the header from EF, appends `"column" = now()` for every marked read column, and leaves the `WHERE` clause with key and concurrency token and the `RETURNING` clause to EF. That is why the read-back value and optimistic concurrency keep working. With `xmin` as row version, SaveChanges sends:

SaveChanges after a change of Title

```
UPDATE "Documents" SET "Title" = @p0, "UpdatedAt" = now()
WHERE "Id" = @p1 AND xmin = @p2
RETURNING "UpdatedAt", xmin;
```

`IUpdateSqlGenerator` is a singleton (table above): the generator must not keep state per command; everything it needs comes from the parameters. It replaces a provider service with `ReplaceService`, here next to the tenancy plugin, which does not touch this service:

Registration

```
options.UseNpgsql(connectionString)
    .UseAcmeTenancy()
    .ReplaceService<IUpdateSqlGenerator, DatabaseClockUpdateSqlGenerator>();

modelBuilder.Entity<Document>(b =>
{
    b.Property(d => d.UpdatedAt).HasDatabaseClockOnUpdate();
    b.Property(d => d.Version).IsRowVersion();            // Npgsql: the system column xmin
});
```

The base class decides how far the example carries. It derives from `NpgsqlUpdateSqlGenerator`, an internal Npgsql class (EF1001): replacing it with EF's base class would lose Npgsql's own SQL, such as partial updates of JSON columns. That is the reason for this chapter's rating. The limits:

- **One per context.** There is exactly one update SQL generator. A second plugin that replaces it wins or loses depending on the order of registration; the two can only be combined by deriving one from the other.
- **Only SaveChanges.** `ExecuteUpdate` does not go through this pipeline (chapter 30); it has to set the column itself. Entities mapped to stored procedures do not either.
- **Provider-specific.** The override relies on PostgreSQL's `RETURNING`; SQL Server writes `OUTPUT` and needs a generator of its own.

`UpdatePipelineTests` checks the example against PostgreSQL: the recorded `UPDATE` has the assignment, the `WHERE` with `xmin` and the `RETURNING`; the entity afterwards holds exactly the database's value; and a second writer with an outdated version gets a `DbUpdateConcurrencyException` and changes nothing. Without `ReplaceService` the SQL test fails.

# 33 Complex types and JSON

Version dependency: **medium**

## Complex types

Complex types (since EF Core 8) are value types without their own identity whose properties end up in the owner's table. For plugins this means: conventions iterating `entityType.GetProperties()` do **not** see their properties. They hang off `IComplexProperty` and must be iterated separately:

Pattern for conventions (sketch)

```
foreach (var complexProperty in entityType.GetComplexProperties())
{
    foreach (var property in complexProperty.ComplexType.GetProperties())
    {
        // Column names are prefixed (e.g. "Address_City"); always determine them via
        // property.GetColumnName(table), never assemble them yourself
    }
}
```

Fluent API methods for properties should additionally be offered on `ComplexTypePropertyBuilder` where they make sense there. The sample plugin does not need this: isolation and value generation only concern entities.

## JSON columns

Owned types and (since EF Core 10) complex types can be stored as a single JSON column with `ToJson()`. Values in it are read and written through the type mapping's `JsonValueReaderWriter` (chapter 12). A value converter alone is enough only if EF can wrap the provider type's reader/writer around it (`JsonConvertedValueReaderWriter`); a custom mapping with neither makes model building fail. The sample's unit and integration tests check both sides:

tests/.../JsonTests.cs

```
using var ctx = Contexts.Create(model: m =>
    m.Entity<Shipment>().OwnsOne(s => s.Details, d => d.ToJson()));

var property = ctx.DesignModel()
    .FindEntityType(typeof(ShipmentDetails))!
    .FindProperty(nameof(ShipmentDetails.ApprovedBy))!;

Assert.Same(TenantIdJsonReaderWriter.Instance,
    property.GetTypeMapping().JsonValueReaderWriter);
```

JSON outside the database is a separate matter. System.Text.Json does not know EF's `JsonValueReaderWriter` and would write `TenantId` as an object with `Value` and `IsEmpty`. A `[JsonConverter(typeof(TenantIdJsonConverter))]` on the type in the core package turns it into a plain GUID, also as a dictionary key, without a web API having to register anything; invalid values lead to a `JsonException`. Both ways need a building block of their own, because EF and System.Text.Json serialize independently of each other (`TenantIdJsonConverterTests`).

> **Note**
>
> PostgreSQL arrays (`List<TenantId>` as `uuid[]`) are not JSON: Npgsql maps primitive collections natively as arrays. Whether a provider supports collections with converted element types is provider-specific and deserves its own test before you document it.

# 34 Models per runtime context

Version dependency: **stable**

EF builds the model once per internal service provider and context type and caches it (chapter 2). If the model should depend on run-time data, such as one schema per tenant, the cache key must contain that data. That is what `IModelCacheKeyFactory` is for:

tests/.../SchemaPerTenantTests.cs (shortened, simplified)

```
public sealed class SchemaModelCacheKeyFactory : IModelCacheKeyFactory
{
    public object Create(DbContext context, bool designTime)
        => context is ISchemaScoped scoped
            ? (context.GetType(), scoped.Schema, designTime)
            : (object)(context.GetType(), designTime);
}

public sealed class SchemaContext : DbContext, ISchemaScoped
{
    // Constructor: (DbContextOptions<SchemaContext> options, string schema)
    public string Schema { get; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
        => modelBuilder.HasDefaultSchema(Schema);
}

// Registration
options.UseNpgsql(connectionString)
       .ReplaceService<IModelCacheKeyFactory, SchemaModelCacheKeyFactory>();
```

The test `SchemaPerTenantTests` checks both sides of the cache key: two schemas produce two models with their own default schema, and the same schema reuses the cached model instead of building it again.

This is the basis for “schema per tenant”, an alternative to RLS. The three usual approaches compared:

| Approach | Strengths | Weaknesses |
|---|---|---|
| Shared tables + RLS (sample) | One model, one schema, simple migrations; the database enforces separation | All tenants share tables and indexes; misconfigured roles undermine separation |
| Schema per tenant | Strong separation, separate backups per schema possible | One model per tenant in the cache (memory!), migrations for every schema |
| Database per tenant | Maximum separation, independent scaling | One model suffices, but connections, migrations and operations per database |

> **Model cache and memory**
>
> Every key value produces its own model in memory. With hundreds of tenants and large models this becomes expensive. In addition, values that change the model must never come from unchecked input.

# 35 Advanced migrations

Version dependency: **stable**

## Order of operations

Generated and hand-written operations run in the order they appear in `Up()`. This is a real source of errors. In the sample, the policy `support_read` applies `TO shop_support`. EF generates it directly after `CREATE TABLE`. If the role is only created at the end of `Up()`, the migration fails.

Migrations/20261021_Initial.cs (hand-edited, simplified)

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    // 1. Whatever policies reference: BEFORE the generated operations
    migrationBuilder.CreateApplicationRole("shop_support");

    // 2. Generated by EF: tables, RLS, policies (TO shop_support)
    migrationBuilder.CreateTable(name: "Customers", /* ... */);

    // 3. Whatever requires the tables: AFTER the generated operations
    migrationBuilder.CreateApplicationRole("shop_app", canLogin: true);
    migrationBuilder.GrantTablePrivileges("shop_app");   // also future tables, sequences
}
```

## Idempotent scripts

`dotnet ef migrations script --idempotent` produces a script that can run against any database state. For this, Npgsql wraps every operation in a `DO $EF$ ... $EF$` block that checks the history table. Custom operations that produce a `DO $$` block themselves end up inside it. This works because the dollar quotes use different tags. Assumptions like this belong in a test:

tests/...IntegrationTests/AdvancedTests.cs (excerpt)

```
var script = db.GetService<IMigrator>()
    .GenerateScript(options: MigrationsSqlGenerationOptions.Idempotent);

await db.Database.ExecuteSqlRawAsync(script);   // nested DO valid?
await db.Database.ExecuteSqlRawAsync(script);   // second run is skipped
```

## Operations outside of transactions

Some commands must not run inside a transaction, in PostgreSQL for example `CREATE INDEX CONCURRENTLY`. Custom operations with such SQL end their command with `EndStatement(builder, suppressTransaction: true)`. EF then executes it outside the migration transaction. The price: if the migration fails afterwards, this command is not rolled back; it should therefore be idempotent.

## Migration bundles

`dotnet ef migrations bundle` produces an executable that applies migrations without the SDK. It contains the startup project including the plugin assemblies; custom operations and the plugin's SQL generator therefore take effect there too. It is important that the design-time factory or host uses the same plugin options as the application, otherwise SQL for a different model is produced. Page 154 shows how the bundle fits into a pipeline with separate roles.

Part IX

# Operation and integration

Operating in production: Row Level Security, tenant-safe relationships, retrofitting, performance, and the way from the plugin into the application and into CI.


› Chapter 36: Row Level Security in production

› Chapter 37: Tenant-safe relationships

› Chapter 38: Retrofitting and operating multi-tenancy

› Chapter 39: Plugin performance

› Chapter 40: Using the plugin in ASP.NET Core

› Chapter 41: CI and API stability

# 36 Row Level Security in production

Version dependency: **stable**

RLS is only as strong as its operating conditions. These pitfalls silently render the policies ineffective or unusable:

| Pitfall | Consequence | Remedy |
|---|---|---|
| App runs as table owner | Policies are ignored | `FORCE ROW LEVEL SECURITY` (plugin default) and separate roles |
| Superuser or `BYPASSRLS` | RLS is always bypassed, even with FORCE | App role with `NOSUPERUSER NOBYPASSRLS` (custom operation, chapter 22) |
| Setting not set | Without `nullif`: error; with it: empty results | The interceptor sets the value on every open |
| No index on the tenant column | Every query scans the whole table | Index convention (chapter 10) |
| Transaction pooling (PgBouncer) | Session settings reach other clients or get lost | `UseTransactionPooling()`: the setting transaction-locally (see below) |
| Connection opened outside EF | The session keeps an old tenant or none | Let EF open connections, or `ApplyTenantSession()` (chapter 18) |
| “Bypass” switch for the app role | Anyone who can run SQL as the app role can flip it | Separate roles for cross-tenant access (see below) |

## The threat model

RLS is a layer of protection with clear limits. Whoever uses it should know what it protects against, what it relies on and where it ends:

| Aspect | In the sample |
|---|---|
| Protected | The rows of tenant-owned tables in the supported mappings (page 52). Not protected are shared master data, the schema and, without composite keys, the existence of foreign values (chapter 37). |
| Trusted | The database server and its roles; the migration role as table owner; the code of the application and the plugin, including the interceptors; the `ITenantProvider` and the source it reads the tenant from. |
| Assumptions | The app role is neither superuser nor owner and has no `BYPASSRLS`. The tenant comes from an authenticated claim, not from a header or parameter the client chooses freely. EF opens the connections, and there is no transaction pooling, except with `UseTransactionPooling()`; with it, transactions come from EF (page 144). The model uses only configurations marked as supported in the contract on page 194. |
| Limit | The app role may set the session setting itself; it has to, so that the interceptor can set it. Whoever runs arbitrary SQL under this role switches the tenant with it. |

The limit is not a flaw of the plugin but a property of the session model. `ThreatModelTests` shows it against PostgreSQL: one `SELECT set_config('app.tenant_id', ...)` under the app role is

enough, and the query after it returns the other tenant's rows. `SqlServerThreatModelTests` shows the same with `sp_set_session_context`. This defines what RLS protects against here and what it does not:

- **Protected:** forgotten filters, faulty LINQ queries, the application's raw SQL with a wrong `WHERE`, bulk operations, reports that accidentally run through the app role. In all these cases the plugin sets the tenant, and the code does not change it.
- **Not protected:** SQL injection and compromised application code. Both can change the setting, even within a single statement, since `set_config` is a function. What helps against them are parameters instead of concatenated strings (EF parameterizes LINQ and `FromSql`/`SqlQuery` with interpolated strings; `FromSqlRaw` with concatenated values does not) and privileges that give the app role only the tables it needs.
- **Outside the database:** a forged or wrongly assigned claim grants access to exactly that tenant. That is the job of authentication; the plugin takes the tenant from the `ITenant Provider`.

The sample deliberately does not implement possible hardenings. SQL Server can freeze the key until the end of the connection with `sp_set_session_context ..., @read_only = 1` (SQL Server: Row-Level Security); the test above shows that a later change then fails. The price: `Apply TenantSession`, too, can no longer change the tenant of an open connection. PostgreSQL has no read-only setting; there remain a role per tenant with `SET ROLE` or a signed tenant token checked by a `SECURITY DEFINER` function. Both move complexity into the database and are only named as a direction here. Cross-tenant access still belongs in roles of its own (section below).

## Transaction pooling

PgBouncer in "transaction" mode shares a server connection between clients per transaction. A session value set with `set_config(..., false)` can then reach another client, a security hole. The safe variant sets the value *transaction-locally* with `set_config(..., true)`: it ends with commit or rollback. For that, the interceptor moves from `IDbConnectionInterceptor` to `IDbTransaction Interceptor` and sets the tenant in `TransactionStarted`, in a command of the new transaction. The sample offers this as an option:

Application

```
services.AddDbContext<ShopContext>((sp, options) => options
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp, t => t.UseTransactionPooling()));
```

Runtime/PostgreSqlTransactionSessionInterceptor.cs (shortened)

```
public override async ValueTask<DbTransaction> TransactionStartedAsync(
    DbConnection connection,
    TransactionEndEventData eventData,
    DbTransaction result,
    CancellationToken cancellationToken = default)
{
    await using var command = CreateSessionCommand(connection, result, eventData.Context);
    await command.ExecuteNonQueryAsync(cancellationToken);
    return result;
}
```

```
// SaveChanges always in a transaction, even for a single command
private static void RequireTransaction(DbContextEventData eventData)
{
    // ...
        context.Database.AutoTransactionBehavior = AutoTransactionBehavior.Always;
}

protected virtual DbCommand CreateSessionCommand(
    DbConnection connection, DbTransaction transaction, DbContext? context)
{
    // ...
    var command = connection.CreateCommand();
    command.Transaction = transaction;
    command.CommandText = "SELECT set_config(@name, @value, true)";   // is_local = true
    // ...
}
```

`UseAcmeTenancy(sp)` then registers this interceptor instead of the connection interceptor. Three rules come with it:

- **Reading in transactions.** Queries outside a transaction have no tenant and see no rows. That is safe, but the application has to put reading work that needs data into a transaction.
- **Saving always in a transaction.** EF omits the automatic transaction when `SaveChanges` runs only one command (`AutoTransactionBehavior.WhenNeeded`). A single insert would then run without a tenant and fail on the policy. The interceptor is therefore also an `ISaveChangesInterceptor` and sets `Always`.
- **No session value.** If the option is set and a `TenantConnectionInterceptor` is registered anyway, it refuses to set the value with a `TenancyException` instead of leaving a value that outlives the transaction. That also rules out `ApplyTenantSession`: a call fails with a message that the tenant is set per transaction and the commands belong in one.

The interceptor only acts on transactions that EF begins itself. That makes the way a transaction comes about part of the contract. The middle column also applies to SQL Server, which has no transaction pooling mode:

| Transaction path | Session mode (default) | `UseTransactionPooling()` |
|---|---|---|
| `BeginTransaction()` through EF | Supported and tested: EF opens the connection, the interceptor sets the session | Supported and tested: tenant in `TransactionStarted`, gone after commit and rollback |
| `SaveChanges` without its own transaction | Supported and tested | Supported and tested: the interceptor enforces a transaction, also for a single command |
| Query outside a transaction | Supported and tested | No tenant, no rows (fail closed, tested) |
| `UseTransaction()` with a transaction of its own | Only under conditions: EF opened the connection, or `ApplyTenantSession()` after opening it outside EF (`ConnectionLifetimeTests`); the call itself is not verified | Not supported: EF reports `TransactionUsed`, not `TransactionStarted`; no tenant, no rows (fail closed, tested) |

| Transaction path | Session mode (default) | `UseTransactionPooling()` |
|---|---|---|
| Ambient `TransactionScope` | Supported and tested: the connection enlists when it opens | Not supported: EF begins no transaction of its own; reads see no rows, writes fail on the policy (fail closed, tested) |

`TransactionPoolingTests` checks against PostgreSQL: inside the transaction the value is set and RLS applies, after commit and after rollback it is empty on the same connection, a single insert gets its transaction, the connection interceptor refuses the session value, `ApplyTenantSession` explains why it is not available, and `UseTransaction` as well as `TransactionScope` fail closed. PgBouncer itself is not part of the test; what is checked is the property that matters: that no value remains after the transaction.

## Policy performance

PostgreSQL inserts policies into every query like additional `WHERE` conditions. `current_setting` is `STABLE`. That guarantees the same result for all rows within one statement and *allows* the optimizer to combine several calls into one (PostgreSQL: Function Volatility Categories); it does not guarantee a fixed number of calls. How often the function actually runs depends on the plan. If the comparison is part of an `Index Cond`, PostgreSQL evaluates the comparison value once and reads only the tenant's rows. If it is a `Filter` over a seq scan, it may be evaluated per row, and then the scan weighs more than the function. Expensive policies come from subqueries (for example permissions from another table). In addition, PostgreSQL holds back conditions of the query whose functions are not `LEAKPROOF` behind the policy conditions, which can prevent optimizations (PostgreSQL: Row Security Policies).

Only the plan under real conditions is therefore meaningful: `EXPLAIN (ANALYZE, BUFFERS)` as the app role with a tenant set, with a realistic amount of data and current statistics. Measured as owner or superuser, the policies are missing from the plan, because superusers and roles with `BYPASSRLS` bypass them, and so does the owner without `FORCE`. The overhead shows in comparing the same query with and without the policy, not in the number of function calls.

## Checking the policies of a running database

A correct EF model does not prove that the running database has the protection objects: a migration may have been skipped, someone may have switched RLS off by hand, or a table was created outside EF. The repository therefore contains two read-only diagnosis scripts, `docs/deployment/check-tenant-policies.postgresql.sql` and `...sqlserver.sql`. They compare an *expected* state with the actual one; a mere list of existing policies could not show a missing one. The expected tables come from the design-time model, as a configuration line: every table with RLS and its tenant column. Deliberate exceptions have no RLS in the model and are therefore not expected:

tests/...IntegrationTests/PolicyDiagnosisTests.cs › ExpectedTables

```
public static string ExpectedTables(DbContext context)
{
    var tables =
        from entityType in context.GetService<IDesignTimeModel>().Model.GetEntityTypes()
        where entityType.IsRlsEnabled()
        let table = StoreObjectIdentifier.Create(entityType, StoreObjectType.Table)
```

```
        where table is not null
        let property = entityType.FindProperty(entityType.GetTenantPropertyName()!)!
        select $"{table.Value.Schema ?? "public"}.{table.Value.Name}:"
               + property.GetColumnName(table.Value);
    return string.Join(",", tables);
}
```

The line goes into a session setting (PostgreSQL, `SET diag.expected`) or into the session context (SQL Server, `sp_set_session_context N'diag.expected'`), together with the app role and, if they differ from the defaults, setting name, policy name, `diag.force` (PostgreSQL; `off` for tables with `force: false`, valid for the whole run), session key and policy schema. Then the script runs in the same session and returns one row per expected table:

Shell

```
psql "$AUDIT_CONNECTION" \
    -c "SET diag.expected = 'public.Orders:TenantId,public.Shipments:TenantId'" \
    -c "SET diag.app_role = 'shop_app'" \
    -f docs/deployment/check-tenant-policies.postgresql.sql

 schema_name | table_name |    status     |              reason
-------------+------------+---------------+----------------------------------
 public      | Orders     | NOT PROTECTED | row level security is disabled
 public      | Shipments  | OK            |
```

| Check | PostgreSQL | SQL Server |
|---|---|---|
| Table exists | `pg_class` | `sys.tables` |
| Protection active | RLS enabled; `FORCE` if expected (plugin default) | Policy with `STATE = ON` |
| Plugin’s predicate | Policy `tenant_isolation`, `FOR ALL`, `USING` and `WITH CHECK` compare the tenant column with the setting | `FILTER`, `BLOCK AFTER INSERT` and `AFTER UPDATE` with the predicate function on the tenant column; the function reads the expected session key |
| App role | No superuser, no `BYPASSRLS`, not the owner or a member of it; no further permissive policy that applies to it | Not in `db_owner`, no `ALTER ANY SECURITY POLICY` or `CONTROL` |
| Temporal table | Not applicable | History table without a policy: not protected (the model rejects temporal tables; this finds a table converted afterwards) |
| Read rights | `SELECT` on `pg_policy` (granted to everyone by default) | `VIEW DEFINITION` on the database or `db_owner` |

The result has three states. **OK** means every check above passed. **NOT PROTECTED** names the first failed check, for example a disabled RLS, a missing policy or an app role that owns the table. **NOT VERIFIABLE** means that the diagnosis cannot decide: the app role is unknown, no table is expected, or the caller lacks the read rights. On SQL Server this is the normal case for the app login, because catalog views silently hide what a caller has no rights on; there, a missing policy and an invisible one would look the same. Run the scripts with an audit role that has these read rights (on

SQL Server `VIEW DEFINITION`), not with the app role, and do not extend the app role's rights for it. The scripts never repair anything.

`PolicyDiagnosisTests` and `SqlServerPolicyDiagnosisTests` run both scripts against temporary databases: in the correct state, and after typical breakages, such as `DISABLE ROW LEVEL SECURITY`, a dropped policy, `STATE = OFF`, a missing block predicate, a table made temporal afterwards or a wrong session key. The limit: the diagnosis checks the objects the plugin creates and the app role's attributes, not every way to bypass them. Functions with `SECURITY DEFINER`, views without `security_invoker`, rights granted through other roles, and the threat model's limit (page 143) are outside of what it sees. A passed diagnosis is a plausibility check of the configuration, not proof of security.

## Legitimate cross-tenant access

Sooner or later someone needs data across tenants: reports for the operator, maintenance jobs, support. The tempting shortcut is a switch in the application, for example an additional session setting `app.bypass` that the policy honors. That is not a security boundary: whatever the application role can set, an injected SQL statement can set too. Separation must come from database roles instead:

| Need | Solution | Why it is safe |
|---|---|---|
| Reports, analytics | Separate read-only role with `BYPASSRLS`, only `SELECT`, its own connection string and a context configuration without tenant | The app role cannot become this role; its credentials live elsewhere |
| Support reads selected data | Additional permissive policy for a support role: `HasPolicy(...)` with `For(Select)` and `To("shop_support")` | Restricted to SELECT and to members of the role |
| Maintenance per tenant | Background job that iterates the tenants and sets each one (chapter 40) | Runs through the normal policies |

Reporting role (tests/...IntegrationTests/CrossTenantAccessTests.cs, simplified)

```
-- Created by an administrator: BYPASSRLS requires a superuser
CREATE ROLE shop_reporting LOGIN PASSWORD '...' NOSUPERUSER BYPASSRLS;
GRANT USAGE ON SCHEMA public TO shop_reporting;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO shop_reporting;
```

The application role, in contrast, cannot switch the policies off. `SET row_security = off` does not disable RLS for a role without `BYPASSRLS`; PostgreSQL instead rejects every query the policies would affect. `CrossTenantAccessTests` checks all three: the reporting role sees all tenants, it cannot write, and the app role cannot help itself.

> **Audit cross-tenant access**
>
> Every connection of the reporting role is an access to all tenants. Log it (`log_connections`, pgaudit) and grant the role to as few people and services as possible. The plugin's `CreateApplicationRole` deliberately always creates roles with `NOBYPASSRLS`: bypass roles do not belong in the application's migrations.

# 37 Tenant-safe relationships

Version dependency: **stable**

Which table mappings the plugin protects and which it rejects while the model is built is set by the matrix on page 52; this chapter does not repeat it. It is about the second axis: relationships between protected tables. RLS governs which *rows* a session sees. Whether a row only refers to rows of the same tenant is not governed by RLS. That is *referential tenant consistency*, and it comes from the model.

The sample uses customer, order and item: `Customer` → `Order` → `OrderItem`, all three `ITenantOwned`, plus a shared `Product` without a tenant.

## Visibility is not consistency

PostgreSQL documents explicitly that checks of foreign keys, unique and primary keys bypass RLS to maintain data integrity, and warns of “covert channels” through these checks (PostgreSQL: Row Security Policies). The SQL Server documentation says nothing explicit about it; `SqlServerThreatModelTests` shows that the foreign key check there also sees rows the filter predicate hides. With foreign keys on the ID alone, as EF creates them by default, this follows:

| Observation | Cause |
|---|---|
| Tenant A adds an item to an order of B | The FK check finds the order, although A cannot see it. |
| A learns whether a foreign ID exists | Unknown IDs fail on the foreign key, foreign ones do not. |
| The own item vanishes from `Include(i => i.Order)` | A required navigation becomes an `INNER JOIN`, and the order is invisible. |
| A deletes an own order, an item of B disappears | The cascading delete is a referential action and bypasses RLS as well. |
| A learns that an e-mail address exists at B | A global unique index reports the collision. |

`RelationshipTests` proves every row against PostgreSQL with a restricted role. None of these cases is a read leak in the strict sense; A sees no foreign row. But the data loses its tenant boundary, and the error messages answer questions A should not be able to ask.

## Composite foreign keys

The solution is to make the tenant part of every reference. The principal gets an alternate key of tenant and ID, the dependent side a foreign key of its tenant column and the ID:

`tests/...IntegrationTests/RelationshipTests.cs › TenantSafeContext`

```
modelBuilder.Entity<Customer>(b =>
{
    b.HasAlternateKey(c => new { c.TenantId, c.Id });
    b.HasIndex(c => new { c.TenantId, c.Email }).IsUnique();   // unique per tenant
});
```

```
modelBuilder.Entity<Order>(b =>
{
    b.HasAlternateKey(o => new { o.TenantId, o.Id });
    b.HasOne(o => o.Customer).WithMany(c => c.Orders)
        .HasForeignKey(o => new { o.TenantId, o.CustomerId })
        .HasPrincipalKey(c => new { c.TenantId, c.Id })
        .OnDelete(DeleteBehavior.Restrict);                  // customers are not cascaded
});

modelBuilder.Entity<OrderItem>()
    .HasOne(i => i.Order).WithMany(o => o.Items)
    .HasForeignKey(i => new { i.TenantId, i.OrderId })
    .HasPrincipalKey(o => new { o.TenantId, o.Id })
    .OnDelete(DeleteBehavior.Cascade);                        // items belong to the order
```

Now an item of A refers to the pair (A, ID). An order of B never satisfies it, and the error is the same as for an unknown ID: there is nothing left to learn. The tenant column is part of the foreign key and the tenant property at the same time. The SaveChanges interceptor sets it on new items, and fixup through the navigation sets the same value. No index is needed for the policy, because key and foreign key start with the tenant column; the index convention recognizes that.

The price is an additional unique index per principal and a wider foreign key. The plugin does not create these keys automatically. A convention could, but it would have to rewrite every relationship between protected types and reconcile it with explicit configuration. The sample leaves the decision to the application's model.

## Uniqueness per tenant

A unique index over a business column applies to the whole table, that is across all tenants. Uniqueness per tenant is almost always what is meant: `(TenantId, Email)` instead of `Email`. Where uniqueness really has to be global, for example for the tenants' subdomains, the column belongs in the tenant directory, which deliberately holds no tenant data.

## Shared master data

`Product` has no tenant, no policy and is readable by everyone. Foreign keys from tenant data to master data are unproblematic; the test reads the product through the item. The opposite direction, master data with references to tenant data, must not exist. Writing to master data should not be allowed to the app role; `GrantTablePrivileges` grants privileges on all tables of a schema, a finer grant is up to the application's migrations.

## Join tables

The plugin rejects implicit many-to-many tables when one side is tenant-owned. A join entity with a tenant property of its own is protected (page 52). For consistency it gets the same composite foreign keys as every dependent table: `(TenantId, LeftId)` and `(TenantId, RightId)` on the alternate keys of both sides. Then a link can only connect two rows of the same tenant.

## Delete rules

Cascading deletes run in the database as referential actions and bypass RLS. With composite foreign keys they still stay within the tenant, because every dependent row carries the same tenant. Without them a cascade deletes foreign rows; the test `Naive_cascade_deletes_rows_of_another_tenant` shows exactly that. For roots such as the customer, `Restrict` is usually right, for parts of an aggregate such as items `Cascade`. EF also cascades in the change tracker, but only for loaded entities; the rule in the database is what counts.

# 38 Retrofitting and operating multi-tenancy

Version dependency: **stable**

The previous chapters assume a new database. Usually, however, there already are tables with data, a running application and users who keep working during the change. This chapter shows the way from a table without a tenant column to RLS, and after that operation under retries and lost connections. All steps are executable in `OperationsTests` against PostgreSQL. The chapter closes with deployment: migrations in the pipeline with separate roles, and changes without downtime.

## From a table without a tenant to RLS

Whoever simply lets an existing entity implement `ITenantOwned` gets a single migration: a column with the empty tenant as default, RLS, policy, index. After it nobody sees the old rows any more. A backfill in the same migration does not reliably help either: with `FORCE ROW LEVEL SECURITY` the policy also applies to the migration role, and without a tenant set its `UPDATE` finds no row. The way therefore leads through two migrations, following the pattern expand, fill, protect:

| No. | Step | Approach |
|---|---|---|
| 1 | Expand | The entity implements `ITenantOwned` but stays excluded with `HasNoRowLevelSecurity()`. The migration only adds the column. |
| 2 | Fill | As owner, in the same or a migration of its own: every row gets its tenant from whatever knows the assignment, for example a mapping table or the tenant of the parent row. |
| 3 | Switch the application | An app version whose model no longer has the exclusion: the interceptor sets the tenant on new rows. The database does not protect yet; the model does not check that. |
| 4 | Close the gap | Assign rows that were created without a tenant between steps 1 and 3. |
| 5 | Protect | Second migration: exclusion removed, so `ENABLE`/`FORCE`, policy and index. First of all a guard that stops as long as a row has no tenant. |
| 6 | Tighten | Optionally composite keys and unique indexes per tenant (chapter 37). |

The guard is a step of its own at the start of `Up`, before the operations EF generates:

Guard (SQL)

```
DO $$ BEGIN
  IF EXISTS (SELECT 1 FROM "Tickets"
             WHERE "TenantId" = '00000000-0000-0000-0000-000000000000') THEN
    RAISE EXCEPTION 'Tickets without a tenant: backfill first';
  END IF;
END $$;
```

Migrations/…_EnableTicketIsolation.cs (example)

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    // Before anything else: do not protect without a complete assignment
    migrationBuilder.Sql(TicketGuardSql);      // the guard above, as a constant

    // ... generated by EF: ENABLE/FORCE ROW LEVEL SECURITY, CREATE POLICY, CREATE INDEX
}
```

The migration runs in a transaction. If the guard fires, the table stays without RLS, and no half-done state arises; `Guard_stops_enabling_rls_while_rows_lack_a_tenant` checks this. After a complete assignment every tenant sees exactly its rows.

### App versions, Down and rollback

During the change two app versions often run at the same time, and each meets one of the two states of the database:

| Database | App without tenant | App with tenant |
|---|---|---|
| After migration 1 (column, no RLS) | Runs; new rows without a tenant (step 4) | Runs, sets the tenant; not yet protected |
| After migration 2 (RLS) | Fail closed: reads nothing, writes fail | Protected |

The old app version must therefore no longer run once migration 2 is applied. Rolling the application back to a version without a tenant thus first requires the Down of migration 2. The Down removes policy and RLS; column and values stay, but the table is unprotected again. The test checks both. A Down of migration 1 drops the column and with it the assignment; it belongs in development only.

Policy changes follow the same pattern. If the setting name changes (`UseSettingName`), the migration replaces the policy in its transaction. Between the migration and the new app version the policy reads a setting the running app does not set, and the database refuses everything until both match. A transition in which the interceptor sets both settings is conceivable but not implemented. For SQL Server, page 189 describes the same case.

## Retries, transactions and lost connections

The plugin sets the tenant when EF opens a connection. This determines the behavior under EF's resilience mechanisms:

- **Retries.** An execution strategy such as `EnableRetryOnFailure` repeats an operation after a transient error on a newly opened connection. The interceptor runs again, the tenant is set. The test ends the session on the server with `pg_terminate_backend`; the next query fails, is repeated and returns only rows of the tenant.
- **Explicit transactions.** With a retrying strategy EF refuses commands in a transaction the application began itself, because a retry would have to repeat the whole transaction. The transaction therefore belongs into `CreateExecutionStrategy().ExecuteAsync(…)`. Inside, EF opens the connection once, sets the tenant and keeps it until the commit. If the connection breaks in the middle of the transaction, the strategy repeats the whole delegate on a new connection; side effects outside the database then have to be idempotent.

- **Session reinitialization.** Npgsql resets a connection's state when it returns to the pool, as long as `No Reset On Close` is not enabled (Npgsql: Connection String Parameters). SqlClient also resets the session context when it reuses a pooled connection (`sp_reset_connection`). The plugin relies on neither, but sets the tenant on every open; only connections that EF does not open need `ApplyTenantSession` (chapter 18).

For transaction pooling, `UseTransactionPooling()` sets the tenant per transaction (page 144). Not implemented is a read-only session key (page 143). `OperationsTests` contains the lost connection and the transaction inside the strategy; the cases of open connections are in `ConnectionLifetimeTests`.

## Deployment: pipeline, roles and zero downtime

The samples create the database in development with `DevelopmentDatabase`. In production, every schema change belongs in a pipeline that strictly separates two roles:

- **`shop_owner`** owns schema, tables and policies and runs the migrations.
- **`shop_app`** is the application's role: only `SELECT`, `INSERT`, `UPDATE`, `DELETE`, always under the policies.

The application therefore never calls `db.Database.Migrate()` on startup. Its role would need DDL rights for that, and whoever may alter a table may also run `DISABLE ROW LEVEL SECURITY` or drop the policy. `FORCE` binds the owner to the policy but does not protect the policy from the owner. A hole in the application, such as an SQL injection, would then no longer be contained by the database. Operations add to that: in a rolling deployment several instances start at once, and a failed migration should stop the rollout, not keep the instances in a restart loop.

### Once: the roles

Before the first pipeline runs, the database administrators create the roles. That is not a migration, because migrations already run as the owner. The script is in the repository:

`docs/deployment/setup-roles.sql`

```
CREATE ROLE shop_owner LOGIN NOSUPERUSER NOBYPASSRLS PASSWORD :'owner_password';
CREATE ROLE shop_app LOGIN NOSUPERUSER NOBYPASSRLS PASSWORD :'app_password';
CREATE DATABASE shop OWNER shop_owner;

...
GRANT USAGE ON SCHEMA public TO shop_app;

ALTER DEFAULT PRIVILEGES FOR ROLE shop_owner IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO shop_app;
ALTER DEFAULT PRIVILEGES FOR ROLE shop_owner IN SCHEMA public
    GRANT USAGE, SELECT ON SEQUENCES TO shop_app;
```

The elision is `\connect shop`: from there on, the script runs in the new database. The `DEFAULT PRIVILEGES` matter: every table that a later migration creates as `shop_owner` is usable by `shop_app` right away. Without them, the new application version fails with `permission denied`, although migration and policy are correct. The same holds for sequences: an `INSERT` into a table with a `serial` column calls `nextval` on its sequence, and EF's HiLo pattern does so before the `INSERT`; without `USAGE` on the sequence the app role gets `permission denied for sequence`. An identity

column (`GENERATED ... AS IDENTITY`, Npgsql's default) needs no privilege of its own on its internal sequence for an `INSERT`. The plugin's `GrantTablePrivileges` operation also sets default privileges for tables and sequences, but only for objects that the role running it creates later; the script sets them for `shop_owner` independently of who runs it.

### The pipeline




The template `docs/deployment/migrations-deploy.yml` implements this with three jobs. It is not active in the repository; an application copies it to `.github/workflows` and replaces the deploy step with its platform's:

`docs/deployment/migrations-deploy.yml (shortened)`

```
jobs:
  build:
    steps:
      ...
      - name: Migration bundle
        run: >
          dotnet ef migrations bundle --project "$APP_PROJECT" --context "$DB_CONTEXT"
          --configuration Release --self-contained -r linux-x64 -o artifacts/efbundle
      ...
      - run: dotnet publish "$APP_PROJECT" -c Release -o artifacts/app

  migrate:
    needs: build
    environment: production
    steps:
      ...
      - name: Apply pending migrations as shop_owner
        env:
          OWNER_CONNECTION: ${{ secrets.SHOP_OWNER_CONNECTION_STRING }}
        run: |
          chmod +x artifacts/efbundle           # artifacts lose the executable bit
```

```
          artifacts/efbundle --connection "$OWNER_CONNECTION"

  deploy:
    needs: migrate
    environment: production
    ...
```

The bundle contains the startup project with the plugin; for its options, what section “Migration bundles” in chapter 35 says applies. `--connection` overrides the design-time factory’s connection, and a second run applies nothing. Only the `migrate` job knows the owner connection string; the application’s is given to the hosting platform, not to the pipeline.

The consumer test (`eng/consumer-test/run.sh --nativeaot`, page 105) runs exactly this sequence against PostgreSQL in Docker: it runs `setup-roles.sql` unchanged, builds the bundle from the generated migration, applies it twice as `shop_owner` (the second run changes nothing), checks that `shop_app` may not alter the table (`must be owner`), and starts the application as `shop_app`, which sees only its tenant’s rows.

### Zero downtime under RLS: expand and contract

In a rolling or blue-green deployment, the migration runs before the new version, and for a while old and new instances work against the same schema. Every migration must therefore work with the previous version of the application. The expand-and-contract pattern splits a change into three phases for that, like the conversion in section “From a table without a tenant to RLS”:

| Phase | Schema | What to watch under RLS |
|---|---|---|
| 1 Expand | New columns nullable or with a default, new tables, new indexes; nothing is removed or renamed. | New tables with `ITenantOwned` get their policy right away; the old version does not use them. If a policy has to change, a second one is added that allows the new form. |
| 2 Roll out | Unchanged. The new version writes to the new columns, old instances drain. | Both versions set the same tenant; the policies must admit the rows of both versions. |
| 3 Contract | Follow-up migration: drop old columns, set `NOT NULL`, remove temporary policies. | Only when no old instance is running; before that a guard as in section “From a table without a tenant to RLS”. |

For policies, a PostgreSQL property applies: several `PERMISSIVE` policies of a table are combined with `OR`. An additional policy thus widens what is visible and serves as the transition in phase 1; removing it in phase 3 is the contraction. Each of these policies must separate tenants on its own, otherwise the transition opens the table. Because DDL is transactional in PostgreSQL, replacing a policy within one migration leaves no moment without a policy. The diagnosis (page 146) reports such a transition policy as NOT PROTECTED as long as it applies to the app role; between phase 1 and phase 3 that is expected.

Rolling back the application is possible without a database change until phase 3, because the schema still supports the old version. After that it needs the Down of the follow-up migration, and dropped columns do not come back with it. A Down therefore does not belong in the pipeline; an error after phase 3 is fixed forward with a new migration.

# 39 Plugin performance

Version dependency: **stable**

Plugin code runs at three frequencies. Knowing which code runs how often tells you where optimization pays off:

| Frequency | Code | Consequence |
|---|---|---|
| Once per internal provider or model | Conventions, type mapping plugins, singleton options | Readability over micro-optimization |
| Once per query shape | Translators, query interceptors | Moderate; the result is cached (chapter 13) |
| On every execution | Interceptors for commands, connections, SaveChanges, materialization; evaluatable filter when the funcletizer extracts the parameter values | Hot path: no reflection, no unnecessary allocations |

This leads to a few rules the sample follows: `MethodInfo` and `PropertyInfo` lookups live in static fields. The SaveChanges interceptor only collects entries when there are tenant-relevant changes. The connection interceptor costs one round trip per open; with many small queries in one request the application therefore opens the connection once with `OpenConnectionAsync()`, and “one context, one tenant” applies (chapter 18). Such assumptions should be measured, not guessed. The repository contains a BenchmarkDotNet project; the benchmark for the SaveChanges logic runs without a database:

benchmarks/.../SaveChangesInterceptorBenchmark.cs (shortened)

```
[MemoryDiagnoser]
public class SaveChangesInterceptorBenchmark
{
    [Params(1, 100)]
    public int Entities { get; set; }

    // GlobalSetup: context with UseAcmeTenancy() (no connection) and the interceptor

    [Benchmark(Baseline = true)]
    public int TrackOnly()                  // what EF does anyway before saving
    {
        AddOrders();
        var count = _db.ChangeTracker.Entries().Count();
        _db.ChangeTracker.Clear();
        return count;
    }

    [Benchmark]
    public int TrackAndApplyTenant()        // the same plus the plugin's SaveChanges logic
    {
        AddOrders();
        _interceptor.ApplyTenant(_db);
        var count = _db.ChangeTracker.Entries().Count();
        _db.ChangeTracker.Clear();
        return count;
```

```
    }
}
```

Running the benchmark

```
dotnet run -c Release --project benchmarks/Acme.EntityFrameworkCore.Tenancy.Benchmarks \
    -- --filter '*SaveChanges*' --job short
```

| Method | Entities | Mean | Ratio | Allocated |
|---|---|---|---|---|
| TrackOnly | 1 | 1.5 µs | 1.00 | 1.8 KB |
| TrackAndApplyTenant | 1 | 2.3 µs | 1.54 | 2.7 KB |
| TrackOnly | 100 | 123 µs | 1.00 | 134 KB |
| TrackAndApplyTenant | 100 | 170 µs | 1.38 | 202 KB |

Example results (short job, .NET 10, author's machine; absolute values vary, the ratios are more meaningful). The plugin adds about half a microsecond per saved entity, roughly 40 to 50 percent on top of change tracking alone. That sounds like much, but a single round trip to the database costs hundreds of microseconds; the connection interceptor's extra round trip per open (chapter 18) therefore weighs far more than the SaveChanges logic. That is the kind of insight a benchmark provides: it shows where optimizing is worthwhile and where it is not.

## Reading and query compilation

Two more benchmarks cover the other frequencies. `MaterializationInterceptorBenchmark` measures what `VerifyMaterializedTenants()` costs when reading. It needs real rows from a `DbDataReader` and starts PostgreSQL with Testcontainers (Docker) for that. The connection stays open, so the connection interceptor's round trip is not measured; `AsNoTracking` keeps change tracking out. A third variant with an empty `IMaterializationInterceptor` separates EF's cost from the plugin's:

benchmarks/.../MaterializationInterceptorBenchmark.cs (shortened)

```
[MemoryDiagnoser]
public class MaterializationInterceptorBenchmark
{
    [Params(10, 100, 1000)]
    public int Rows { get; set; }

    // GlobalSetup: container, 1000 rows of one tenant, connections open

    [Benchmark(Baseline = true)]
    public Task<int> Read() => CountAsync(_plain);

    [Benchmark]
    public Task<int> ReadIntercepted() => CountAsync(_intercepted);

    [Benchmark]
    public Task<int> ReadAndVerify() => CountAsync(_verifying);

    private async Task<int> CountAsync(BenchmarkContext db)
        => (await db.Orders.AsNoTracking().OrderBy(o => o.Id).Take(Rows).ToListAsync()).Count;

    // Create: UseAcmeTenancy(sp, t => t.VerifyMaterializedTenants(verify))
```

```
}
```

| Method | Rows | Mean | Ratio | Allocated |
|---|---|---|---|---|
| `Read` | 10 | 200 µs | 1.00 | 12 KB |
| `ReadIntercepted` | 10 | 210 µs | 1.08 | 21 KB |
| `ReadAndVerify` | 10 | 220 µs | 1.12 | 22 KB |
| `Read` | 100 | 278 µs | 1.00 | 36 KB |
| `ReadIntercepted` | 100 | 764 µs | 2.77 | 121 KB |
| `ReadAndVerify` | 100 | 741 µs | 2.68 | 136 KB |
| `Read` | 1000 | 846 µs | 1.00 | 269 KB |
| `ReadIntercepted` | 1000 | 3858 µs | 4.70 | 1120 KB |
| `ReadAndVerify` | 1000 | 4288 µs | 5.22 | 1268 KB |

The result is surprising: most of the cost is not the check, but having a materialization interceptor registered at all. EF then produces, for every row, the data that `MaterializationInterceptionData` offers, and leaves the fast, compiled materialization path: at 1000 rows about 0.85 KB and 3 µs per row. The plugin's check (provider from the cache, read the value, compare) adds around 0.15 KB and 0.4 µs per row. At ten rows both disappear in the round trip; at a thousand rows reading on the client becomes about five times as expensive.

`QueryCompilationBenchmark` needs no database: `ToQueryString` runs the pipeline up to the SQL without a connection. `Compile` empties EF's query cache (the `IMemoryCache` of the internal service provider) before every call; `CacheHit` runs the same query shape with a new parameter value. Compared are EF Core alone, the plugin, and the plugin with its query filter:

| Context | Compile | Allocated | Cache hit | Allocated |
|---|---|---|---|---|
| EF Core | 2.64 ms | 146 KB | 18 µs | 9.9 KB |
| Plugin | 2.68 ms | 147 KB | 15 µs | 9.9 KB |
| Plugin with query filter | 2.78 ms | 160 KB | 15 µs | 10.2 KB |

Translator plugins and the evaluatable filter cost nothing measurable when compiling; the query filter makes the expression tree, and thus the translation, a few percent longer. On a cache hit every difference disappears in the noise. All values come from a short job (.NET 10, author's machine); its error bars are wide, but the orders of magnitude are clear:

| Phase | Frequency | Plugin overhead |
|---|---|---|
| Compilation | Once per query shape | Hardly measurable, a few percent with the query filter |
| Cache hit | Every query | Not measurable |
| SaveChanges | Every save | About 0.5 µs per entity (40 to 50 percent of change tracking) |
| Materialization | Every row read | About 3.5 µs per row with `VerifyMaterializedTenants()`, none without |

A dry run checks that all benchmarks run, without meaningful measurements:

```
All benchmarks, dry run
```

```
dotnet run -c Release --project benchmarks/Acme.EntityFrameworkCore.Tenancy.Benchmarks \
    -- --filter '*' --job dry
```

### When the check on reading pays off

`VerifyMaterializedTenants()` is defense in depth: it does not protect against an attack that RLS does not stop, but notices when RLS does not apply at all, for example because the application connects as the owner, a superuser or a role with `BYPASSRLS`, or works on a self-opened connection without `ApplyTenantSession`. That leads to the recommendation:

- **Turn it on** when a misconfiguration costs more than the CPU time: data with high protection requirements, many environments with their own configuration, small result sets as typical API queries return them. There the cost disappears in the round trip.
- **Turn it off** for read-heavy paths with large result sets, such as exports and reports. There the database carries the isolation alone, backed by the role separation (page 154) and by tests that read with the application's restricted role.
- **In between**: always on in tests and staging, in production only for the context of the sensitive data. The option applies per context configuration, not globally.

> **Tip**
>
> For the startup of large models, the compiled model (chapter 24) is the most effective lever. A plugin's conventions then no longer run at run time at all; that is why it pays to test compiled model support rather than merely claim it.

## Second-level caches

A second-level cache (for example `EFCoreSecondLevelCacheInterceptor`, or results kept in Redis or `IMemoryCache`) is the obvious next step when queries are too slow. With multi-tenancy it is also the shortest way to a data leak. The cache identifies a result by the SQL text or the normalized LINQ expression, and with row level security the tenant appears in neither: the policy reads it from the session.

| Step | What happens |
|---|---|
| Tenant A queries | The database filters by the session value, the cache stores the rows of A under the SQL text |
| Tenant B runs the same query | Same SQL text, same key: the cache returns the rows of A. The database is never asked, so the policies never run |

The plugin's other safeguards do not help either. The SaveChanges interceptor only sees saving. The materialization check (chapter 17) only runs when EF builds entities from rows; a cache that stores the entities or DTOs hands them out without that step. A filter with the tenant as a query parameter (Cosmos DB, test mode) puts the tenant into the key only if the cache includes parameter values, which not every cache does.

> **The tenant belongs in every cache key**
>
> A cache in front of a tenant-owned query is only safe if its key contains the current tenant, set by the application, not derived from the SQL. Without a tenant nothing is cached. Most cache libraries have a hook for the key or a key prefix; where they have none, the cache does not belong in front of tenant data. A test with two tenants and the same query shows whether the hook works.

Example

```
// Wherever query results are cached: the tenant belongs in every key
public sealed class TenantCacheKeys
{
    private readonly ITenantProvider _tenants;

    public TenantCacheKeys(ITenantProvider tenants) => _tenants = tenants;

    public string For(string queryKey)
        => _tenants.CurrentTenantId is { } tenant
            ? $"tenant:{tenant}:{queryKey}"
            : throw new InvalidOperationException("No tenant: do not cache the result.");
}
```

# 40 Using the plugin in ASP.NET Core

Version dependency: **stable**

The sample `samples/ShopApi` shows the way from the plugin into a web API. The tenant comes from the claim `tenant_id` of the signed-in user:

samples/ShopApi/HttpTenantProvider.cs

```
public sealed class HttpTenantProvider : ITenantProvider
{
    public const string ClaimType = "tenant_id";

    private readonly IHttpContextAccessor _httpContextAccessor;

    public HttpTenantProvider(IHttpContextAccessor httpContextAccessor)
    {
        ArgumentNullException.ThrowIfNull(httpContextAccessor);

        _httpContextAccessor = httpContextAccessor;
    }

    // Determined anew on every access: also works as a singleton (pooling)
    public TenantId? CurrentTenantId
    {
        get
        {
            var value = _httpContextAccessor.HttpContext?.User.FindFirst(ClaimType)?.Value;
            return Guid.TryParse(value, out var id) ? new TenantId(id) : null;
        }
    }
}
```

samples/ShopApi/Program.cs (excerpt)

```
builder.Services.AddHttpContextAccessor();
builder.Services.AddAcmeTenancy<HttpTenantProvider>();
builder.Services.AddDbContext<ShopApiContext>((sp, options) => options
    .UseNpgsql(builder.Configuration.GetConnectionString("Shop"))
    .UseAcmeTenancy(sp));

// No tenant filter in code: RLS in the database provides the separation
app.MapGet("/orders", async (ShopApiContext db) =>
    await db.Orders.OrderBy(o => o.Id).ToListAsync());
```

In the development environment the sample creates its database, the tables with RLS and the app role on startup, and the header `X-Demo-Tenant` simulates the claim. With the database from `docker-compose.yml` the whole sample runs like this:

Shell

```
docker compose up -d
dotnet run --project samples/ShopApi

curl -X POST -H "X-Demo-Tenant: 0199f0d2-0000-7000-8000-00000000000a" \
    "http://localhost:5000/orders?total=10"
```

```
curl -X POST -H "X-Demo-Tenant: 0199f0d2-0000-7000-8000-00000000000b" \
    "http://localhost:5000/orders?total=99"
curl -H "X-Demo-Tenant: 0199f0d2-0000-7000-8000-00000000000a" \
    "http://localhost:5000/orders"        # only the order of tenant ...0a
curl "http://localhost:5000/orders"       # without a tenant: []
```

> **Where the tenant comes from**
>
> The tenant must come from a trustworthy source: a claim from a signed token. A header or query parameter that the client sets freely undermines the entire isolation. The sample contains a header simulation for the development environment only and labels it explicitly.

For `AddDbContextPool`, register the provider as a singleton. This is possible because `HttpTenantProvider` determines the tenant anew on every access; the interceptors ask it on every connection and every save:

Variant with pooling

```
builder.Services.AddAcmeTenancy<HttpTenantProvider>(ServiceLifetime.Singleton);
builder.Services.AddDbContextPool<ShopApiContext>((sp, options) => options
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp));   // sp: root provider; interceptors are created once
```

## DbContext factories

`AddDbContextFactory` is the usual choice in Blazor Server and for parallel work. Unlike `AddDbContext`, it registers the factory and the options as **singletons** by default. The options callback then receives the root provider, not the scope of the current request. For a scoped tenant provider that holds state per scope, this has consequences:

| Registration | Behavior |
|---|---|
| Singleton factory, scope validation on (ASP.NET Core in Development) | Exception: “Cannot resolve scoped service ’ITenantProvider’ from root provider.” |
| Singleton factory, scope validation off (production default) | No error. The interceptors get an instance from the root container that no request ever fills: no tenant, saving throws, queries see nothing |
| Factory registered as `Scoped` | Every scope gets its own factory with its own tenant provider: correct |

Factory with the tenant of the request

```
builder.Services.AddDbContextFactory<ShopApiContext>(
    (sp, options) => options.UseNpgsql(connectionString).UseAcmeTenancy(sp),
    ServiceLifetime.Scoped);         // factory and options per scope
```

A provider that determines the tenant anew on every access, such as `HttpTenantProvider`, works in every variant; the problem affects providers with state per scope. `AddPooledDbContextFactory` always behaves like pooling: the options exist once, so the tenant provider must be such a singleton. `DbContextFactoryTests` shows all three variants.

## Outside of HTTP requests

Background services, message handlers and scheduled jobs have no request and no claim. The tenant then comes from the job itself, for example from the message or from a list of tenants to process. The sample's `AmbientTenantProvider` stores it in an `AsyncLocal`, which flows with the asynchronous control flow:

samples/ShopSample/AmbientTenantProvider.cs (shortened)

```
public sealed class AmbientTenantProvider : ITenantProvider
{
    private static readonly AsyncLocal<TenantId?> Current = new();

    public TenantId? CurrentTenantId => Current.Value;

    public static IDisposable Use(TenantId tenant)
    {
        var previous = Current.Value;
        Current.Value = tenant;
        return new Restore(previous);     // sets the previous tenant back on Dispose
    }

    // private sealed class Restore : IDisposable { ... Current.Value = _previous; }
}
```

Job pattern (tests/.../BackgroundTenantTests.cs, simplified)

```
// Tenant first, then scope and context: the interceptors are created inside the scope
using (AmbientTenantProvider.Use(tenant))
{
    await using var scope = scopeFactory.CreateAsyncScope();
    var db = scope.ServiceProvider.GetRequiredService<ShopContext>();
    // ... work for this tenant ...
}
```

Because the provider reads the `AsyncLocal` on every access, it can be registered as a singleton. Parallel jobs keep their own tenant, and after the `using` block no tenant is left behind; `BackgroundTenantTests` checks both with 20 parallel jobs. An application with web requests *and* jobs combines both sources in one provider: the claim first, otherwise the ambient tenant.

**Messages.** With a message broker (RabbitMQ, Kafka, Azure Service Bus) or a framework on top of it (MassTransit, Wolverine), the sender writes the current tenant into a header of the message, and the receiving side sets it before the consumer runs. The place for that is the framework's consume filter or middleware, so that no single consumer can forget it:

Consume filter (simplified, independent of the framework)

```
// Wherever the broker hands over a message: tenant from the header, then scope and context
public async Task HandleAsync(BrokerMessage message, CancellationToken cancellationToken)
{
    if (!message.Headers.TryGetValue("X-Tenant-Id", out var value)
        || !Guid.TryParse(value, out var id))
        throw new InvalidOperationException("Message without a tenant: rejected.");

    using (AmbientTenantProvider.Use(new TenantId(id)))
    {
        await using var scope = scopeFactory.CreateAsyncScope();
```

```
        var handler = scope.ServiceProvider.GetRequiredService<IOrderHandler>();
        await handler.HandleAsync(message, cancellationToken);   // uses ShopContext
    }
}
```

Two rules carry over from the jobs. The header is read before the scope is created, because the context and its interceptors live in the scope. And a message without a valid tenant is rejected and ends up in the dead letter queue; it is never processed for all tenants. A scoped tenant provider with a setter works the same way, as long as the filter sets the tenant in the consumer's scope before the context is resolved.

> **Never fall back to "all tenants"**
>
> Without a tenant, a job must fail or see nothing, never everything. The plugin behaves this way: the session value stays empty, the policies match no rows and saving throws. Jobs that really need to read all tenants use the read-only reporting role from chapter 36; maintenance that writes goes through the tenants one by one.

# 41 CI and API stability

Version dependency: **stable**

## The workflow

The GitHub Actions workflow in the repository implements the recommendations from chapters 26 and 27. The job `build` builds and tests without and with a database (the Cosmos model tests blocking), `handbook` builds the handbook, `consumer` checks the packages as a consumer. The job `cosmos` (emulator, not blocking yet) is left out of the excerpt:

`.github/workflows/ci.yml (shortened)`

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { global-json-file: global.json }
      - run: dotnet build -c Release
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.Tests -c Release --no-build
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.IntegrationTests
          -c Release --no-build
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.SqlServer.Tests
          -c Release --no-build
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.Cosmos.Tests
          -c Release --no-build --filter "Category!=Emulator"

  handbook:                           # both PDFs with LuaLaTeX, in docs/handbook
    runs-on: ubuntu-latest
    container: texlive/texlive@sha256:540d…   # TeX Live 2026, pinned by digest
    steps:
      # ... checkout ...
      - run: python3 check_code.py       # line length, excerpts against the sources
      - run: python3 check_structure.py  # editions in sync, references, tables
      - run: latexmk EF-Core-Plugin-Handbuch.tex
      - run: latexmk EF-Core-Plugin-Handbook.tex

  consumer:                           # check the packages as a consumer
    steps:
      - run: eng/consumer-test/run.sh --nativeaot

  handbook-examples:                  # run the listings marked with "% run:"
    steps:
      - run: python3 docs/handbook/check_examples.py
```

Code excerpts go stale silently when the code changes. A comment line `% source: <path>` directly before the listing therefore binds an excerpt to its source file, and `check_code.py` compares the two: comments and elisions (`...`) do not count, so the German edition may translate comments and excerpts may be shortened; every other line must appear in the source in the same order, line breaks may differ. With the suffix `(strings)` translated messages are left out as well. An excerpt whose title names a file of the repository and that is not bound must be called “simplified”. `check_structure.py` checks that both editions have the same sections, listings, bindings, tables and

callouts per chapter, that every reference target exists and that the column widths of every table add up.

Both scripts read the book; neither runs anything. A command that ends successfully without doing its job, such as a test filter that matches no test, would pass them. A few listings are therefore *executed*: a comment line `% run: <kind>` marks them, and `check_examples.py` runs them in the CI job `handbook-examples`, in temporary directories and containers that it removes again. Currently these are two listings: the test command in the exercises appendix, for which the script reads the counters from a TRX result file and requires at least one executed and no failed test, and the session of `ShopApi` in chapter 40, whose curl requests run against the sample and PostgreSQL and must show the isolation. Every other listing is not executed: C# excerpts are bound with `% source:` or marked as simplified, workflow and configuration excerpts are shortened, and listings with elisions are not programs. The packages as a whole, the `dotnet ef` tooling and NativeAOT are covered by the consumer test, not by this script.

The sample builds against stable versions only (.NET 10, EF Core 10). A second workflow, `compatibility.yml`, runs weekly and tests against the newest stable 10.0.x patches, because the regular CI builds against the beginning of the version range. The channels and a sketch for the next EF major version with previews and RCs are described in section “Patches and previews in CI” of chapter 27.

## Protecting the public API

A plugin has two public APIs: the one for users (Fluent API, options, types) and the one for extenders (virtual methods, dependencies, public services). Both break easily and unnoticed. The `Microsoft.CodeAnalysis.PublicApiAnalyzers` package keeps the public API in two text files (`PublicAPI.Shipped.txt`, `PublicAPI.Unshipped.txt`) and reports every deviation. Every API change thus becomes visible as a diff in the pull request.

> **Setup**
>
> On the first build, the analyzer reports every public API as RS0016 because the files are empty. The code fix “Add all items … to the public API” fills them, on the command line with `dotnet format analyzers --diagnostics RS0016 --severity warn`. At release time, the content moves from Unshipped to Shipped; from then on, removing a shipped API is a visible breaking change.

> **Warnings are not enough**
>
> As long as RS0016 and RS0017 are only warnings, they get lost in the build output. In the sample, the files thus stayed empty up to edition 1.0: 178 warnings in a full build, none in incremental ones, and nobody noticed. `Directory.Build.props` therefore turns both into errors (`WarningsAsErrors`); a new public API now has to be entered in `PublicAPI.Unshipped.txt`, otherwise the build breaks.

Part X

# Deep extensions

Custom SQL expressions, aggregates, run-time metadata and scaffolding: the tools for very complex plugins.


- Chapter 42: Custom SQL expression nodes
- Chapter 43: Aggregate translators
- Chapter 44: Property-level Fluent API and runtime annotations
- Chapter 45: Scaffolding in practice

# 42 Custom SQL expression nodes

Version dependency: **volatile**

Chapter 14 builds SQL from existing building blocks: functions, constants, casts. That is enough for most plugins. A **custom expression node** (a subclass of `SqlExpression`) becomes necessary when at least one of the following applies:

| Reason | Example |
|---|---|
| The syntax is not a function call | Operators such as `@@`, `<->`, `IS DISTINCT FROM`, keywords such as `AT TIME ZONE` |
| The nullability semantics differ | `IS DISTINCT FROM` is never NULL, even when an operand is NULL |
| Other parts should recognize the node | Optimizations that only apply to this expression |

The sample implements the null-safe comparison `EF.Functions.IsNotDistinctFrom(a, b)` and `IsDistinctFrom(a, b)` in a small package of its own, `Acme.EntityFrameworkCore.Functions.PostgreSQL`, activated with `UseNpgsql(cs, o => o.UseAcmeFunctions())`. Npgsql does not offer the comparison as a function; it meets all three reasons.

> **A teaching example, deliberately not about tenancy**
>
> The comparison has nothing to do with multi-tenancy. It lives in a sample package of its own because a custom node forces a plugin to replace two single services of the provider, the SQL generator and the nullability processor. Up to edition 1.1 the tenancy plugin did so for this example alone, which made it harder to combine with other plugins. A tenancy plugin needs no custom node; function composition (chapter 14) is enough. The tests run both plugins side by side, and the pgvector example at the end of this chapter adds a third one that derives from the functions plugin's generator.

## Anatomy of a node

| Member | Purpose | Consequence if missing |
|---|---|---|
| `VisitChildren` / `Update` | Visitors can replace children; unchanged children yield the same instance | EF throws at the first visitor |
| `Equals` / `GetHashCode` | Structural comparison for the query cache and optimizations | Cache misses, wrong deduplication |
| `Print` | Readable representation in DebugView and error messages | Unclear errors |
| `Quote` | C# code for precompiled queries and NativeAOT | No AOT support |
| SQL generator | Writes the node as SQL | **Silently** missing SQL |
| Nullability processor | Knows the NULL semantics | Exception during translation |

The last two rows also apply to combinations with other plugins: if a second plugin replaces the generator or the processor, the node's SQL is silently missing or the query throws. `CompositionMatrixTests` shows both (chapter 6).

Query/Expressions/DistinctFromExpression.cs (shortened)

```
public class DistinctFromExpression : SqlExpression
{
    private static ConstructorInfo? _quotingConstructor;

    public DistinctFromExpression(SqlExpression left, SqlExpression right, bool negated,
        RelationalTypeMapping? typeMapping)
        : base(typeof(bool), typeMapping)
    {
        // ... ThrowIfNull ...
        Left = left;
        Right = right;
        IsNegated = negated;
    }

    public virtual SqlExpression Left { get; }
    public virtual SqlExpression Right { get; }
    public virtual bool IsNegated { get; }

    protected override Expression VisitChildren(ExpressionVisitor visitor)
        => Update((SqlExpression)visitor.Visit(Left), (SqlExpression)visitor.Visit(Right));

    public virtual DistinctFromExpression Update(SqlExpression left, SqlExpression right)
        => left == Left && right == Right
            ? this
            : new DistinctFromExpression(left, right, IsNegated, TypeMapping);

    public override Expression Quote()
        => New(
            _quotingConstructor ??= typeof(DistinctFromExpression).GetConstructor(
                [typeof(SqlExpression), typeof(SqlExpression), typeof(bool),
                 typeof(RelationalTypeMapping)])!,
            Left.Quote(), Right.Quote(), Constant(IsNegated),
            RelationalExpressionQuotingUtilities.QuoteTypeMapping(TypeMapping));

    protected override void Print(ExpressionPrinter expressionPrinter)
    {
        expressionPrinter.Visit(Left);
        expressionPrinter.Append(IsNegated ? " IS DISTINCT FROM " : " IS NOT DISTINCT FROM ");
        expressionPrinter.Visit(Right);
    }

    public override bool Equals(object? obj)
        => obj is DistinctFromExpression other
           && (ReferenceEquals(this, other)
               || base.Equals(other) && Left.Equals(other.Left)
                  && Right.Equals(other.Right) && IsNegated == other.IsNegated);

    public override int GetHashCode()
        => HashCode.Combine(base.GetHashCode(), Left, Right, IsNegated);
}
```

The translator creates the node and aligns the type mappings of both sides: if one side has a mapping (a column, say), the other gets the same one. That way a constant becomes the matching literal:

Query/DistinctFromTranslator.cs (excerpt)

```
var definition = method.GetGenericMethodDefinition();
if (definition != IsNotDistinctFromMethod && definition != IsDistinctFromMethod)
    return null;

var left = arguments[1];                        // arguments[0] is DbFunctions
var right = arguments[2];
var typeMapping = ExpressionExtensions.InferTypeMapping(left, right)
                  ?? _typeMappingSource.FindMapping(left.Type);

return new DistinctFromExpression(
    _sql.ApplyTypeMapping(left, typeMapping)!,
    _sql.ApplyTypeMapping(right, typeMapping)!,
    negated: definition == IsDistinctFromMethod,
    _typeMappingSource.FindMapping(typeof(bool)));
```

## Extending the SQL generator

The `QuerySqlGenerator` writes SQL using a visitor. It hands unknown nodes to `ExpressionVisitor.VisitExtension`, which only visits the children. Without an override there is no error, but **SQL without the operator**. The functions plugin derives from the Npgsql generator:

Query/Internal/FunctionsNpgsqlQuerySqlGenerator.cs (shortened)

```
public class FunctionsNpgsqlQuerySqlGenerator : NpgsqlQuerySqlGenerator
{
    public FunctionsNpgsqlQuerySqlGenerator(QuerySqlGeneratorDependencies dependencies,
        IRelationalTypeMappingSource typeMappingSource,
        bool reverseNullOrderingEnabled, Version postgresVersion)
        : base(dependencies, typeMappingSource, reverseNullOrderingEnabled, postgresVersion)
    {
    }

    protected override Expression VisitExtension(Expression extensionExpression)
        => extensionExpression switch
        {
            DistinctFromExpression e => VisitDistinctFrom(e),
            _ => base.VisitExtension(extensionExpression),
        };

    protected virtual Expression VisitDistinctFrom(DistinctFromExpression expression)
    {
        VisitOperand(expression.Left);
        Sql.Append(expression.IsNegated ? " IS DISTINCT FROM " : " IS NOT DISTINCT FROM ");
        VisitOperand(expression.Right);
        return expression;
    }

    // The node as an operand of other operators: always in parentheses
    protected override bool RequiresParentheses(SqlExpression outerExpression,
        SqlExpression innerExpression)
        => innerExpression is DistinctFromExpression
           || base.RequiresParentheses(outerExpression, innerExpression);
}
```

Parentheses are needed in two directions. `VisitOperand` parenthesizes compound operands *inside* the node. For the node *as* an operand, EF asks `RequiresParentheses`, which only knows EF's own nodes. Without the override the plugin wrote `a IS DISTINCT FROM 'x' <> a IS DISTINCT FROM 'y'`; PostgreSQL binds `<>` tighter than `IS` and reports a syntax error. `DeepQueryTests` checks the SQL, `AdvancedTests` the query against PostgreSQL.

The generator is created through `IQuerySqlGeneratorFactory`, a singleton single service. The plugin replaces it with its own factory taking the same dependencies as Npgsql's (`QuerySqlGeneratorDependencies`, type mapping source, `INpgsqlSingletonOptions`).

## Extending the nullability processor

After translation, EF optimizes NULL comparisons. To do so it must know for every node whether it can become NULL. Unknown nodes end up in `VisitCustomSqlExpression`, whose base implementation throws an exception. The processor hangs off a chain of three classes, all of which must be replaced:

Query/Internal/FunctionsNpgsqlSqlNullabilityProcessor.cs (shortened)

```
public class FunctionsNpgsqlSqlNullabilityProcessor : NpgsqlSqlNullabilityProcessor
{
    // Constructor: (…ProcessorDependencies, …ProcessorParameters) : base(...)

    protected override SqlExpression VisitCustomSqlExpression(
        SqlExpression sqlExpression, bool allowOptimizedExpansion, out bool nullable)
    {
        if (sqlExpression is DistinctFromExpression distinctFrom)
        {
            // Visit children (they can be optimized), their nullability does not matter
            var left = Visit(distinctFrom.Left, out _)!;
            var right = Visit(distinctFrom.Right, out _)!;

            nullable = false;                   // IS [NOT] DISTINCT FROM is never NULL
            return distinctFrom.Update(left, right);
        }

        return base.VisitCustomSqlExpression(
            sqlExpression, allowOptimizedExpansion, out nullable);
    }
}

public class FunctionsNpgsqlParameterBasedSqlProcessor : NpgsqlParameterBasedSqlProcessor
{
    protected override Expression ProcessSqlNullability(
        Expression selectExpression, ParametersCacheDecorator parametersDecorator)
        => new FunctionsNpgsqlSqlNullabilityProcessor(Dependencies, Parameters)
            .Process(selectExpression, parametersDecorator);
}

// + FunctionsNpgsqlParameterBasedSqlProcessorFactory
//   : IRelationalParameterBasedSqlProcessorFactory
```

Infrastructure/FunctionsOptionsExtension.cs › ApplyServices (addition)

```
// 4) Deep extension: custom SQL expression node
services.Replace(ServiceDescriptor
    .Singleton<IQuerySqlGeneratorFactory, FunctionsNpgsqlQuerySqlGeneratorFactory>());
services.Replace(ServiceDescriptor
    .Scoped<IRelationalParameterBasedSqlProcessorFactory,
        FunctionsNpgsqlParameterBasedSqlProcessorFactory>());
```

**The price of custom nodes**

Custom nodes require replacing two single services and deriving from internal Npgsql classes. Two plugins that both introduce custom nodes can only be combined if one derives from the other. Hence: function composition first; custom nodes only when one of the three reasons applies. And the replacement belongs prominently in the documentation.

**Experimental API**

`Quote` and `RelationalExpressionQuotingUtilities` belong to the experimental API for precompiled queries. The compiler reports their use as EF9100; the sample suppresses it project-wide and documents this in `Directory.Build.props`.

tests/.../DeepQueryTests.cs (excerpt)

```
[Fact]
public void IsDistinctFrom_in_projection_passes_nullability_processor()
{
    // Without a custom nullability processor, EF would throw here
    using var ctx = Contexts.Create();

    var sql = ctx.Customers
        .Select(c => EF.Functions.IsDistinctFrom(c.Name, (string?)null))
        .ToQueryString();

    Assert.Contains("IS DISTINCT FROM", sql);
}
```

## Practical example: operators for vector search and pgvector

Embeddings and similarity search bring operators into SQL that are not functions: pgvector writes the L2 distance as `<->`, the cosine distance as `<=>` and the negative inner product as `<#>`; full-text search uses `@@`. `SqlBinaryExpression` does not help here, because its operator is an `ExpressionType`, and there is none for `<=>`. `CustomOperatorTests` therefore shows `EF.Functions.CosineDistance(a, b)` as a node of its own, `VectorDistanceExpression`, with the operator as a string, with translator, generator and nullability processor. It is test code: an application uses `Pgvector.EntityFrameworkCore`, which brings the `vector` type, its type mapping and these translations. Npgsql itself has `PgUnknownBinaryExpression` for such cases, a node with an arbitrary operator and propagating nullability that needs no replaced generator or processor. It lives in an `Internal` namespace, though (EF1001).

Three things decide whether such an operator produces correct SQL.

**Type mappings.** The translator gives both operands the same mapping, inferred from the side that has one, usually the column. The parameter thus reaches the database with the column's store type:

tests/.../CustomOperatorTests.cs › Translator

```
if (method != CosineDistanceMethod)
    return null;

var left = arguments[1];                        // arguments[0] is DbFunctions
var right = arguments[2];
var operandMapping = ExpressionExtensions.InferTypeMapping(left, right)
                     ?? _typeMappingSource.FindMapping(left.Type);

return new VectorDistanceExpression(
    _sql.ApplyTypeMapping(left, operandMapping)!,
    _sql.ApplyTypeMapping(right, operandMapping)!,
    "<=>",
    _typeMappingSource.FindMapping(typeof(double)));
```

In the test the column is a `float[]`, i.e. `real[]`. pgvector defines its operators only for `vector`, though; against a real database this needs a type mapping with that store type, as the package registers it. The result gets the mapping of `double`, i.e. `double precision`.

**Precedence and parentheses.** EF does not know the precedence of a foreign operator. PostgreSQL ranks user-defined operators between the arithmetic and the comparison operators: `a <=> b * 2` means `a <=> (b * 2)`. Without a countermeasure, the generator wrote exactly that for `CosineDistance(...) * 2 + 1`. The solution is the same two directions as above: parenthesize operands inside the node, and always parenthesize the node as an operand through `RequiresParentheses`. `Print` parenthesizes the same way, so that the DebugView shows what the database reads.

tests/.../CustomOperatorTests.cs › Generator

```
public class VectorQuerySqlGenerator : FunctionsNpgsqlQuerySqlGenerator
{
    // ... constructor, VisitExtension, VisitOperand

    protected virtual Expression VisitVectorDistance(VectorDistanceExpression expression)
    {
        VisitOperand(expression.Left);
        Sql.Append($" {expression.Operator} ");
        VisitOperand(expression.Right);
        return expression;
    }

    // Without the override: "a <=> b * 2.0", read as a <=> (b * 2.0)
    protected override bool RequiresParentheses(SqlExpression outerExpression,
        SqlExpression innerExpression)
        => innerExpression is VectorDistanceExpression
           || base.RequiresParentheses(outerExpression, innerExpression);
}
```

The generator derives from the plugin’s generator, not from Npgsql’s: both replace the same single service, and only this way do both nodes remain translatable. That is the price from the box above, on a small scale.

**Nullability.** An ordinary operator is NULL as soon as an operand is NULL. The processor reports that onwards:

tests/.../CustomOperatorTests.cs › Nullability

```
if (sqlExpression is VectorDistanceExpression distance)
{
    // NULL in, NULL out
    var left = Visit(distance.Left, out var leftNullable)!;
    var right = Visit(distance.Right, out var rightNullable)!;

    nullable = leftNullable || rightNullable;
    return distance.Update(left, right);
}
```

The report has visible consequences. For `!(CosineDistance(...) < 0.5)` EF writes `>= 0.5` for a required column, but a `CASE` for an optional one, because `NOT (NULL < 0.5)` is NULL in SQL but `true` in C#. Without the processor, translation fails. The tests check the SQL of all cases:

tests/.../CustomOperatorTests.cs (excerpt)

```
var sql = ctx.Documents
    .Select(d => EF.Functions.CosineDistance(d.Embedding, query) * 2 + 1)
    .ToQueryString();

Assert.Contains("SELECT (d.\"Embedding\" <=> @query) * 2.0 + 1.0", sql);
```

# 43 Aggregate translators

Version dependency: **medium**

Aggregates such as `Sum` or `string_agg` combine several rows, typically after `GroupBy`. An `IAggregateMethodCallTranslator` receives, instead of a single argument, an `Enumerable Expression` that describes *what* is being aggregated:

| Property | Meaning | Origin in LINQ |
|---|---|---|
| `Selector` | The aggregated expression | `g.Select(o => o.Total > 0)` |
| `IsDistinct` | Distinct values only | `g.Select(...).Distinct()` |
| `Predicate` | Filter before the aggregate | `g.Where(o => o.Id > 10)` |
| `Orderings` | Order (for order-dependent aggregates) | `g.OrderBy(...)` |

The functions sample package from chapter 42 adds `EF.Functions.Every(values)` for PostgreSQL's `bool_and`; like the comparison, it is a teaching example unrelated to tenancy:

Query/EveryAggregateTranslator.cs (excerpt)

```
public virtual SqlExpression? Translate(MethodInfo method, EnumerableExpression source,
    IReadOnlyList<SqlExpression> arguments,
    IDiagnosticsLogger<DbLoggerCategory.Query> logger)
{
    if (method != EveryMethod || source.Selector is not SqlExpression selector)
        return null;

    // Npgsql's factory implements DISTINCT, FILTER (WHERE ...) and ORDER BY in aggregates
    if (_sql is not NpgsqlSqlExpressionFactory npgsql)
        return null;

    var boolean = _typeMappingSource.FindMapping(typeof(bool))!;

    // bool_and returns NULL for an empty set, Enumerable.All returns true
    return _sql.Coalesce(
        npgsql.AggregateFunction("bool_and", [selector], source,
            nullable: true, argumentsPropagateNullability: [false],
            typeof(bool), boolean),
        _sql.Constant(true, boolean),
        boolean);
}
```

Generated SQL for g.Where(o => o.Id > 10).Select(o => o.Total > 0)

```
SELECT COALESCE(bool_and(o."Total" > 0.0) FILTER (WHERE o."Id" > 10), TRUE)
FROM "Orders" AS o
GROUP BY o."TenantId"
```

**Semantics before syntax**

An aggregate translator must match the semantics of the .NET side, not just the database syntax. Typical pitfalls: empty sets (NULL instead of 0, true or an empty string), NULL values in the input, and predicates.

> Npgsql itself aligns `string.Join` with `string_agg` using `COALESCE`; the sample follows this pattern.

Databases without a `FILTER` clause (such as SQL Server) emulate predicates with `CASE WHEN predicate THEN value END`. A provider-neutral aggregate translator must know both; this one uses the Npgsql factory and therefore lives in the PostgreSQL-specific sample package.

# 44 Property-level Fluent API and runtime annotations

Version dependency: **volatile**

## Property builder

So far the plugin assumes that the tenant property is called `TenantId` and that `ITenantOwned` is implemented. With a Fluent API method on `PropertyBuilder` it works without either, for example with existing models using a property `MandantId` or an explicit interface implementation:

Usage

```
modelBuilder.Entity<Contract>().Property(c => c.Mandant).IsTenantProperty();
```

Metadata/TenancyPropertyBuilderExtensions.cs (core package)

```
public static PropertyBuilder IsTenantProperty(
    this PropertyBuilder builder, bool isTenant = true)
{
    ArgumentNullException.ThrowIfNull(builder);

    if (isTenant)
        builder.Metadata.SetAnnotation(TenancyAnnotationNames.IsTenantProperty, true);
    else
        builder.Metadata.RemoveAnnotation(TenancyAnnotationNames.IsTenantProperty);

    return builder;
}

public static PropertyBuilder<TProperty> IsTenantProperty<TProperty>(
    this PropertyBuilder<TProperty> builder, bool isTenant = true)
    => (PropertyBuilder<TProperty>)IsTenantProperty((PropertyBuilder)builder, isTenant);

public static IConventionPropertyBuilder? IsTenantProperty(
    this IConventionPropertyBuilder builder,
    bool isTenant = true,
    bool fromDataAnnotation = false)
    => builder.HasAnnotation(TenancyAnnotationNames.IsTenantProperty,
           isTenant ? true : null, fromDataAnnotation) is null
        ? null
        : builder;
```

The method lives in the core package because it only sets metadata. The conventions of all provider packages find the tenant property through the same core function:

Metadata/TenancyEntityTypeExtensions.cs (core package, shortened, simplified)

```
public static IReadOnlyProperty? FindTenantPropertyCandidate(
    this IReadOnlyEntityType entityType)
{
    var marked = entityType.GetProperties()
        .Where(p => p[TenancyAnnotationNames.IsTenantProperty] is true)
```

```
            .ToList();

        if (marked.Count > 1)
            throw new InvalidOperationException("... several tenant properties ...");
        if (marked.Count == 1)
            return marked[0];

        return typeof(ITenantOwned).IsAssignableFrom(entityType.ClrType)
            ? entityType.FindProperty(nameof(ITenantOwned.TenantId))
            : null;
    }
```

| Builder | Typical plugin methods | Note |
|---|---|---|
| `PropertyBuilder` / `<T>` | Column options, roles of properties | Here: `IsTenantProperty` |
| `IndexBuilder` / `<T>` | Index methods, operator classes, filters | Annotation on `Metadata` (IMutableIndex) |
| `KeyBuilder` | Key options (clustering) | As above |
| `ComplexTypePropertyBuilder` | Properties in complex types | Only offer where the option makes sense there (chapter 33) |
| `ModelBuilder` | Model-wide options | Annotation on the model |

## Runtime annotations

The SaveChanges interceptor needs to know at run time which entities are isolated and what their tenant property is called. The convention writes this into an annotation on the entity type; the interceptor reads it from the *runtime model*:

Convention and core package (excerpt)

```
// Convention (design-time model)
entityType.Builder.HasAnnotation(TenancyAnnotationNames.TenantProperty, property.Name);

// Interceptor (runtime model)
public static string? GetTenantPropertyName(this IReadOnlyEntityType entityType)
    => entityType.GetRootType()[TenancyAnnotationNames.TenantProperty] as string;
```

Two questions decide whether this works. Does the annotation reach the runtime model? Yes: `RuntimeModelConvention` only removes EF's own design-time annotations and keeps all others. Does it reach the compiled model? Yes: EF writes simple values automatically (chapter 24). The custom `ICSharpRuntimeAnnotationCodeGenerator` of the PostgreSQL package therefore has the opposite job: it removes the RLS annotations that only migrations need:

Design/Internal/TenancyCSharpRuntimeAnnotationCodeGenerator.cs (shortened, simplified)

```
public class TenancyCSharpRuntimeAnnotationCodeGenerator
    : NpgsqlCSharpRuntimeAnnotationCodeGenerator
{
    // Constructor: (CSharpRuntimeAnnotationCodeGeneratorDependencies,
    //               RelationalCSharpRuntimeAnnotationCodeGeneratorDependencies) : base(...)

    public override void Generate(IEntityType entityType,
        CSharpRuntimeAnnotationCodeGeneratorParameters parameters)
    {
```

```
        if (!parameters.IsRuntime)
        {
            foreach (var key in parameters.Annotations.Keys
                         .Where(k => k.StartsWith(RlsAnnotationNames.Prefix,
                                                  StringComparison.Ordinal))
                         .ToList())
            {
                parameters.Annotations.Remove(key);
            }
        }

        base.Generate(entityType, parameters);
    }
}
```

When the compiled model is generated for NativeAOT (`ForNativeAot`), EF also writes the relational model. The annotation provider has copied the RLS annotations onto the tables, so they appear a second time there. The generator therefore applies the same filter in an override of `Generate( ITable, ...)`.

| Annotation | Design time | Run time | Compiled model |
|---|---|---|---|
| `Acme:Rls:*` (policies) | yes (migrations) | yes, unused | filtered |
| `Acme:Tenancy:TenantProperty` | yes | yes (interceptor) | yes |
| `Acme:Tenancy:IsTenant Property` | yes (convention) | yes, unused | yes (small) |

**Two kinds of runtime annotations**

In this book, “runtime annotation” means an ordinary annotation that is read at run time. EF also has a runtime annotation API in a narrower sense (`AddRuntimeAnnotation`, `GetOrAddRuntimeAnnotation Value` on `IAnnotatable`). It stores values on the finished model only, for example derived data that should not be recomputed on every access. Such values never appear in the snapshot or in migrations. For configuration that a plugin sets while the model is built, the ordinary annotation is the right choice.

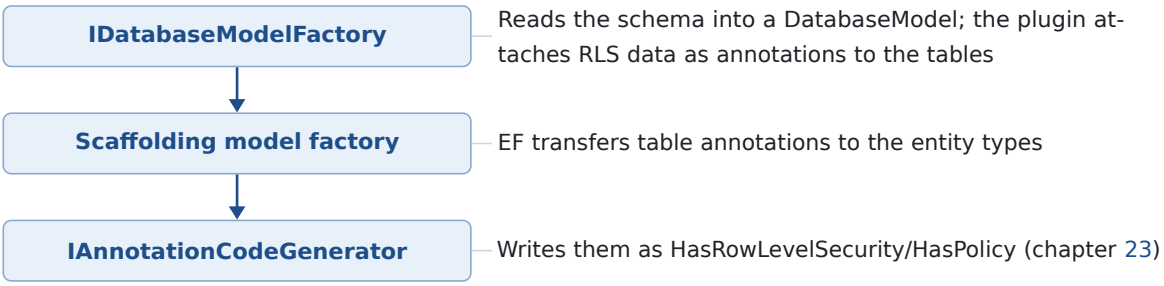
**Tip**

For every annotation it should be clear who reads it and when. Annotations read only at design time do not belong in the compiled model; those read at run time must be in it and need a test against the generated model. The sample’s compiled model test checks both.

# 45 Scaffolding in practice

Version dependency: **volatile**

Chapter 25 names the two design-time services through which a plugin recognizes its schema objects during scaffolding. This chapter shows the implementation. The route has three stations, two of which the plugin already knows:




Scaffolding/TenancyDatabaseModelFactory.cs (shortened)

```
public class TenancyDatabaseModelFactory : NpgsqlDatabaseModelFactory
{
    public TenancyDatabaseModelFactory(
        IDiagnosticsLogger<DbLoggerCategory.Scaffolding> logger)
        : base(logger)
    {
    }

    public override DatabaseModel Create(
        DbConnection dbConnection, DatabaseModelFactoryOptions options)
    {
        var model = base.Create(dbConnection, options);
        // ... open the connection if needed ...
        ReadRowLevelSecurity(dbConnection, model);    // pg_class.relrowsecurity / ...force...
        ReadPolicies(dbConnection, model);            // pg_policies
        return model;
    }

    protected virtual void ReadPolicies(DbConnection connection, DatabaseModel model)
    {
        // SELECT schemaname, tablename, policyname, permissive, roles::text[], cmd,
        //        qual, with_check FROM pg_catalog.pg_policies
        // ...
        var policy = new RlsPolicy(
            Name: reader.GetString(2),
            Command: Enum.Parse<PolicyCommand>(reader.GetString(5), ignoreCase: true),
            Permissive: reader.GetString(3) == "PERMISSIVE",
            Roles: roles is ["public"] ? null : roles,    // "public" = default without TO
            Using: reader.IsDBNull(6) ? null : reader.GetString(6),
            WithCheck: reader.IsDBNull(7) ? null : reader.GetString(7));

        table[RlsAnnotationNames.PolicyPrefix + policy.Name] = policy.Serialize();
    }
}
```

The factory is registered like the code generator in the design-time services with `RemoveAll`, but with `AddScoped`: EF and Npgsql register the database model factory as scoped, and it depends on the scaffolding logger, which is scoped as well. The data format matches that of the Fluent API (a missing `Forced` counts as `false`; the annotation provider treats both alike); snapshot, migrations and code generator therefore work without further changes.

> **Normalized expressions**
>
> PostgreSQL stores policy expressions in normalized form, for example `(("TenantId" = (NULLIF( current_setting('app.tenant_id'::text, true), ''::text))::uuid))`. Scaffolded policies contain exactly this text. That is correct, but looks different from the form generated by the convention. Teams switching to migrations after scaffolding should unify the policies once, otherwise the first migration produces a DROP/CREATE with identical content.

tests/...IntegrationTests/AdvancedTests.cs (excerpt)

```
[Fact]
public void Scaffolding_reads_rls_and_policies()
{
    using var ctx = ContextFor(null);
    var factory = new TenancyDatabaseModelFactory(
        ctx.GetService<IDiagnosticsLogger<DbLoggerCategory.Scaffolding>>());

    var model = factory.Create(
        _fixture.OwnerConnectionString, new DatabaseModelFactoryOptions());
    var orders = model.Tables.Single(t => t.Name == "Orders");

    Assert.Equal(true, orders[RlsAnnotationNames.Enabled]);
    Assert.Equal(true, orders[RlsAnnotationNames.Forced]);
}
```

Part XI

# Multiple providers

Which building blocks belong in a shared core, which in provider packages, how a second provider is built on the same core, and what carries over to a database without SQL.


› Chapter 46: What belongs in the core

› Chapter 47: A second provider: SQL Server

› Chapter 48: Without SQL: what carries over to Cosmos DB

# 46 What belongs in the core

Version dependency: **stable**

A second provider is the acid test for the package architecture. It shows which parts are truly provider-independent, and the first impression is misleading: a building block can know no SQL syntax and no provider type and still need a relational database, because it works with ADO.NET connections or with tables. The sample therefore has three layers, each in a package of its own:

| Building block | Layer | Reason |
|---|---|---|
| `TenantId`, converter, JSON reader/writer | Core | Pure .NET types and EF value converters |
| `ITenantOwned`, `ITenantProvider`, attributes | Core | For entity assemblies without a provider dependency |
| `IsTenantProperty()`, candidate lookup | Core | Metadata only |
| Runtime annotation, `GetTenantPropertyName` | Core | Read by the provider-neutral interceptors |
| SaveChanges interceptor | Core | Works only with the change tracker |
| Protection of bulk updates (query expression interceptor) | Core | Only inspects the LINQ expression tree |
| Evaluatable filter with attribute | Core | Recognizes function classes of all packages |
| Check while loading (materialization interceptor) | Core | Only reads the model, the context and the row's values |
| Value generation, logging, `AddAcmeTenancy` | Core | No database involved |
| Query filter with the tenant as a parameter, test mode | Core | Need no session; used by Cosmos DB and by application tests |
| `TenantSessionInterceptor` (abstract) | Relational | Works on the `DbConnection`; only the SQL command differs per provider |
| `TenantTableMappingValidator` | Relational | Tables and columns: TPT, TPC, entity splitting |
| `Database.ApplyTenantSession()` | Relational | Uses the session interceptor of the provider |
| `TenantActivityEnrichmentInterceptor` | Relational | Command interceptor; needs only `DbCommand`, no SQL dialect |
| Type mapping, translators, migrations, RLS conventions | Provider | Store types, SQL syntax, schema objects |
| Custom SQL nodes, aggregates | Provider | Depend on syntax and provider factory |

The rule of thumb therefore has three stages: **whatever needs no database belongs in the core; whatever needs a relational database but no SQL dialect belongs in the relational layer; SQL syntax and store types belong in the provider.** The core package references only `Microsoft.EntityFrameworkCore`, the package `Acme.EntityFrameworkCore.Tenancy.Relational` adds `Microsoft.EntityFrameworkCore.Relational`, and the two relational provider packages build on it. `PackageLayerTests` makes sure that no relational type creeps back into the core. That this pays off shows in chapter 48: the Cosmos package builds on the core alone.

**Separate the relational layer from the start**

If you might support a non-relational provider such as Cosmos DB later, put the relational layer into a package of its own from the beginning. Moving types into another assembly afterwards breaks binary compatibility, and entity assemblies that only referenced the core lose the transitive dependency on `Microsoft.EntityFrameworkCore.Relational` they may have relied on for `ToTable` or `HasColumnType`. The sample made exactly this move in edition 1.2.

**Name collisions between provider packages**

Two provider packages that both define an extension method `UseAcmeTenancy` on `DbContextOptionsBuilder` cannot be referenced at the same time: the call would be ambiguous. The PostgreSQL package uses EF's builder, the SQL Server and Cosmos packages the provider's builder (`UseSqlServer(cs, x => x.UseAcmeTenancy())`). If you plan several provider packages, use the provider builder consistently from the start, as the NetTopologySuite plugin does.

# 47 A second provider: SQL Server

Version dependency: **medium**

The package `Acme.EntityFrameworkCore.Tenancy.SqlServer` implements the same idea with SQL Server's means. The differences show what a plugin really has to solve anew per database:

| Aspect | PostgreSQL | SQL Server |
|---|---|---|
| Tenant in the session | `set_config(name, value, false)` | `sp_set_session_context` |
| `CurrentTenant()` in SQL | `nullif(current_setting(...), '')::uuid` | `CAST(SESSION_CONTEXT(N'…') AS uniqueidentifier)` |
| Column type | `uuid` | `uniqueidentifier` |
| RLS mechanism | `CREATE POLICY` per table | Predicate function + `CREATE SECURITY POLICY` |
| Read / write | `USING` / `WITH CHECK` | `FILTER` / `BLOCK ... AFTER INSERT/UPDATE` |
| Table owner | Only bound with `FORCE` | No FORCE; predicates are part of the query |
| `DROP TABLE`, renaming | Policies disappear with it or follow | Policy must go first and be recreated afterwards (SCHEMABINDING) |
| DDL in batches | Transactional, freely combinable | `CREATE FUNCTION/SCHEMA` alone in a batch: `EXEC(N'...')` |
| Table mappings | Same contract from the relational package (chapter 10, page 52) | |
| Temporal tables | No system-versioned tables | Rejected for tenant-owned types: the policy does not cover the history table |
| Deliberate exception | Attribute or `HasNoRowLevelSecurity()` | Attribute only |

## Activation through the provider builder

```
services.AddAcmeTenancy<HttpTenantProvider>();                    // core package
services.AddDbContext<ShopContext>((sp, o) => o
    .UseSqlServer(connectionString, sql => sql.UseAcmeTenancy(sp)));

// Overload without IServiceProvider; the one taking sp (above) adds the interceptors
public static SqlServerDbContextOptionsBuilder UseAcmeTenancy(
    this SqlServerDbContextOptionsBuilder sqlServerOptionsBuilder,
    string sessionKey = "TenantId",
    string policySchema = "acme")
{
    // From the provider builder to the actual DbContextOptionsBuilder
    var optionsBuilder =
        ((IRelationalDbContextOptionsBuilderInfrastructure)sqlServerOptionsBuilder)
        .OptionsBuilder;

    var extension = optionsBuilder.Options.FindExtension<SqlServerTenancyOptionsExtension>()
                    ?? new SqlServerTenancyOptionsExtension();
    // ... validate and set settings ...
```

```
    ((IDbContextOptionsBuilderInfrastructure)optionsBuilder)
        .AddOrUpdateExtension(extension.WithSettings(settings));

    return sqlServerOptionsBuilder;
}
```

## Migrations: security policy instead of policy expression

For SQL Server, the convention only records the tenant column; the SQL is produced in the generator. The predicate function is the same for all tables and is created idempotently:

Migrations/Internal/SqlServerTenancyMigrationsSqlGenerator.cs (excerpt, simplified)

```
protected virtual void CreatePolicy(string table, string? schema, string column,
    MigrationCommandListBuilder builder)
{
    var function = Delimit(PredicateFunctionName, Settings.PolicySchema);
    var target = Delimit(table, schema);
    var predicate = $"{function}({Delimit(column)})";

    // CREATE SCHEMA/FUNCTION must be alone in a batch -> EXEC(N'...')
    Statement(builder, $"IF SCHEMA_ID({Literal(Settings.PolicySchema)}) IS NULL "
        + $"EXEC({Literal($"CREATE SCHEMA {Delimit(Settings.PolicySchema)}")})");

    var functionSql =
        $"CREATE FUNCTION {function}(@TenantId uniqueidentifier) "
        + "RETURNS TABLE WITH SCHEMABINDING AS RETURN SELECT 1 AS [Allowed] "
        + $"WHERE @TenantId = CAST(SESSION_CONTEXT({Literal(Settings.SessionKey)}) "
        + "AS uniqueidentifier)";
    Statement(builder, $"IF OBJECT_ID({Literal(function)}, N'IF') IS NULL "
        + $"EXEC({Literal(functionSql)})");
    // ... check an existing function: if it reads another key, THROW 50000

    Statement(builder,
        $"CREATE SECURITY POLICY {PolicyName(table, schema ?? "dbo")} "
        + $"ADD FILTER PREDICATE {predicate} ON {target}, "
        + $"ADD BLOCK PREDICATE {predicate} ON {target} AFTER INSERT, "
        + $"ADD BLOCK PREDICATE {predicate} ON {target} AFTER UPDATE "
        + "WITH (STATE = ON)");
}

protected override void Generate(DropTableOperation operation, IModel? model,
    MigrationCommandListBuilder builder, bool terminate = true)
{
    // SCHEMABINDING: the policy first, found by the table it binds
    DropPolicy(operation.Name, TableSchema(operation.Schema, model), builder);
    base.Generate(operation, model, builder, terminate);
}
```

`SCHEMABINDING` only accepts two-part names. A table without an explicit schema is therefore qualified by the callers with the model's default schema or, failing that, with `dbo`; otherwise SQL Server rejects the security policy.

> **Nested SQL**
>
> The `EXEC(N'...')` contains SQL in a string literal that in turn contains literals. The nesting is produced

> exclusively through the string mapping's `GenerateSqlLiteral`, which doubles quotes correctly; identifiers go through `DelimitIdentifier`. Session key and schema are additionally validated during configuration.

## Schema changes on protected tables

`SCHEMABINDING` protects the predicate function, but it also binds the table. SQL Server rejects `sp_rename` for the table and the tenant column, `ALTER SCHEMA ... TRANSFER`, `ALTER COLUMN` and `DROP COLUMN` as long as a security policy binds the table: "object participates in enforced dependencies". This cannot be solved per operation. EF orders a migration's operations itself, and when the tenant column is renamed, the `RenameColumnOperation` comes *before* the `AlterTableOperation` that carries the changed annotation. A plain table rename produces no `AlterTableOperation` at all.

The generator therefore overrides the entry point that receives the whole operation list. It finds all tables whose bound objects change, follows their renames and wraps the list. At the front it removes the policies of the affected tables under their old names. At the end it recreates them with the final table, schema and column names from the target model:

Generated SQL when renaming the tenant column (shortened)

```
DECLARE @policy nvarchar(400) = (SELECT TOP (1) ... FROM sys.security_policies AS p
    WHERE EXISTS (SELECT 1 FROM sys.security_predicates AS s WHERE s.object_id = p.object_id
        AND s.target_object_id = OBJECT_ID(N'[dbo].[Invoice]')
        AND s.predicate_definition LIKE N'%[[]fn[_]tenant[_]predicate]%'));
IF @policy IS NOT NULL EXEC(N'DROP SECURITY POLICY ' + @policy);
EXEC sp_rename N'[Invoice].[TenantId]', N'Mandant', 'COLUMN';
EXEC sp_rename N'[Invoice].[IX_Invoice_TenantId]', N'IX_Invoice_Mandant', 'INDEX';
IF SCHEMA_ID(N'acme') IS NULL EXEC(N'CREATE SCHEMA [acme]');
IF OBJECT_ID(N'[acme].[fn_tenant_predicate]', N'IF') IS NULL EXEC(N'CREATE FUNCTION ...');
CREATE SECURITY POLICY [acme].[tenant_dbo_Invoice]
    ADD FILTER PREDICATE [acme].[fn_tenant_predicate]([Mandant]) ON [dbo].[Invoice], ...
```

Three decisions are built into this:

- **A transaction instead of switching off.** EF runs a migration's commands in a transaction, and DDL is transactional in SQL Server. If a step fails, SQL Server also rolls back the removal of the policy. A permanently removed policy, or one switched off with `STATE = OFF`, cannot come about this way. The Down path is the same wrapping with the names swapped.
- **Removal by binding, not by name.** Only a policy whose predicate uses the plugin's predicate function and that binds exactly this table is dropped. Policies of others and tables without RLS stay untouched, and policies under an earlier name are found all the same.
- **Schema-qualified names.** Up to 1.0 the policy was called `tenant_Table`, so tables with the same name in two schemas collided. New policies are called `tenant_Schema_Table`. Existing installations keep the old name until the table is touched the next time; then the generator replaces the policy. The remaining ambiguity (schema `a_b` with table `c` versus schema `a` with table `b_c`) and names longer than 128 characters are rejected by the convention while the model is built.

Limits: restoring needs the target model. That is the normal case, since `dotnet ef migrations add` creates a designer file for every migration. If it is missing, or a changed table does not appear in it, the generator stops instead of dropping a policy. The application's own schema-bound objects (views with `SCHEMABINDING`, its own security policies) are not handled by the plugin; they still block

the same operations. Data operations that EF sorts between the schema changes run inside the transaction without a policy, that is with the rights of the migration role. `SqlServerMigrationTests` checks with diffs generated by EF against a real SQL Server: create, rename column and table, move schema, every step back and drop, with read and write isolation through the app login after every step. In addition: tables with the same name in two schemas, a policy under the old name and someone else's policy on a table without RLS.

## Changing the session key and the policy schema

`sessionKey` and `policySchema` are options, but they act in two places: at run time in the session interceptor, which sets the key, and in the database in the predicate function, which reads it. Up to 1.0 the generator read both values from the *current* options and created the function only if it was missing. Switching from `TenantId` to `NewTenant` therefore produced no migration. The interceptor then set the new key, the function kept reading the old one, and no tenant saw any rows. Worse still: run again, for example for a new test database, an old migration would have created objects with the new values.

Since 1.1 the package separates the two roles. The convention writes key and schema into the model as the model annotations `Acme:SqlServerRls:SessionKey` and `Acme:SqlServerRls:PolicySchema`, and so into the snapshot. The annotation provider passes them on to the relational model, and the differ produces an `AlterDatabaseOperation` with the old and the new value when they change. The generator reads both values from the target model of the migration it is generating; the options only apply to models that do not record the values. That way every migration stays with the state it was generated from.

| Case | Behavior of the migration |
|---|---|
| Unchanged | No operation, no phantom change |
| Key or schema changed, old value in the snapshot | In one transaction: remove the policies of all protected tables, drop the old function, create the function with the new values, recreate the policies. The Down path does the same in reverse. |
| Key or schema changed, tables dropped in the same migration | The policies of the dropped tables are removed before the old function as well, since they still reference it until `DROP TABLE`; only the tables of the target model get a policy again. On the Down path the restored table gets its policy with `CREATE TABLE`, already on the function with the old values. |
| First migration of a context, or the first after the upgrade from 1.0 (old value unknown) | Nothing is dropped. An existing function must read the key that the model names, otherwise the migration stops with `THROW 50000`. |
| Down to a snapshot before 1.1 | Function and policies stay as they are. |

The third row is a deliberate limit. Without a recorded old value there is no proof that an existing function belongs to the plugin and not to another context. An existing installation therefore changes its key in two steps: first the first migration after the upgrade with the key unchanged, then the change in a migration of its own.

The same check protects several contexts in one database. There is one function per policy schema, and every policy of the plugin checks the function's definition after creating it. If a second context configures another key in the same schema, its migration fails instead of binding its tables to a function that reads the wrong key. Contexts with different keys need different policy schemas.

Run time and database can still drift apart briefly. If a new app version with a new key runs before the migration has been applied, the interceptor sets a key that the function does not read. The result is not a data leak but *fail closed*: no rows, and writes fail on the BLOCK predicate. The migration therefore belongs before the new app version. `SqlServerSessionSettingsTests` checks without a database the annotations, the snapshot, the empty diff, the detected change and the historical migration. Against a real SQL Server it checks changing key and schema with Up and Down, the upgrade of a 1.0 installation and the second context.

## Run time: same flow, different command

Runtime/SqlServerTenantSessionInterceptor.cs

```
public class SqlServerTenantSessionInterceptor : TenantSessionInterceptor
{
    public SqlServerTenantSessionInterceptor(ITenantProvider tenants)
        : base(tenants)
    {
    }

    protected override DbCommand CreateSessionCommand(
        DbConnection connection, DbContext? context, TenantId? tenant)
    {
        var key = context?.GetService<ISqlServerTenancySingletonOptions>()
                      .Settings.SessionKey
                  ?? SqlServerTenancySettings.Default.SessionKey;

        var command = connection.CreateCommand();
        command.CommandText = "EXEC sp_set_session_context @key = @key, @value = @value";
        AddParameter(command, "@key", key);
        AddParameter(command, "@value", tenant?.Value);    // NULL: no rows visible
        return command;
    }
}
```

## Deliberately not ported

The SQL Server package is a self-contained but leaner provider. Fluent API policies, the optional query filter of the convention (`ApplyQueryFilter`; the core's filter works here too), the snapshot code generator and scaffolding are missing there, and the custom SQL node and the aggregate of the functions sample package exist for PostgreSQL only. That is a product decision, not a technical obstacle: each of these building blocks can be transferred to SQL Server with the same patterns. The list belongs in the package's documentation so users know what to expect.

tests/...SqlServer.Tests/SqlServerTests.cs (excerpt)

```
[Fact]
public async Task Block_predicate_rejects_rows_of_other_tenant()
{
    await using var db = ContextFor(TenantId.New());

    var ex = await Assert.ThrowsAsync<SqlException>(() => db.Database.ExecuteSqlAsync(
        $"INSERT INTO [Orders] ([TenantId], [Total]) VALUES ({Guid.NewGuid()}, 1)"));

    Assert.Equal(33504, ex.Number);      // violation of a BLOCK PREDICATE
```

```
}
```

As with PostgreSQL, the tests run on two levels: without a database for model, query and migration SQL, and with Testcontainers against a real SQL Server. There, the application connects with its own login instead of `sa` to reproduce the production situation.

# 48 Without SQL: what carries over to Cosmos DB

Version dependency: **medium**

PostgreSQL and SQL Server share the same idea: the database itself refuses foreign rows, and the plugin only has to tell it the tenant. Azure Cosmos DB has neither SQL DDL nor a session nor row level security. The package `Acme.EntityFrameworkCore.Tenancy.Cosmos` is therefore deliberately a lean contrast example, not a full provider: it shows which parts of the plugin carry over without SQL, and what is lost. It references only the core package from chapter 46, without the relational layer.

| Building block | In the Cosmos package | Note |
|---|---|---|
| `TenantId`, `ITenantOwned`, `AddAcmeTenancy` | Reused | Unchanged from the core |
| SaveChanges interceptor | Reused | Sets and checks the tenant before saving |
| Materialization check | Reused | The only check behind the query filter |
| Value generation for `TenantId` keys | Reused | Convention from the core |
| Type mapping | New | `TenantId` as a JSON string |
| Partition key convention | New | Replaces the RLS convention |
| Query filter | From the core | Tenant as a query parameter instead of a session value |
| Session interceptor, migrations | Not ported | There is no session and no schema |

## The tenant as partition key

In Cosmos DB, the partition key decides where a document lives and which partition a query reads. The tenant is the natural candidate: every query of the application concerns exactly one tenant. A convention sets it for every tenant-owned root entity type, with the same candidate lookup as the RLS conventions. It runs with convention source, so an explicit `HasPartitionKey` of the user wins:

`Conventions/CosmosTenantPartitionKeyConvention.cs (excerpt)`

```
foreach (var entityType in modelBuilder.Metadata.GetEntityTypes())
{
    // Only on the root of a hierarchy; owned types live in their owner's document
    if (entityType.BaseType is not null || entityType.IsOwned())
        continue;

    // ITenantOwned without a mapped TenantId: exception, as in the relational packages
    if (entityType.FindTenantPropertyCandidate() is not { } candidate)
        continue;      // (shortened)

    if (entityType.ClrType.IsDefined(typeof(NoTenantIsolationAttribute), inherit: true))
        continue;

    entityType.Builder.HasAnnotation(TenancyAnnotationNames.TenantProperty, candidate.Name);
```

```
    string[] partitionKey = [candidate.Name];
    if (entityType.Builder.CanSetPartitionKey(partitionKey))
        entityType.Builder.HasPartitionKey(partitionKey);
}
```

Two details differ from the relational providers. Cosmos accepts only strings, numbers and booleans as partition key values; the core's converter to `Guid` is not enough, so the package maps `TenantId` to a string through an `ITypeMappingSourcePlugin`. Cosmos has no plugin interface of its own for this, but its type mapping source asks the core plugins first; the mapping itself builds on `CosmosTypeMapping` from an `.Internal` namespace. And a container has exactly one partition key: data without a tenant, such as shared master data, needs a container of its own (`ToContainer`).

## The query filter on the current tenant

The relational filter compares with `EF.Functions.CurrentTenant()`, which becomes `current_setting()` in SQL: the database knows the tenant from the session. Cosmos knows nothing; the tenant has to travel with every query. A convention cannot do that, because it has no access to the context. The filter is therefore added in `OnModelCreating`, with the context itself. The extension needs neither SQL nor Cosmos and lives in the core package; the test mode for applications (chapter 26) uses it as well:

Usage and Metadata/TenancyModelBuilderExtensions.cs (excerpt)

```
// OnModelCreating: modelBuilder.HasTenantQueryFilters(this);

var currentTenant = Expression.Call(
    typeof(TenancyModelBuilderExtensions).GetMethod(nameof(CurrentTenant))!,
    Expression.Constant(context));

// e => (TenantId?)EF.Property<TenantId>(e, "TenantId") == CurrentTenant(context)
var parameter = Expression.Parameter(entityType.ClrType, "e");
var body = Expression.Equal(
    Expression.Convert(
        Expression.Call(typeof(EF), nameof(EF.Property), [typeof(TenantId)],
            parameter, Expression.Constant(property.Name)),
        typeof(TenantId?)),
    currentTenant);

modelBuilder.Entity(entityType.ClrType)
    .HasQueryFilter(QueryFilterKey, Expression.Lambda(body, parameter));
```

EF treats a context instance in a query filter specially: it replaces it with the context that runs the query and evaluates `CurrentTenant(context)` before every execution. The tenant thus becomes a query parameter, and the compiled query stays in the cache for all tenants. Because the filter compares the partition key with that parameter, EF Core lifts it into the request: the query goes to a single partition instead of fanning out to all of them. A test makes this visible through the event data:

tests/...Cosmos.Tests/EmulatorTests.cs (excerpt)

```
await using var ctx = _fixture.CreateContext(
    new MutableTenantProvider { CurrentTenantId = tenant },
    configure: o => o.LogTo(
        (id, _) => id == CosmosEventId.ExecutingSqlQuery,
```

```
        e => partitions.Add(((CosmosQueryEventData)e).PartitionKeyValue.ToString())));

await ctx.Orders.Where(o => o.Total > 0).ToListAsync();

// EF lifts the filter's equality on the partition key into the request
var partition = Assert.Single(partitions);
Assert.Contains(tenant.Value.ToString("D"), partition);
```

**No row level security: the application is the only line of defense**

Cosmos DB has no row level security. The isolation is exactly as strong as the application layer: the query filter, the SaveChanges interceptor and, if enabled, the materialization check (`UseAcmeTenancy(sp, verifyMaterializedTenants: true)`). Everything that bypasses EF bypasses the isolation: queries through the Cosmos SDK, scripts and tools with the account key, and `IgnoreQueryFilters`. Raw queries with `FromSql` keep the filter, but run across all partitions. The threat model of chapter 36 (page 143) therefore applies without its database-side half. Where tenants must be separated by the database, the architecture has to do it: a **container per tenant** with access by role-based access control, or an **account per tenant**. Both cost more and give up the shared container, but a leak then needs a wrong credential, not just a wrong query.

## Comparison of the three providers

The table is also the usage contract of the three packages. For configurations that decide the isolation it names one of four states and the evidence: *supported and tested* (a test in the repository checks the behavior), *rejected while the model is built* (an `InvalidOperationException` on the first access to `Model`), *only under conditions* (works if the named condition holds), and *not verified* (no test; do not rely on it).

| Aspect | PostgreSQL | SQL Server | Cosmos DB |
|---|---|---|---|
| Who enforces the isolation | Database (RLS policies) | Database (security policy) | Application only (query filter) |
| Tenant in the request | Session setting | `SESSION_CONTEXT` | Query parameter, partition key |
| Schema objects | Policies via migrations | Predicate function, security policy | None |
| Raw queries | Still filtered by RLS | Still filtered by the policy | `FromSql` filtered but cross-partition; SDK unfiltered |
| Packages used | Core, relational, provider | Core, relational, provider | Core, provider |
| Internal API (EF1001) | 10 types | 2 types | 1 type (`CosmosTypeMapping`) |
| Tests | Testcontainers PostgreSQL | Testcontainers SQL Server | Linux emulator (preview) |
| Protected mappings (TPH, owned types in the table or as JSON, table splitting, join entity) | Supported and tested, also in the database (`TableMappingTests`, `MappingIsolationTests`) | Supported; tested on model and DDL (`SqlServerTableMappingTests`), in the database only one table per entity type | Not verified: hierarchies and owned types have no test |

| Aspect | PostgreSQL | SQL Server | Cosmos DB |
|---|---|---|---|
| TPT, TPC, entity splitting, owned type in its own table, implicit join table | Rejected while the model is built | Rejected while the model is built | Not applicable (no tables) |
| Temporal table | Not applicable | Rejected while the model is built (`SqlServerTemporalTableTests`) | Not applicable |
| `ITenantOwned` without a mapped tenant property | Rejected while the model is built (`TenantPropertyTests`) | Rejected while the model is built (`SqlServerTenantPropertyValidationTests`) | Rejected while the model is built (`ModelTests`) |
| Deliberate exception | Attribute or `HasNoRowLevelSecurity()`, logged as a warning; tested | Attribute; tested | Attribute; tested |
| Transactions | `BeginTransaction`, `SaveChanges` and `TransactionScope` supported and tested; `UseTransaction` only under conditions; transaction pooling: page 144 | As PostgreSQL, without transaction pooling | Not applicable (no session) |

## Deliberately not ported

Migrations and scaffolding (Cosmos has no schema), the session interceptor and `Database.ApplyTenantSession()` (there is no session), and the protection of bulk updates (EF Core 10 has no `ExecuteUpdate` for Cosmos). Cosmos also only supports asynchronous I/O: `SaveChanges()` and `ToList()` throw, so the application and its tests use the async variants throughout.

> **Testing against the emulator**
>
> The tests run against the Linux emulator of Cosmos DB (vNext), a preview. The image tag is pinned, the connection uses gateway mode over the container's HTTP client, and the test project references a newer Cosmos SDK than EF Core 10.0.0 brings: the SDK 3.51 cannot read every binary response of the current emulator. The project has two groups. The model tests (`ModelTests`: partition key, query filter, validation) need neither the emulator nor Docker and run in the build job of the CI with the filter `Category!=Emulator`; if one fails, the build fails, in the compatibility run as well. Only the tests in `EmulatorTests` carry the trait `Category=Emulator`. Because the preview emulator occasionally hangs, they run in a job of their own with a time limit that does not block the build yet. A new test without the trait lands in the required group, so no test drops out of both.

Part XII

# Appendices

For reference: extension points with lifetimes, files of the sample, terms and a debugging toolbox.


› Appendix A: Extension points and lifetimes

› Appendix B: Files of the sample plugin

› Appendix C: Glossary

› Appendix D: Debugging toolbox

› Appendix E: Exercises with solutions

› Appendix F: Sources

› License

# A Extension points and lifetimes

This appendix leads from a mechanism to its properties and to the chapter that covers it. The opposite direction, from the problem to the mechanism, is taken by the decision guide in chapter 4. Lifetime and cardinality come from EF Core 10's service tables (`EntityFrameworkServicesBuilder`, `EntityFrameworkRelationalServicesBuilder`, `EntityFrameworkRelationalDesignServicesBuilder`), the statements about provider implementations from Npgsql 10; `ServiceRegistrationTests` checks the sample's registrations against the same tables.

| Marker | Meaning |
|---|---|
| Singleton, scoped | Once per internal provider or once per context instance, with pooling across several requests (chapter 8); "DT": in the service provider of `dotnet ef` |
| multiple, single | Plugin interface, EF asks all registrations; or single service with exactly one winner |
| public, provider base, pubternal | As in the decision guide: public API only; replacing means inheriting from the provider's implementation; even EF's own type is internal |
| `TryAdd`, `Replace`, `ReplaceService` | Added or replaced in the extension's `ApplyServices`; replaced by the application on the options builder |
| † | Elevated upgrade risk: replacing or deriving touches internal API of EF or Npgsql (EF1001, upgrade playbook in chapter 27) |
| ● ◐ ○ | Preferred mechanism; use with care; last resort |
| **exclusive** | Replacing rules out every other plugin that replaces the same service (compatibility matrix in chapter 6) |

| Service and task | Lifetime, cardinality | API | Registration | Rating | Ch. |
|---|---|---|---|---|---|
| **Configuration and services** | | | | | |
| `ISingletonOptions` Configuration for singleton services | Singleton, multiple | public | `TryAdd`, twice | ● combinable | 6, 8 |
| `IResettableService` Reset scoped state under pooling | Scoped, multiple | public | register additionally | ● combinable | 8 |
| **Model** | | | | | |
| `IConventionSetPlugin` Add conventions and validation | Scoped, multiple | public | `TryAdd` | ● combinable, order | 10, 11 |
| `IModelCacheKeyFactory` Model per runtime context | Singleton, single | public | `ReplaceService` | ◐ **exclusive** | 34 |
| `IModelCustomizer` Model building before `OnModelCreating` | Singleton, single | public | `Replace` | ○ **exclusive**; prefer conventions | 4 |
| `IModelValidator` Validation of the finished model | Singleton, single | provider base† | `Replace` | ○ **exclusive**; prefer a finalized convention | 11 |

| Service and task | Lifetime, cardinality | API | Registration | Rating | Ch. |
|---|---|---|---|---|---|
| **Types and values** | | | | | |
| `IRelationalTypeMappingSourcePlugin`<br>Custom CLR types as columns; core: `ITypeMappingSourcePlugin` | Singleton, multiple | public | `TryAdd` | ● combinable | 12 |
| `IRelationalTypeMappingSource`<br>All type mappings of the provider | Singleton, single | provider base† | `Replace` | ○ **exclusive** | 12 |
| `IValueGeneratorSelector`<br>Chooses value generators | Scoped, single | provider base† | `Replace` | ○ **exclusive**; prefer a factory set by a convention | 31 |
| **Queries** | | | | | |
| `IEvaluatableExpressionFilterPlugin`<br>Protect functions from the funcletizer | Singleton, multiple | public | `TryAdd` | ● combinable | 15 |
| `IMethodCallTranslatorPlugin`, `IMemberTranslatorPlugin`<br>Methods and members to SQL | Scoped, multiple | public | `TryAdd` | ● combinable; same method: order | 14 |
| `IAggregateMethodCallTranslatorPlugin`<br>Aggregate functions | Scoped, multiple | public | `TryAdd` | ● combinable | 43 |
| `ISqlExpressionFactory`<br>Builds SQL expressions; inject into translators | Scoped, single | public | inject | ● use only | 14 |
| `IMethodCallTranslatorProvider` and similar<br>Translator infrastructure, also for members and aggregates | Scoped, single | provider base† | `Replace` | ○ **exclusive**; prefer plugins | 16 |
| `IQueryTranslationPreprocessorFactory`<br>Rewrite the LINQ tree before translation | Scoped, single | public | `Replace` | ◐ **exclusive**; prefer a query interceptor | 13 |
| `IQueryTranslationPostprocessorFactory`<br>Post-process the translated tree | Scoped, single | provider base† | `Replace` | ○ **exclusive** | 13, 16 |
| `IQuerySqlGeneratorFactory`<br>SQL text, write custom nodes | Singleton, single | provider base† | `Replace` | ○ **exclusive** | 42, 6 |

| Service and task | Lifetime, cardinality | API | Registration | Rating | Ch. |
|---|---|---|---|---|---|
| `IRelationalParameterBasedSqlProcessorFactory`<br>NULL semantics with parameter values | Scoped, single | provider base† | `Replace` | ○ **exclusive** | 42, 6 |
| **Runtime** | | | | | |
| `IInterceptor`: SaveChanges, Connection, Command, Transaction, IdentityResolution<br>Hooks into saving and execution | Scoped, multiple | public | `AddInterceptors` or injected | ● combinable, order | 18 |
| `IMaterializationInterceptor` (an `ISingletonInterceptor`)<br>Materialization | Singleton, multiple | public | `TryAdd` as `ISingletonInterceptor` or one shared instance | ● combinable | 17 |
| `IQueryExpressionInterceptor` (also an `ISingletonInterceptor`)<br>LINQ tree before compilation | Singleton by interface, multiple | public | one shared instance or `TryAdd` as `IInterceptor`; registered as `ISingletonInterceptor` EF never calls it | ● combinable | 16, 30 |
| `IModificationCommandFactory`<br>Update pipeline: one command per row | Singleton, single | provider base† | `Replace` | ○ **exclusive** | 32 |
| `ICommandBatchPreparer`, `IModificationCommandBatchFactory`<br>Update pipeline: order and batches | Scoped, single | pubternal† or provider base† | `Replace` | ○ **exclusive** | 32 |
| `IUpdateSqlGenerator`<br>Write INSERT, UPDATE, DELETE | Singleton, single | provider base† | `Replace` | ○ **exclusive** | 32 |
| **Migrations** | | | | | |
| `IRelationalAnnotationProvider`<br>Carry annotations to tables and columns | Singleton, single | provider base† | `Replace` | ◐ **exclusive**; declarative path | 21 |
| `IMigrationsSqlGenerator`<br>SQL for migration operations | Scoped, single | provider base† | decorator or `Replace` | ◐ **exclusive** when replaced, decorator combinable | 21, 6 |

| Service and task | Lifetime, cardinality | API | Registration | Rating | Ch. |
|---|---|---|---|---|---|
| `IMigrationsModelDiffer`<br>Compares relational models | Scoped, single | pubternal† | `Replace` | ○ **exclusive**; prefer annotations | 22 |
| `ISqlGenerationHelper`<br>Quote identifiers, terminate statements | Singleton, single | public | inject | ● use only | 10, 21 |
| **Design time (through `IDesignTimeServices`)** | | | | | |
| `IAnnotationCodeGenerator`<br>Annotations as Fluent API in snapshot and scaffolding | Singleton DT, single | provider base† | `RemoveAll` + `Add` | ◐ **exclusive** | 23 |
| `ICSharpRuntimeAnnotationCodeGenerator`<br>Filter annotations in the compiled model | Singleton DT, single | pubternal† | `RemoveAll` + `Add` | ◐ **exclusive** | 44, 24 |
| `ICSharpMigrationOperationGenerator`<br>Custom operations as C# in migrations | Singleton DT, single | public | `RemoveAll` + `Add` | ○ **exclusive** | 22 |
| `IDatabaseModelFactory`<br>Read the schema for scaffolding | Scoped DT, single | provider base† | `RemoveAll` + `AddScoped` | ◐ **exclusive** | 45 |
| `IProviderConfigurationCodeGenerator`<br>Use call in the scaffolded context | Singleton DT, single | provider base† | `RemoveAll` + `Add` | ○ **exclusive** | 25 |

## Extension points without a service of their own

Many mechanisms act through the model, the options or migrations and have no lifetime in the internal provider. They are almost always the first choice:

| Extension point | Task | Activation | Ch. |
|---|---|---|---|
| `IDbContextOptionsExtension` | Configuration, services, provider key | Use method | 5, 7 |
| Annotations, Fluent API | Configuration on the model | Extension methods | 9, 44 |
| Runtime annotation, runtime annotation API | Information for runtime code | Convention or `GetOrAddRuntimeAnnotationValue` | 44 |
| `ValueConverter`, `JsonValueReaderWriter` | Convert values | Type mapping or `HasConversion` | 12 |
| `ValueGenerator` with factory | Values for new entities | Convention via `HasValueGeneratorFactory` | 31 |

| Extension point | Task | Activation | Ch. |
|---|---|---|---|
| Custom `SqlExpression` | Syntax that is not a function call | Translator plus generator and processor | 42 |
| Custom `MigrationOperation` | Steps outside the model | `MigrationBuilder` extension | 22 |
| `IDesignTimeServices` | Services for `dotnet ef` | `DesignTime ServicesReference` | 23 |

# B Files of the sample plugin

| File | Content | Ch. |
|---|---|---|
| **Core package** | | |
| `TenantId.cs`, `TenantIdConverter.cs` | Strongly typed ID, converter | 12 |
| `TenantIdValueComparer.cs` | Shared value comparers, NativeAOT-ready | 24 |
| `TenantIdJsonReaderWriter.cs`, `TenantIdJsonConverter.cs` | JSON columns in EF; JSON in web APIs | 12, 33 |
| `ITenantOwned.cs`, `ITenantProvider.cs` | Marker, tenant source | 3 |
| `TenancyException.cs` | Exception for isolation violations | 18 |
| `NoTenantIsolationAttribute.cs` | Data annotation | 10 |
| `TenancyDbFunctionsExtensions.cs`, `TenancyDbFunctionsAttribute.cs` | `EF.Functions.CurrentTenant()`, attribute for the client check | 14, 15 |
| `Diagnostics/*` | Event IDs, definitions, logger extensions | 19 |
| `Interceptors/TenantSaveChangesInterceptor.cs` | SaveChanges logic | 18 |
| `Interceptors/TenantMaterializationInterceptor.cs` | Check while loading (optional) | 17 |
| `Interceptors/TenantQueryExpressionInterceptor.cs` | Rejects `ExecuteUpdate` that sets the tenant (optional) | 30 |
| `Metadata/*` (core) | Runtime annotation, IsTenantProperty, candidate lookup | 44 |
| `Query/TenancyEvaluatableExpressionFilterPlugin.cs` | Filter for all packages | 15, 46 |
| `DependencyInjection/*` | AddAcmeTenancy | 7, 46 |
| `ValueGeneration/TenantIdValueGenerator.cs`, `…Factory.cs` | Value generation | 31 |
| `Metadata/TenancyModelBuilderExtensions.cs` | Query filter with the tenant as a parameter | 48, 26 |
| `Testing/*` | Test mode for applications (SQLite, InMemory) | 26 |
| `Conventions/TenantIdKeyValueGenerationConvention.cs` | Generator for keys | 31 |
| **Relational package** | | |
| `Interceptors/TenantSessionInterceptor.cs` | Abstract base for session commands | 18, 46 |
| `Interceptors/TenantActivityEnrichmentInterceptor.cs` | Tenant as a tag on the `Activity` (optional) | 19 |
| `Metadata/TenantTableMappingValidator.cs` | Supported table mappings (TPT, TPC, entity splitting) | 10, 46 |
| `TenancyDatabaseFacadeExtensions.cs` | `Database.ApplyTenantSession()` | 18 |
| **PostgreSQL package** | | |
| `Infrastructure/TenancyOptionsExtension.cs` | Extension, info, ApplyServices | 5, 6 |

| File | Content | Ch. |
|---|---|---|
| `Infrastructure/TenancySettings.cs` | Settings as a record | 5 |
| `Infrastructure/ITenancySingletonOptions.cs` | Singleton options | 6 |
| `Infrastructure/TenancyOptionsBuilder.cs` | Options builder | 7 |
| `Infrastructure/TenancyDbContextOptionsBuilderExtensions.cs` | UseAcmeTenancy | 7 |
| `Infrastructure/TenancyIdentifiers.cs` | Identifier validation | 10, 22 |
| `Storage/*` | Type mapping and plugin | 12 |
| `Query/*` | Translators and translator plugins | 14 |
| `Metadata/*` | Annotations, RlsPolicy, PolicyCommand, annotation provider | 9, 21 |
| `Builders/*` | Fluent API, PolicyBuilder | 9 |
| `Conventions/*` | Isolation, attribute, validation, plugin | 10, 11 |
| `Migrations/*` | SQL generator, custom operations | 21, 22 |
| `Design/*` | Code generator, design time | 23 |
| `Runtime/TenantConnectionInterceptor.cs` | Set the session setting | 18 |
| `Runtime/PostgreSqlTransactionSessionInterceptor.cs` | Tenant per transaction (transaction pooling) | 36 |
| `Design/Internal/TenancyCSharpRuntime…Generator.cs` | Compiled model filter | 44 |
| `Scaffolding/*` | Read RLS from the database | 45 |
| **SQL Server package** | | |
| `Infrastructure/*` | Extension, provider-builder activation | 47 |
| `Storage/*`, `Query/*` | uniqueidentifier, SESSION_CONTEXT | 47 |
| `Conventions/*`, `Metadata/*` | Isolation, column annotation | 47 |
| `Migrations/*` | Predicate function, security policy | 47 |
| `Runtime/*` | sp_set_session_context | 47 |
| **Cosmos DB package** | | |
| `Conventions/*` | Tenant as partition key, convention set plugin | 48 |
| `Storage/*` | TenantId as a string, type mapping plugin | 48 |
| `Infrastructure/*` | Options extension, UseAcmeTenancy | 48 |
| **Functions sample package (no tenancy)** | | |
| `AcmePgDbFunctionsExtensions.cs`, `Infrastructure/*` | EF.Functions methods, options extension, UseAcmeFunctions | 42 |
| `Query/FunctionsTranslatorPlugins.cs`, `Query/FunctionsEvaluatableExpressionFilterPlugin.cs` | Translator plugins, evaluatable filter | 42, 43 |
| `Query/Expressions/*`, `Query/DistinctFromTranslator.cs` | Custom SQL node | 42 |
| `Query/Internal/*` | SQL generator, nullability processor, factories | 42 |
| `Query/EveryAggregateTranslator.cs` | Aggregate translator | 43 |

| File | Content | Ch. |
|---|---|---|
| **Further** | | |
| docs/deployment/* | Role setup, migrations pipeline, diagnosis scripts for the policies (PostgreSQL, SQL Server) | 36, 38 |
| samples/ShopSample | Console sample with design-time factory | 3, 35 |
| samples/ShopApi | ASP.NET Core sample (minimal API) | 40 |
| .github/workflows/compatibility.yml | Weekly against the newest stable 10.0.x patches | 27 |
| .github/workflows/ci.yml | Build, tests, handbook | 41 |
| eng/consumer-test/run.sh | Packages in a fresh app: `dotnet ef`, compiled model, NativeAOT | 24, 27 |
| PublicAPI.*.txt | Public API analyzer | 41 |
| docs/handbook/* | Sources of this book (German and English) | – |
| tests/...Tests | Unit tests without a database | 26 |
| tests/.../MinimalPluginTests.cs | The mini plugin | 2 |
| tests/.../CompositionTests.cs | Decorator example (audit plugin) and composition matrix of two plugins | 6 |
| tests/.../FailureCaseTests.cs | The book’s “What happens if …?” cases, the wrong variant next to the right one | – |
| tests/.../ServiceRegistrationTests.cs | Lifetime and cardinality of every registration against EF’s service table | 27 |
| tests/.../StateAndLifetimeTests.cs | Failure cases for state and lifetimes, the wrong variant next to the right one | 8 |
| tests/.../MaterializationTests.cs | Check while loading: registration, logic, failure case per context (unit), leak, limits and remembered tenant (integration) | 17 |
| tests/.../DesignTimeServicesTests.cs | Lifetimes of the replaced design-time services | 23, 27 |
| tests/.../DebuggingTests.cs | Debugging techniques | App. D |
| tests/.../SchemaPerTenantTests.cs | Model cache key per schema | 34 |
| benchmarks/... | BenchmarkDotNet project for the SaveChanges logic | 39 |
| tests/...IntegrationTests | Testcontainers tests against PostgreSQL | 18, 26, 30, 32, 36, 45 |
| tests/...SqlServer.Tests | Unit and Testcontainers tests for SQL Server | 26, 36, 47 |
| tests/...Cosmos.Tests | Unit tests and tests against the Cosmos DB emulator | 48 |

# C Glossary

| Term | Explanation |
|---|---|
| Annotation | Key-value metadata on a model element. |
| ConfigurationSource | Origin of a piece of metadata (Explicit, DataAnnotation, Convention); determines who may override whom. |
| Convention | Rule that adapts the model automatically while it is built. |
| Decorator | Service that wraps the previously registered implementation and delegates to it; composes with other plugins (chapter 6). |
| Design-time model | Complete model with all annotations; basis for migrations. Counterpart: the lean runtime model. |
| Differ | `IMigrationsModelDiffer`; compares two relational models. |
| Funcletizer | First query stage; evaluates expressions without data reference in .NET. |
| Identity resolution | The same entity yields the same instance: with tracking through the change tracker, otherwise optionally per query. |
| Materializer | Part of the shaper that creates an entity: constructor, then properties and fields. |
| Nullability processor | Query stage that evaluates NULL semantics and optimizes comparisons; must know custom SQL nodes. |
| Shaper | Compiled expression tree per query shape that builds the result from the rows of the `DbDataReader`. |
| SqlExpression | Node in the SQL expression tree; custom nodes for syntax that is not a function. |
| Runtime annotation | Annotation read at run time, which must therefore survive in the runtime and compiled model. |
| Runtime annotation (EF API) | Narrower meaning in EF's API: a value that code stores on the finished model only (`GetOrAddRuntimeAnnotationValue`), for example as a cache; it never appears in the snapshot or in migrations. |
| Internal service provider | EF's own DI container, one per provider key of the options extensions (chapter 5). |
| Pubternal | Publicly visible but internal API without compatibility guarantees (EF1001). |
| Relational model | Tables, columns, indexes, derived from the EF model. |
| RLS | Row Level Security: row filters enforced by the database itself (PostgreSQL, SQL Server). |
| Snapshot | Generated C# code with the model state of the last migration. |
| Type mapping | Connection between CLR type and database type including conversion, comparison and literals. |

# D Debugging toolbox

When a plugin does not behave as expected, the question is almost always one of six. Each technique below is checked in `DebuggingTests`.

| Question | Technique |
|---|---|
| Is my annotation in the model? | `ToDebugString(MetadataDebugStringOptions.LongDefault)` on the design-time model |
| Which implementation of a single service won? | `context.GetService<IMigrationsSqlGenerator>().GetType()` |
| Is my convention registered? | `GetService<IConventionSetBuilder>().CreateConventionSet()`, then e.g. `ModelFinalizingConventions` |
| What does the query look like before and after translation? | `query.Expression.Print()` and `query.ToQueryString()` |
| At which stage of the pipeline does something change? | `LogTo` at debug level with `CoreEventId.QueryCompilationStarting` (tree after the funcletizer) and `QueryExecutionPlanned` (`Select Expression` and shaper), chapter 13 |
| Does my plugin log? | `LogTo(lines.Add, [eventId], LogLevel.Debug)`, restricted to the plugin's event IDs |

Debugging snippets (simplified, checked in tests/.../DebuggingTests.cs)

```
using var ctx = CreateContext();

// 1. Model: annotations, results of conventions, type mappings
var model = ctx.GetService<IDesignTimeModel>().Model;
Console.WriteLine(model.ToDebugString(MetadataDebugStringOptions.LongDefault));

// 2. Services: which replacement won?
Console.WriteLine(ctx.GetService<IMigrationsSqlGenerator>().GetType());

// 3. Conventions: is mine in the list?
var conventions = ctx.GetService<IConventionSetBuilder>().CreateConventionSet();
Console.WriteLine(string.Join("\n", conventions.ModelFinalizingConventions));

// 4. Queries: LINQ tree before, SQL after translation
var query = ctx.Orders.Where(o => o.TenantId == EF.Functions.CurrentTenant());
Console.WriteLine(query.Expression.Print());
Console.WriteLine(query.ToQueryString());
```

## More tools

| Tool | Use |
|---|---|
| Debugger with Source Link, "Just My Code" off | Step into EF Core and Npgsql; both ship symbols and Source Link. A breakpoint in your own convention shows when and how often it runs. |
| `DebugView` in the debugger | Model, entity types, query objects and the change tracker show the same texts as `ToDebugString`. |
| `dotnet ef ... --verbose` | Shows which assemblies and design-time services the tools load; the first stop when a design-time service seems to be ignored. |

| Tool | Use |
|---|---|
| `EnableSensitiveDataLogging`, `EnableDetailedErrors` | Parameter values and detailed errors in the log; for development only. |
| `ManyServiceProvidersCreatedWarning` | EF throws by default when more than 20 internal providers are created: almost always a hash mistake (chapter 5) or a singleton instance per context (chapter 17). |
| EF Core's source code on GitHub | The authoritative documentation for internal APIs; the folder structure matches the namespaces. |

> **Reproduce in a test, not in the app**
>
> Almost every plugin problem can be reproduced without a database: build a context with the plugin and look at the model, the resolved services or `ToQueryString()`. Such a test iterates faster than the application and remains as a regression test afterwards.

# E Exercises with solutions

Each exercise asks a question that the text answers and names a test of the sample that records the answer. To check a solution, change the sample as the exercise describes and watch which test fails.

## Exercise 1: Convention versus explicit configuration

A user configures a policy for `Order` with the name of the plugin's policy: `HasPolicy("tenant_isolation", p => p.Using("true"))`. The isolation convention sets the same annotation at `ModelFinalizing`. Which configuration applies? What changes if the convention sets the annotation with `SetAnnotation` directly on the entity type instead of through the builder?

> **Solution**
>
> The explicit configuration applies. The convention builder sets annotations with `ConfigurationSource.Convention` and leaves an explicit configuration in place; `HasAnnotation` then returns `null` (chapter 10). `SetAnnotation` on the type knows no configuration source and would silently overwrite the user's policy. Check: `ConventionTests.Explicit_policy_with_same_name_wins_over_convention`.

## Exercise 2: The tenant in the cache key

A plugin includes the current tenant in `GetServiceProviderHashCode`, so that every tenant gets "its" model. What happens with 1000 tenants?

> **Solution**
>
> Every tenant gets an internal service provider of its own with a model of its own. From the 21st provider on, EF throws the `ManyServiceProvidersCreatedWarning` as an error by default, and every provider keeps model, query cache and services in memory. The tenant is per-request state and belongs in the interceptor (chapter 8); only when the *model* really differs does a model cache key of its own help (chapter 34). Check: `StateAndLifetimeTests.Tenant_in_the_service_provider_key_needs_a_provider_per_tenant`.

## Exercise 3: Nullability of a translator

Without a tenant, `CurrentTenant()` becomes `NULL` in SQL. Someone therefore declares the expression nullable. What does `o.TenantId != EF.Functions.CurrentTenant()` return without a tenant, for example under the reporting role without RLS?

> **Solution**
>
> For a nullable expression EF adds C# semantics: `<>` becomes a comparison with `OR ... IS NULL`, and without a tenant the condition is true for *all* rows. Declared not nullable, it stays a plain `<>`, which is never true with `NULL`: no rows, the safe direction (chapter 14). Check: `QueryTranslationTests.Inequality_with_CurrentTenant_never_matches_without_a_tenant`.

## Exercise 4: A migration diff on SQL Server

The tenant column is renamed with `HasColumnName("Mandant")`. In which order does EF generate the operations, and why is it not enough to recreate the security policy at the `AlterTableOperation`?

> **Solution**
>
> EF generates `RenameColumn`, `RenameIndex` and only then the `AlterTableOperation` with the changed annotation. The schema-bound policy, however, already blocks the `sp_rename`. The generator has to wrap the whole operation list (page 188). Check: `SqlServerMigrationOrderTests` without a database, `SqlServerMigrationTests` against SQL Server.

## Exercise 5: An update from the API

An API receives an order and saves it with `Update`. Why does EF mark the tenant property as modified, and what stops an attacker from sending the ID of a foreign order with the own tenant?

> **Solution**
>
> `Update` of an untracked entity marks every property; original and current value are the same. That is no change of tenant; the interceptor checks the value only (page 84). Against the foreign ID it does not help, because without accessing the database it cannot know whom the row belongs to. The `UPDATE` does not see the row because of the policy, changes zero rows, and EF reports a concurrency error. Check: `DetachedUpdateTests` against PostgreSQL.

Every check can be run with a filter, for example:

Shell

```
dotnet test tests/Acme.EntityFrameworkCore.Tenancy.Tests \
    --filter "FullyQualifiedName~Inequality_with"
```

# F Sources

The selection is limited to primary sources: the official documentation of EF Core, Npgsql, PostgreSQL, SQL Server and .NET, and the source code of exactly the versions the book refers to. Implementation details are therefore bound to tags (`v10.0.0`), not to branches that move on. All addresses were reachable at the editorial deadline.

| Source | Ch. |
|---|---|
| **EF Core: releases and extension points** | |
| EF Core releases and planning<br>`https://learn.microsoft.com/en-us/ef/core/what-is-new/` | 27 |
| Breaking changes in EF Core 10<br>`https://learn.microsoft.com/en-us/ef/core/what-is-new/ef-core-10.0/breaking-changes` | 27 |
| Writing a database provider<br>`https://learn.microsoft.com/en-us/ef/core/providers/writing-a-provider` | 6 |
| Model bulk configuration (conventions)<br>`https://learn.microsoft.com/en-us/ef/core/modeling/bulk-configuration` | 10 |
| Value conversions<br>`https://learn.microsoft.com/en-us/ef/core/modeling/value-conversions` | 12 |
| Value comparers<br>`https://learn.microsoft.com/en-us/ef/core/modeling/value-comparers` | 24 |
| Global query filters<br>`https://learn.microsoft.com/en-us/ef/core/querying/filters` | 10 |
| Interceptors<br>`https://learn.microsoft.com/en-us/ef/core/logging-events-diagnostics/interceptors` | 18 |
| Logging, events and diagnostics<br>`https://learn.microsoft.com/en-us/ef/core/logging-events-diagnostics/` | 19 |
| **EF Core: model, migrations, operations** | |
| Inheritance (TPH, TPT, TPC)<br>`https://learn.microsoft.com/en-us/ef/core/modeling/inheritance` | 10 |
| Owned entity types<br>`https://learn.microsoft.com/en-us/ef/core/modeling/owned-entities` | 10 |
| Table and entity splitting<br>`https://learn.microsoft.com/en-us/ef/core/modeling/table-splitting` | 10 |
| Custom migrations operations<br>`https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/operations` | 22 |
| Design-time services<br>`https://learn.microsoft.com/en-us/ef/core/cli/services` | 23 |
| Connection resiliency<br>`https://learn.microsoft.com/en-us/ef/core/miscellaneous/connection-resiliency` | 38 |

| Source | Ch. |
|---|---|
| Advanced performance topics (compiled models)<br>https://learn.microsoft.com/en-us/ef/core/performance/advanced-performance-topics | 24 |
| NativeAOT and precompiled queries<br>https://learn.microsoft.com/en-us/ef/core/performance/nativeaot-and-precompiled-queries | 24 |
| **Source code of the version used** | |
| EntityFrameworkServicesBuilder (EF Core 10.0.0)<br>https://github.com/dotnet/efcore/blob/v10.0.0/src/EFCore/Infrastructure/EntityFrameworkServicesBuilder.cs | A |
| EntityFrameworkRelationalServicesBuilder (EF Core 10.0.0)<br>https://github.com/dotnet/efcore/blob/v10.0.0/src/EFCore.Relational/Infrastructure/EntityFrameworkRelationalServicesBuilder.cs | A |
| ConstructorBindingFactory (EF Core 10.0.0)<br>https://github.com/dotnet/efcore/blob/v10.0.0/src/EFCore/Metadata/Internal/ConstructorBindingFactory.cs | 17 |
| Npgsql EF Core provider 10.0.0<br>https://github.com/npgsql/efcore.pg/tree/v10.0.0 | 6 |
| **Npgsql and PostgreSQL** | |
| Npgsql Entity Framework Core provider<br>https://www.npgsql.org/efcore/ | 3 |
| Npgsql connection string parameters<br>https://www.npgsql.org/doc/connection-string-parameters.html | 38 |
| PostgreSQL: Row Security Policies<br>https://www.postgresql.org/docs/17/ddl-rowsecurity.html | 36 |
| PostgreSQL: CREATE POLICY<br>https://www.postgresql.org/docs/17/sql-createpolicy.html | 21 |
| PostgreSQL: Function Volatility Categories<br>https://www.postgresql.org/docs/17/xfunc-volatility.html | 36 |
| PostgreSQL: System Administration Functions (set_config)<br>https://www.postgresql.org/docs/17/functions-admin.html | 18 |
| **SQL Server** | |
| SQL Server: Row-Level Security<br>https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security | 47 |
| CREATE SECURITY POLICY<br>https://learn.microsoft.com/en-us/sql/t-sql/statements/create-security-policy-transact-sql | 47 |
| sp_set_session_context<br>https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-set-session-context-transact-sql | 47 |
| **.NET and NuGet** | |
| Native AOT deployment<br>https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/ | 24 |

| Source | Ch. |
| --- | --- |
| Package source mapping<br>`https://learn.microsoft.com/en-us/nuget/consume-packages/package-source-mapping` | 24 |
| Auditing package dependencies<br>`https://learn.microsoft.com/en-us/nuget/concepts/auditing-packages` | 27 |
| Public API analyzers<br>`https://github.com/dotnet/roslyn-analyzers/blob/main/src/PublicApiAnalyzers/PublicApiAnalyzers.Help.md` | 41 |

# License

This handbook and the sample code in the repository are licensed under the MIT License. You may use, copy, modify and distribute them, also commercially, provided that the following copyright and license notice is included.

Copyright (c) 2026 guenni81

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

The DejaVu fonts embedded in this PDF are under their own free license (Bitstream Vera Fonts license; the DejaVu changes are in the public domain); see `docs/handbook/fonts/LICENSE-DejaVu.txt` in the repository.

Product and company names mentioned in this book are trademarks of their respective owners, for example Entity Framework, .NET and SQL Server (Microsoft) and PostgreSQL (PostgreSQL Community Association). They are used for descriptive purposes only. This handbook is an independent project and is not affiliated with or endorsed by Microsoft, the PostgreSQL Global Development Group or the Npgsql project.